

Analiza Evenimentelor de Fotbal cu Apache Spark și TensorFlow

Student	Măgureanu Stefan-Ionut
Grupă	505
Materie	Big Data
Program de studii	Master — Baze de Date și Tehnologii Software
An de studiu	Anul II
An universitar	2025–2026
Dataset	Football Events — Kaggle

Cuprins

- 1. Introducere** — prezentarea setului de date și obiectivele proiectului
 - 1.1 Prezentarea setului de date
 - 1.2 Obiectivele proiectului
 - 1.3 Previzualizare rapidă a datelor
- 2. Cerința 2** — Procesarea datelor cu Spark SQL și DataFrame API
- 3. Cerința 3** — Metode Machine Learning cu Spark MLlib (clasificare + regresie)
- 4. Cerința 4** — Data Pipeline (ETL + Pipeline MLlib serializat)
- 5. Cerința 5** — UDF și optimizarea hiperparametrilor (CrossValidator)
- 6. Cerința 6** — Deep Learning cu TensorFlow
- 7. Cerința 7** — Spark Structured Streaming + inferență în timp real

1. Introducere

1.1 Prezentarea setului de date

Setul de date utilizat este **Football Events**, disponibil pe platforma Kaggle la adresa:

<https://www.kaggle.com/datasets/secareanualin/football-events>

Acesta conține date granulare despre evenimentele din meciuri de fotbal din **cinci ligi europene majore** (Bundesliga, Premier League, La Liga, Serie A, Ligue 1) pe parcursul **sezonelor 2011–2017**.

Setul este compus din **două fișiere CSV**:

events.csv — 941.009 înregistrări

Fiecare rând reprezintă un eveniment dintr-un meci:

Coloană	Descriere
id_odsp	Identificatorul unic al meciului
id_event	Identificatorul unic al evenimentului
time	Minutul în care s-a produs evenimentul
event_type	Tipul evenimentului (0=Anunț, 1=Tentativă șut, 2=Corner, 3=Fault, 4=Cartonaș galben, etc.)
event_team	Echipa care a produs evenimentul
player	Jucătorul implicat
is_goal	1 dacă evenimentul a dus la gol, 0 altfel
location	Zona de pe teren (1–19)
bodypart	Partea corpului folosită (1=picior drept, 2=picior stâng, 3=cap)
situation	Situația de joc (1=joc deschis, 2=fază fixă, 3=corner, 4=lovitură liberă)
shot_place	Locul în poartă unde a mers șutul (1–13)
assist_method	Metoda de assist (0=fără, 1=pasă, 2=centrare, 3=pasă cu capul, 4=pasă în adâncime)
fast_break	1 dacă evenimentul a venit dintr-un contraatac rapid

ginf.csv — 10.112 înregistrări

Fiecare rând reprezintă un meci:

Coloană	Descriere
id_odsp	Identificatorul unic al meciului
date	Data meciului

Coloană	Descriere
league	Liga (D1=Bundesliga, E0=Premier League, SP1=La Liga, I1=Serie A, F1=Ligue 1)
season	Sezonul
ht / at	Echipa gazdă / oaspete
fthg / ftag	Goluri marcate acasă / în deplasare (Full Time)
odd_h / odd_d / odd_a	Cotele de pariuri pentru victorie gazdă / egalitate / victorie oaspete

1.2 Obiectivele proiectului

Proiectul urmărește aplicarea tehnicilor de **Big Data** studiate la curs pe un dataset real de fotbal, cu scopul de a extrage insight-uri relevante și de a construi modele predictive. Obiectivele concrete sunt:

O1 — Analiză exploratorie distribuită (Cerința 2)

Utilizarea Apache Spark (DataFrame API și Spark SQL) pentru curățarea, transformarea și analiza statistică a celor ~950.000 de evenimente, identificând pattern-uri în distribuția golurilor, eficiența șuturilor și performanța echipelor.

O2 — Modele de Machine Learning cu Spark MLlib (Cerința 3)

- **Clasificare:** Predicția dacă o tentativă de șut va fi gol, folosind Random Forest Classifier pe baza caracteristicilor șutului (locație, parte corp, situație).
- **Regresie:** Predicția numărului total de goluri dintr-un meci pe baza statisticilor agregate (corneruri, fault-uri, șuturi pe poartă).

O3 — Data Pipeline reproductibil (Cerința 4)

Construirea unui pipeline ETL complet care automatizează preprocesarea (encoding, normalizare, asamblare features) și poate fi aplicat consistent pe date noi.

O4 — UDF și tuning avansat (Cerința 5)

Definirea unei funcții personalizate pentru calculul unui „scor de calitate al șutului” și optimizarea hiperparametrilor modelului prin Cross-Validation.

O5 — Deep Learning cu TensorFlow (Cerința 6)

Construirea și antrenarea unei rețele neuronale feed-forward pentru clasificarea binară a șuturilor, cu compararea performanței față de modelul MLlib.

O6 — Inferență în timp real (Cerința 7)

Simularea unui flux de date de la un meci live și aplicarea modelului antrenat pentru predicții în timp real cu Spark Structured Streaming.

1.3 Previzualizare rapidă a datelor

```
import os
# Notebook-urile stau în subfolderul notebooks/; ne asigurăm că
# lucrăm din rădăcina proiectului
# (unde se află data/, models/, plots/), astfel încât toate căile
# relative să funcționeze.
if not os.path.isdir('data') and os.path.isdir(os.path.join '..',
'data')):
    os.chdir('..')
# Directoarele de output trebuie să existe înainte de orice
savefig()/save()
os.makedirs('plots', exist_ok=True)
os.makedirs('models', exist_ok=True)
print('Working directory:', os.getcwd())
Working directory: /Users/stefan/Documents/football-events-analysis
import pandas as pd

events_preview = pd.read_csv('data/events.csv', nrows=5)
print(f'Coloane events.csv ({len(events_preview.columns)}):
{list(events_preview.columns)}')
events_preview
Coloane events.csv (22): ['id_odsp', 'id_event', 'sort_order',
'time', 'text', 'event_type', 'event_type2', 'side', 'event_team',
'opponent', 'player', 'player2', 'player_in', 'player_out',
'shot_place', 'shot_outcome', 'is_goal', 'location', 'bodypart',
'assist_method', 'situation', 'fast_break']
   id_odsp  id_event  sort_order  time \
0  UFot0hit/  UFot0hit1         1     2
1  UFot0hit/  UFot0hit2         2     4
2  UFot0hit/  UFot0hit3         3     4
3  UFot0hit/  UFot0hit4         4     7
4  UFot0hit/  UFot0hit5         5     7

                                text  event_type
event_type2 \
0  Attempt missed. Mladen Petric (Hamburg) left f...      1
12.0
1  Corner, Borussia Dortmund. Conceded by Dennis...      2
NaN
2  Corner, Borussia Dortmund. Conceded by Heiko ...      2
NaN
3              Foul by Sven Bender (Borussia Dortmund).      3
NaN
4  Gokhan Tore (Hamburg) wins a free kick in the ...      8
NaN

   side  event_team  opponent  ... player_in
player_out \
```

0	2	Hamburg SV	Borussia Dortmund	...	NaN
NaN					
1	1	Borussia Dortmund	Hamburg SV	...	NaN
NaN					
2	1	Borussia Dortmund	Hamburg SV	...	NaN
NaN					
3	1	Borussia Dortmund	Hamburg SV	...	NaN
NaN					
4	2	Hamburg SV	Borussia Dortmund	...	NaN
NaN					

	shot_place	shot_outcome	is_goal	location	bodypart
assist_method	\				
0	6.0	2.0	0	9.0	2.0
1					
1	NaN	NaN	0	NaN	NaN
0					
2	NaN	NaN	0	NaN	NaN
0					
3	NaN	NaN	0	NaN	NaN
0					
4	NaN	NaN	0	2.0	NaN
0					

	situation	fast_break
0	1.0	0
1	NaN	0
2	NaN	0
3	NaN	0
4	NaN	0

```
[5 rows x 22 columns]
ginf_preview = pd.read_csv('data/ginif.csv', nrows=5)
print(f'Coloane ginif.csv ({len(ginf_preview.columns)}):
{list(ginf_preview.columns)}')
ginf_preview
Coloane ginif.csv (18): ['id_odsp', 'link_odsp', 'adv_stats', 'date',
'league', 'season', 'country', 'ht', 'at', 'fthg', 'ftag', 'odd_h',
'odd_d', 'odd_a', 'odd_over', 'odd_under', 'odd_bts', 'odd_bts_n']
id_odsp link_odsp
adv_stats \
0 UFot0hit/ /soccer/germany/bundesliga-2011-2012/dortmund-...
True
1 Aw5DfllH/ /soccer/germany/bundesliga-2011-2012/augsburg-...
True
2 bkjpaC6n/ /soccer/germany/bundesliga-2011-2012/werder-br...
True
3 CzPV312a/ /soccer/france/ligue-1-2011-2012/paris-sg-lori...
True
4 GU0dmtII/ /soccer/france/ligue-1-2011-2012/caen-valencie...
True
```

	date	league	season	country	ht
at	\				

0	2011-08-05	D1	2012	germany	Borussia Dortmund	
	Hamburg SV					
1	2011-08-06	D1	2012	germany	FC Augsburg	SC
	Freiburg					
2	2011-08-06	D1	2012	germany	Werder Bremen	
	Kaiserslautern					
3	2011-08-06	F1	2012	france	Paris Saint-Germain	
	Lorient					
4	2011-08-06	F1	2012	france	Caen	
	Valenciennes					

	fthg	ftag	odd_h	odd_d	odd_a	odd_over	odd_under	odd_bts
0	3	1	1.56	4.41	7.42	NaN	NaN	NaN
1	2	2	2.36	3.60	3.40	NaN	NaN	NaN
2	2	0	1.83	4.20	4.80	NaN	NaN	NaN
3	0	1	1.55	4.50	9.40	NaN	NaN	NaN
4	1	0	2.50	3.40	3.45	NaN	NaN	NaN

```

import subprocess

events_lines = int(subprocess.check_output(['wc', '-l',
'data/events.csv']).split()[0]) - 1
ginf_lines    = int(subprocess.check_output(['wc', '-l',
'data/ginfcsv']).split()[0]) - 1

print(f'events.csv : {events_lines:,} înregistrări')
print(f'ginfcsv   : {ginf_lines:,} meciuri')
print(f'Total      : {events_lines + ginf_lines:,} rânduri')
events.csv : 941,009 înregistrări
ginfcsv    : 10,112 meciuri
Total      : 951,121 rânduri

```

Cerința 2 — Procesarea datelor cu Spark SQL și DataFrame API

Această secțiune demonstrează utilizarea **Apache Spark** pentru procesarea, curățarea și analiza distribuită a setului de date Football Events. Vor fi folosite **ambele** interfețe Spark:

- **DataFrame API** — operații programatice (filter, groupBy, join, withColumn)
- **Spark SQL** — interogări SQL pe temp views

2.1 Inițializarea sesiunii Spark și încărcarea datelor

Creăm o sesiune Spark în modul local și citim cele două fișiere CSV cu inferarea automată a schemei.

Configurarea sesiunii Spark

Creăm `SparkSession` — punctul de intrare unic către toate API-urile Spark (SQL, `DataFrame`, `MLlib`). Două configurări contează pe o mașină locală:

`spark.driver.memory=4g` (datele și rezultatele agregărilor încap în memoria driver-ului) și `spark.sql.shuffle.partitions=8` (default-ul de 200 de partiții la shuffle e gândit pentru clustere mari).

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, count, sum as spark_sum, avg, max as spark_max, min as
    spark_min,
    round as spark_round, when, isnan, isnull, desc, asc,
    countDistinct, lit, concat_ws, year, month
)
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.figsize'] = (12, 5)
matplotlib.rcParams['axes.spines.top'] = False
matplotlib.rcParams['axes.spines.right'] = False
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder \
    .appName('FootballAnalysis_EDA') \
    .config('spark.driver.memory', '4g') \
    .config('spark.sql.shuffle.partitions', '8') \
    .getOrCreate()

spark.sparkContext.setLogLevel('ERROR')
print('Spark version:', spark.version)
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
26/06/11 20:10:34 WARN NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
Spark version: 4.1.2
```

Citirea datelor cu schema inferată

Citim ambele CSV-uri cu `inferSchema=True` (Spark face o trecere suplimentară peste fișier ca să deducă tipurile) și `nullValue='NA'` — dataset-ul folosește string-ul `'NA'` pentru valori lipsă.

```
events_df = spark.read.csv('data/events.csv', header=True,
inferSchema=True, nullValue='NA')
ginf_df    = spark.read.csv('data/ginf.csv',    header=True,
inferSchema=True, nullValue='NA')

print(f'events_df: {events_df.count():,} rînduri,
{len(events_df.columns)} coloane')
print(f'ginf_df   : {ginf_df.count():,} rînduri,
{len(ginf_df.columns)} coloane')
[Stage 1:>
(0 + 10) / 10]
events_df: 941,009 rînduri, 22 coloane
ginf_df   : 10,112 rînduri, 18 coloane
print('=== Schema events.csv ===')
events_df.printSchema()
print('\n=== Schema ginf.csv ===')
ginf_df.printSchema()
=== Schema events.csv ===
root
|-- id_odsp: string (nullable = true)
|-- id_event: string (nullable = true)
|-- sort_order: integer (nullable = true)
|-- time: integer (nullable = true)
|-- text: string (nullable = true)
|-- event_type: integer (nullable = true)
|-- event_type2: integer (nullable = true)
|-- side: integer (nullable = true)
|-- event_team: string (nullable = true)
|-- opponent: string (nullable = true)
|-- player: string (nullable = true)
|-- player2: string (nullable = true)
|-- player_in: string (nullable = true)
|-- player_out: string (nullable = true)
|-- shot_place: integer (nullable = true)
|-- shot_outcome: integer (nullable = true)
|-- is_goal: integer (nullable = true)
|-- location: integer (nullable = true)
|-- bodypart: integer (nullable = true)
|-- assist_method: integer (nullable = true)
|-- situation: integer (nullable = true)
|-- fast_break: integer (nullable = true)

=== Schema ginf.csv ===
root
|-- id_odsp: string (nullable = true)
|-- link_odsp: string (nullable = true)
```



```

|-- adv_stats: boolean (nullable = true)
|-- date: date (nullable = true)
|-- league: string (nullable = true)
|-- season: integer (nullable = true)
|-- country: string (nullable = true)
|-- ht: string (nullable = true)
|-- at: string (nullable = true)
|-- fthg: integer (nullable = true)
|-- ftag: integer (nullable = true)
|-- odd_h: double (nullable = true)
|-- odd_d: double (nullable = true)
|-- odd_a: double (nullable = true)
|-- odd_over: double (nullable = true)
|-- odd_under: double (nullable = true)
|-- odd_bts: double (nullable = true)
|-- odd_bts_n: double (nullable = true)

```

2.2 Curățarea și inspecția datelor (DataFrame API)

Verificăm valorile lipsă pe coloanele esențiale și aplicăm filtrele necesare pentru a obține un dataset curat pentru analiză.

```

# Numărul de valori null per coloană în events_df
key_cols = ['event_type', 'is_goal', 'location', 'bodypart',
            'situation',
            'shot_place', 'assist_method', 'fast_break', 'time',
            'event_team']

null_counts = events_df.select([
    spark_sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in key_cols
])
print('Valori null pe coloanele cheie:')
null_counts.show()
Valori null pe coloanele cheie:
+-----+-----+-----+-----+-----+-----+-----+-----+
|event_type|is_goal|location|bodypart|situation|shot_place|assist_method|fast_break|time|event_team|
+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|473942|711824|711872|713550|0|0|0|
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Filtrarea șuturilor — subsetul central al proiectului

Din aproximativ 941K evenimente reținem doar tentativele de șut (`event_type == 1`) cu atributele esențiale non-null, iar coloanele opționale (`assist_method` , `fast_break` , `shot_place`) primesc 0 în loc de null. Acest `shots_df` este

baza tuturor analizelor și modelelor din proiect — `is_goal` e target-ul de predicție.

```
# Filtrăm doar tentativele de șut (event_type = 1) cu coloane
esențiale non-null
shots_df = events_df.filter(
    (col('event_type') == 1) &
    col('is_goal').isNotNull() &
    col('location').isNotNull() &
    col('bodypart').isNotNull() &
    col('situation').isNotNull()
)

# Înlocuim valorile null din coloanele opționale cu 0
shots_df = shots_df.fillna({
    'assist_method': 0,
    'fast_break': 0,
    'shot_place': 0
})

print(f'Tentative de șut (curățate): {shots_df.count():,}')
print(f'Din care goluri: {shots_df.filter(col("is_goal") ==
1).count():,}')
print(f'Rata de conversie generală:
{shots_df.filter(col("is_goal")==1).count()/shots_df.count()*100:.2f
}%')
Tentative de șut (curățate): 229,135
Din care goluri: 24,441
Rata de conversie generală: 10.67%
```

2.3 Distribuția tipurilor de evenimente — Spark SQL

Înregistrăm DataFrame-ul ca view temporar și interogăm cu SQL pentru a vedea câte evenimente de fiecare tip există.

```
events_df.createOrReplaceTempView('events')
ginf_df.createOrReplaceTempView('matches')
shots_df.createOrReplaceTempView('shots')

event_type_map = {
    0: 'Anunț', 1: 'Tentativă șut', 2: 'Corner', 3: 'Fault',
    4: 'Cartonaș galben', 5: 'Al 2-lea galben', 6: 'Cartonaș roșu',
    7: 'Substituire', 8: 'Lovitură liberă câștigată', 9: 'Ofsaid',
    10: 'Henț', 11: 'Penalti acordat'
}

event_dist = spark.sql("""
    SELECT event_type,
           COUNT(*) AS total,
           ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM events),
2) AS pct
```

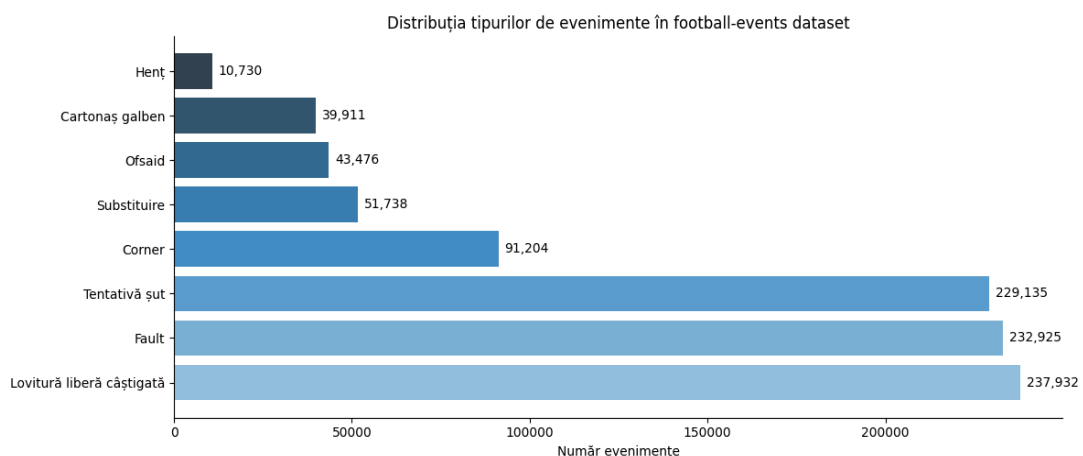
```

FROM events
GROUP BY event_type
ORDER BY total DESC
""")

event_dist_pd = event_dist.toPandas()
event_dist_pd['denumire'] =
event_dist_pd['event_type'].map(event_type_map).fillna('Altul')
print(event_dist_pd[['event_type', 'denumire', 'total',
'pct']].to_string(index=False))
event_type      denumire  total   pct
8 Lovitură liberă câștigată 237932 25.28
3          Fault 232925 24.75
1      Tentativă șut 229135 24.35
2          Corner  91204  9.69
7      Substituire  51738  5.50
9          Ofsaid  43476  4.62
4      Cartonaș galben  39911  4.24
10         Henț  10730  1.14
11      Penalți acordat   2706  0.29
6      Cartonaș roșu   1152  0.12
5      Al 2-lea galben    100  0.01

fig, ax = plt.subplots(figsize=(12, 5))
top = event_dist_pd.nlargest(8, 'total')
bars = ax.barh(top['denumire'], top['total'],
color=sns.color_palette('Blues_d', 8))
ax.bar_label(bars, fmt='{:,}.0f}', padding=5)
ax.set_xlabel('Număr evenimente')
ax.set_title('Distribuția tipurilor de evenimente în football-events
dataset')
plt.tight_layout()
plt.savefig('plots/fig_event_types.png', dpi=120,
bbox_inches='tight')
plt.show()

```



<Figure size 1200x500 with 1 Axes>

2.4 Top echipe după goluri marcate — DataFrame API

Folosind DataFrame API, calculăm numărul total de goluri marcate de fiecare echipă de-a lungul perioadei analizate.

```
top_scorers = events_df \
    .filter(col('is_goal') == 1) \
    .groupBy('event_team') \
    .agg(
        count('*').alias('goluri'),
        countDistinct('id_odsp').alias('meciuri')
    ) \
    .withColumn('goluri_per_meci', spark_round(col('goluri') /
col('meciuri'), 2)) \
    .orderBy(desc('goluri')) \
    .limit(15)

top_scorers.show(15, truncate=False)
+-----+-----+-----+-----+
|event_team|goluri|meciuri|goluri_per_meci|
+-----+-----+-----+-----+
|Barcelona|586|185|3.17|
|Real Madrid|573|182|3.15|
|Bayern Munich|453|161|2.81|
|Paris Saint-Germain|447|187|2.39|
|Napoli|403|172|2.34|
|Juventus|401|189|2.12|
|Borussia Dortmund|392|155|2.53|
|AS Roma|373|176|2.12|
|Lyon|355|167|2.13|
|Atletico Madrid|347|161|2.16|
|Sevilla|330|166|1.99|
|AC Milan|329|166|1.98|
|Fiorentina|321|157|2.04|
|Valencia|314|160|1.96|
|Internazionale|312|162|1.93|
+-----+-----+-----+-----+

import os
os.makedirs('plots', exist_ok=True)

top_pd = top_scorers.toPandas()

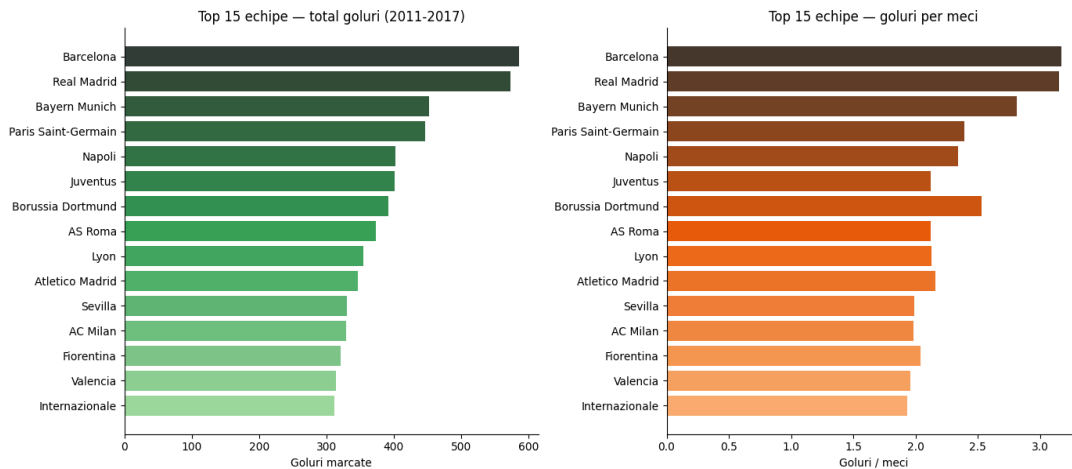
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

axes[0].barh(top_pd['event_team'][:, -1], top_pd['goluri'][:, -1],
              color=sns.color_palette('Greens_d', 15))
axes[0].set_title('Top 15 echipe – total goluri (2011–2017)')
axes[0].set_xlabel('Goluri marcate')

axes[1].barh(top_pd['event_team'][:, -1], top_pd['goluri_per_meci']
```

```
[::-1],
        color=sns.color_palette('Oranges_d', 15))
axes[1].set_title('Top 15 echipe – goluri per meci')
axes[1].set_xlabel('Goluri / meci')

plt.tight_layout()
plt.savefig('plots/fig_top_teams.png', dpi=120, bbox_inches='tight')
plt.show()
```



<Figure size 1400x600 with 2 Axes>

2.5 Rata de conversie a șuturilor pe zonă de teren — Spark SQL

Calculăm câte șuturi și câte goluri provin din fiecare zonă a terenului, obținând rata de conversie (eficiența) pentru fiecare locație.

```
location_map = {
    1: 'Jumătate atac', 2: 'Jumătate apărare', 3: 'Centru careu',
    4: 'Flanc stâng', 5: 'Flanc drept', 6: 'Unghi dificil + distanță lungă',
    7: 'Unghi dificil stânga', 8: 'Unghi dificil dreapta',
    9: 'Stânga careului', 10: 'Stânga 6 yarzi', 11: 'Dreapta careului',
    12: 'Dreapta 6 yarzi', 13: 'Distanță foarte mică',
    14: 'Punctul de penalti', 15: 'În afara careului',
    16: 'Distanță lungă', 17: '>35 yarzi', 18: '>40 yarzi', 19: 'Neînregistrat'
}

conversion_sql = spark.sql("""
    SELECT
        location,
        COUNT(*) AS total_suturi,
        SUM(is_goal) AS goluri,
        ROUND(SUM(is_goal) * 100.0 / COUNT(*), 2) AS rata_conversie
    """)
```

```

FROM shots
WHERE location != 19
GROUP BY location
HAVING COUNT(*) > 100
ORDER BY rata_conversie DESC
""")

conv_pd = conversion_sql.toPandas()
# ROUND() din Spark SQL întoarce decimal.Decimal – cast la float
pentru matplotlib/pandas
conv_pd['rata_conversie'] = conv_pd['rata_conversie'].astype(float)
conv_pd['zona'] = conv_pd['location'].map(location_map)
print(conv_pd[['zona', 'total_suturi', 'goluri',
'rata_conversie']].to_string(index=False))

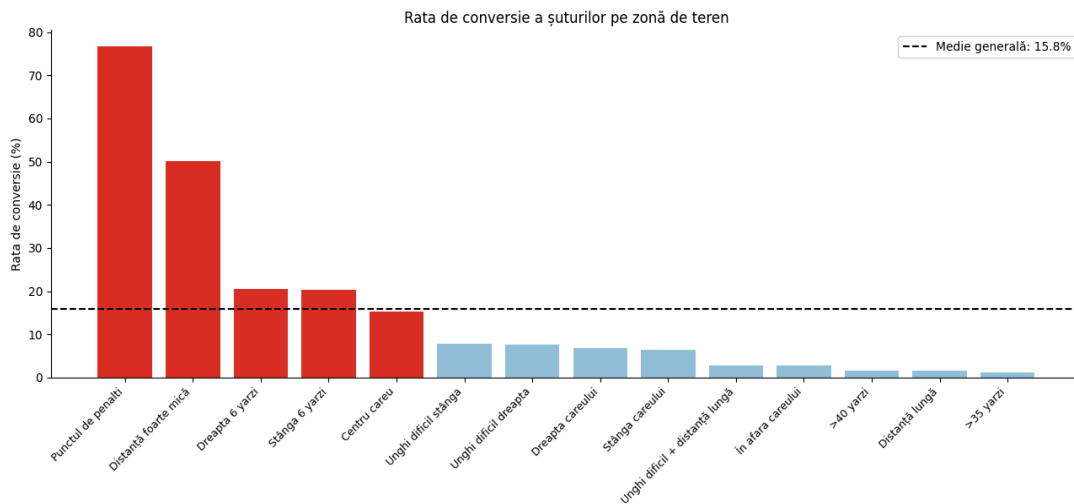
```

	zona	total_suturi	goluri	rata_conversie
	Punctul de penalti	2596	1989	76.62
	Distanță foarte mică	6661	3337	50.10
	Dreapta 6 yarzi	3426	702	20.49
	Stânga 6 yarzi	3492	711	20.36
	Centru careu	69606	10713	15.39
	Unghi dificil stânga	2563	201	7.84
	Unghi dificil dreapta	2458	187	7.61
	Dreapta careului	18441	1273	6.90
	Stânga careului	19033	1234	6.48
	Unghi dificil + distanță lungă	1517	43	2.83
	În afara careului	93667	2555	2.73
	>40 yarzi	256	4	1.56
	Distanță lungă	1991	31	1.56
	>35 yarzi	1978	23	1.16

```

fig, ax = plt.subplots(figsize=(13, 6))
colors = ['#d73027' if r > 15 else '#fee090' if r > 8 else '#91bfdb'
          for r in conv_pd['rata_conversie']]
bars = ax.bar(range(len(conv_pd)), conv_pd['rata_conversie'],
color=colors)
ax.set_xticks(range(len(conv_pd)))
ax.set_xticklabels(conv_pd['zona'], rotation=45, ha='right',
fontsize=9)
ax.set_ylabel('Rata de conversie (%)')
ax.set_title('Rata de conversie a șuturilor pe zonă de teren')
ax.axhline(conv_pd['rata_conversie'].mean(), color='black',
linestyle='--',
label=f'Medie generală:
{conv_pd["rata_conversie"].mean():.1f}%')
ax.legend()
plt.tight_layout()
plt.savefig('plots/fig_conversion_by_zone.png', dpi=120,
bbox_inches='tight')
plt.show()

```



<Figure size 1300x600 with 1 Axes>

2.6 Distribuția golurilor în funcție de minutul meciului

— DataFrame API

Analizăm distribuția temporală a golurilor pe parcursul unui meci, grupând pe intervale de 5 minute.

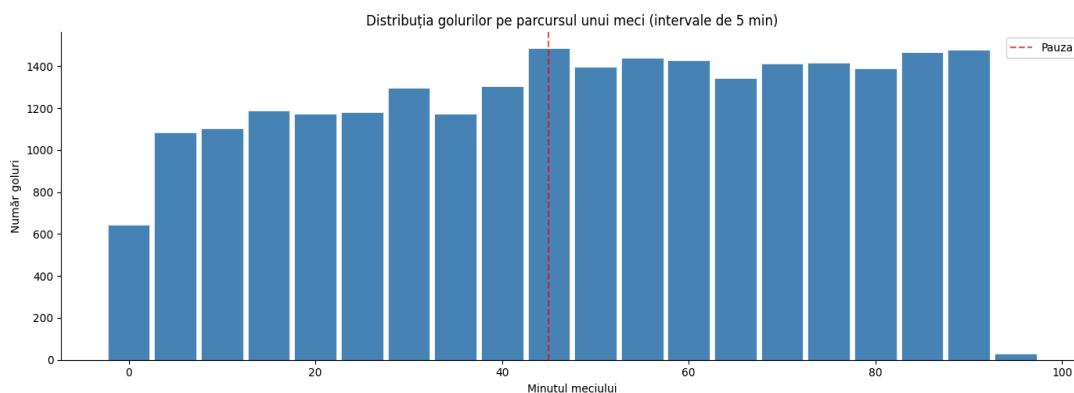
```
from pyspark.sql.functions import floor

goals_by_time = events_df \
    .filter((col('is_goal') == 1) & col('time').isNotNull() &
            (col('time') <= 95)) \
    .withColumn('interval_5min', (floor(col('time') / 5) *
    5).cast('int')) \
    .groupBy('interval_5min') \
    .count() \
    .orderBy('interval_5min')

time_pd = goals_by_time.toPandas()
print('Goluri pe intervale de 5 minute (primele 20 intervale):')
print(time_pd.head(20).to_string(index=False))
Goluri pe intervale de 5 minute (primele 20 intervale):
interval_5min  count
0             645
5             1083
10            1105
15            1190
20            1172
25            1181
30            1296
35            1172
40            1306
45            1488
50            1397
```

55	1440
60	1427
65	1343
70	1412
75	1418
80	1390
85	1468
90	1478
95	28

```
fig, ax = plt.subplots(figsize=(14, 5))
ax.bar(time_pd['interval_5min'], time_pd['count'],
       width=4.5, color='steelblue', edgecolor='white',
       linewidth=0.5)
ax.axvline(45, color='red', linestyle='--', alpha=0.7,
          label='Pauza')
ax.set_xlabel('Minutul meciului')
ax.set_ylabel('Număr goluri')
ax.set_title('Distribuția golurilor pe parcursul unui meci
(intervale de 5 min)')
ax.legend()
plt.tight_layout()
plt.savefig('plots/fig_goals_by_time.png', dpi=120,
          bbox_inches='tight')
plt.show()
```



<Figure size 1400x500 with 1 Axes>

2.7 Statistici per ligă și sezon — Spark SQL

Interogăm tabelul de meciuri pentru a compara intensitatea ofensivă (numărul mediu de goluri per meci) între ligi și sezoane.

```
league_map = {'D1': 'Bundesliga', 'E0': 'Premier League',
              'SP1': 'La Liga', 'I1': 'Serie A', 'F1': 'Ligue 1'}

league_stats = spark.sql("""
    SELECT
        league,
        COUNT(*) AS meciuri,
```



```

        ROUND(AVG(fthg + ftag), 3) AS medie_goluri_meci,
        MAX(fthg + ftag)          AS max_goluri_meci,
        ROUND(AVG(fthg), 3)       AS medie_goluri_acasa,
        ROUND(AVG(ftag), 3)       AS medie_goluri_deplasare,
        SUM(CASE WHEN fthg > ftag THEN 1 ELSE 0 END) AS
victorii_acasa,
        SUM(CASE WHEN fthg = ftag THEN 1 ELSE 0 END) AS egaluri,
        SUM(CASE WHEN fthg < ftag THEN 1 ELSE 0 END) AS
victorii_deplasare
    FROM matches
    GROUP BY league
    ORDER BY medie_goluri_meci DESC
""")

```

```

ls_pd = league_stats.toPandas()
# ROUND() din Spark SQL întoarce decimal.Decimal – cast la float
pentru matplotlib/pandas
for c in ['medie_goluri_meci', 'medie_goluri_acasa',
'medie_goluri_deplasare']:
    ls_pd[c] = ls_pd[c].astype(float)
ls_pd['liga'] = ls_pd['league'].map(league_map)
print(ls_pd[['liga', 'meciuri', 'medie_goluri_meci',
'max_goluri_meci',
'victorii_acasa', 'egaluri',
'victorii_deplasare']].to_string(index=False))

```

	liga	meciuri	medie_goluri_meci	max_goluri_meci	victorii_acasa	egaluri	victorii_deplasare
768	Bundesliga	417	2.882	11	1690	505	2.770
997	La Liga	500	2.742	10	2089	592	2.650
951	Premier League	529	2.507	9	2120	640	2.574
968	Serie A	553			2106	585	
965	Ligue 1	574			2107	568	

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Medie goluri per meci pe ligă
axes[0].bar(ls_pd['liga'], ls_pd['medie_goluri_meci'],
            color=sns.color_palette('Set2', 5))
axes[0].set_title('Medie goluri / meci pe ligă (2011-2017)')
axes[0].set_ylabel('Goluri / meci')
axes[0].set_ylim(2, 3.2)
for i, v in enumerate(ls_pd['medie_goluri_meci']):
    axes[0].text(i, v + 0.02, f'{v:.2f}', ha='center', fontsize=10)

# Distribuția rezultatelor (stacked bar)
bottom_e = ls_pd['victorii_acasa']
bottom_d = ls_pd['victorii_acasa'] + ls_pd['egaluri']
axes[1].bar(ls_pd['liga'], ls_pd['victorii_acasa'], label='Victorie
acasă', color='#2196F3')
axes[1].bar(ls_pd['liga'], ls_pd['egaluri'], bottom=bottom_e,

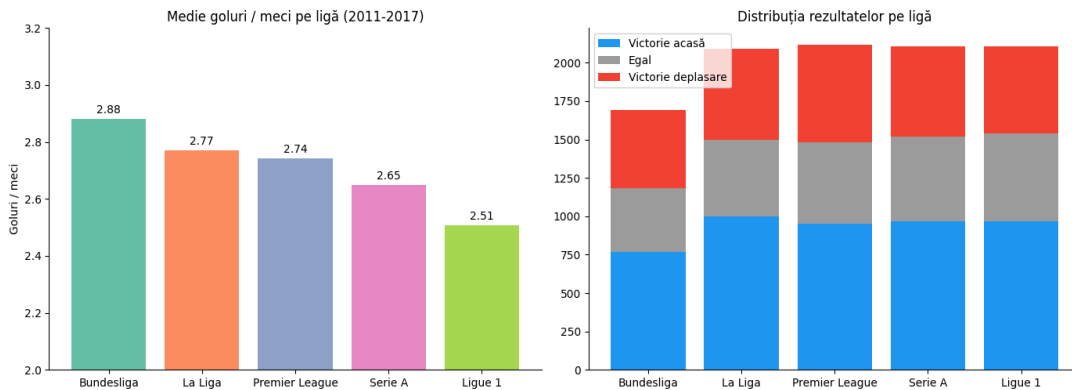
```

```

label='Egal', color='#9E9E9E')
axes[1].bar(ls_pd['liga'], ls_pd['victorii_deplasare'],
bottom=bottom_d,
            label='Victorie deplasare', color='#F44336')
axes[1].set_title('Distribuția rezultatelor pe ligă')
axes[1].legend()

plt.tight_layout()
plt.savefig('plots/fig_league_stats.png', dpi=120,
bbox_inches='tight')
plt.show()

```



<Figure size 1400x500 with 2 Axes>

2.8 Top jucători marcatori — DataFrame API cu join

Realizăm un join între `events_df` și `ginf_df` pe cheia `id_odsp` pentru a obține contextul ligii și sezonului pentru fiecare gol marcat.

```

top_players = events_df \
    .filter((col('is_goal') == 1) & col('player').isNotNull() &
(col('player') != '')) \
    .join(ginf_df.select('id_odsp', 'league', 'season'),
on='id_odsp', how='inner') \
    .groupBy('player', 'league') \
    .agg(
        count('*').alias('goluri'),
        countDistinct('id_odsp').alias('meciuri_cu_gol')
    ) \
    .orderBy(desc('goluri')) \
    .limit(20)

tp_pd = top_players.toPandas()
tp_pd['liga'] = tp_pd['league'].map(league_map)
print(tp_pd[['player', 'liga', 'goluri',
'meciuri_cu_gol']].to_string(index=False))

```

player	liga	goluri	meciuri_cu_gol
lionel messi	La Liga	205	124
cristiano ronaldo	La Liga	198	115

robert lewandowski	Bundesliga	124	83
zlatan ibrahimovic	Ligue 1	111	71
alexandre lacazette	Ligue 1	88	65
karim benzema	La Liga	85	68
gonzalo higuain	Serie A	82	58
antoine griezmann	La Liga	80	61
antonio di natale	Serie A	76	62
sergio aguero	Premier League	75	51
edinson cavani	Ligue 1	72	55
mauro icardi	Serie A	72	54
ruben castro	La Liga	69	59
klaasjan huntelaar	Bundesliga	68	50
luis suarez	La Liga	68	43
stefan kiessling	Bundesliga	67	56
thomas muller	Bundesliga	67	52
pierreemerick aubameyang	Bundesliga	65	48
marco reus	Bundesliga	65	50
harry kane	Premier League	65	46

2.9 Analiza eficienței șuturilor pe tip de acțiune — Spark SQL

Comparăm rata de conversie în funcție de situația de joc (joc deschis, fază fixă, corner, lovitură liberă) și de metoda de assist.

```
situation_map = {1: 'Joc deschis', 2: 'Fază fixă', 3: 'Corner', 4:
'Lovitură liberă'}
bodypart_map = {1: 'Picior drept', 2: 'Picior stâng', 3: 'Cap'}

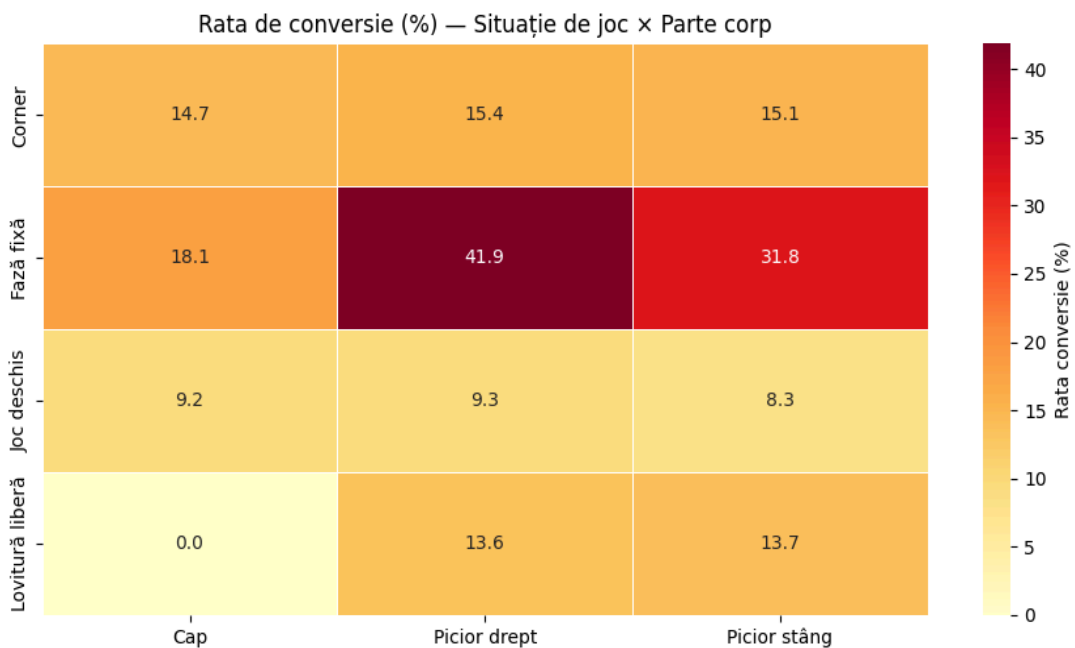
situation_stats = spark.sql("""
    SELECT
        situation,
        bodypart,
        COUNT(*)           AS total_suturi,
        SUM(is_goal)       AS goluri,
        ROUND(SUM(is_goal) * 100.0 / COUNT(*), 2) AS rata_conversie
    FROM shots
    WHERE situation IS NOT NULL AND bodypart IS NOT NULL
    GROUP BY situation, bodypart
    ORDER BY rata_conversie DESC
""")

sit_pd = situation_stats.toPandas()
sit_pd['rata_conversie'] = sit_pd['rata_conversie'].astype(float)
sit_pd['situatie'] = sit_pd['situation'].map(situation_map)
sit_pd['parte_corp'] = sit_pd['bodypart'].map(bodypart_map)

# Pivot pentru heatmap
pivot = sit_pd.pivot_table(
    index='situatie', columns='parte_corp',
    values='rata_conversie', aggfunc='first'
```

```
)

fig, ax = plt.subplots(figsize=(9, 5))
sns.heatmap(pivot, annot=True, fmt='.1f', cmap='YlOrRd',
            linewidths=0.5, ax=ax, cbar_kws={'label': 'Rata
conversie (%)'})
ax.set_title('Rata de conversie (%) – Situație de joc x Parte corp')
ax.set_xlabel('')
ax.set_ylabel('')
plt.tight_layout()
plt.savefig('plots/fig_heatmap_conversion.png', dpi=120,
bbox_inches='tight')
plt.show()
```



<Figure size 900x500 with 2 Axes>

2.10 Expected Goals (xG) estimat din locația șutului – Spark SQL

Ce analizăm: Atribuim fiecărui șut o probabilitate de gol egală cu rata istorică de conversie a zonei din care a fost executat (un model xG simplificat, bazat doar pe locație). Însușind aceste probabilități pe meci obținem *xG-ul* echipei, pe care îl comparăm cu golurile marcate efectiv.

```
# xG (Expected Goals) estimat din rata istorică de conversie pe
locație – Spark SQL
# fiecare locație primește o probabilitate empirică de gol (xG per
șut)
# xG-ul unui meci = suma xG-urilor șuturilor; comparăm cu golurile
reale
```

```

import pandas as pd

xg_analysis = spark.sql("""
    WITH loc_xg AS (
        SELECT location,
               SUM(is_goal) / COUNT(*) AS xg_per_shot
        FROM shots
        WHERE location != 19
        GROUP BY location
        HAVING COUNT(*) > 50
    ),
    match_xg AS (
        SELECT s.event_team,
               s.id_odsp,
               SUM(l.xg_per_shot) AS xg_meci,
               SUM(s.is_goal) AS goluri_meci
        FROM shots s
        JOIN loc_xg l ON s.location = l.location
        GROUP BY s.event_team, s.id_odsp
    )
    SELECT event_team,
           COUNT(*) AS meciuri,
           ROUND(AVG(xg_meci), 3) AS xg_mediu,
           ROUND(AVG(goluri_meci), 3) AS
goluri_reale_mediu,
           ROUND(AVG(goluri_meci) - AVG(xg_meci), 3) AS
overperformance
    FROM match_xg
    GROUP BY event_team
    HAVING COUNT(*) >= 50
    ORDER BY overperformance DESC
""")

xg_pd = xg_analysis.toPandas()

for c in ['xg_mediu', 'goluri_reale_mediu', 'overperformance']:
    xg_pd[c] = xg_pd[c].astype(float)
print('Top 10 echipe care SUPRA-performează xG (finalizare clinică /
noroc):')
print(xg_pd.head(10)[['event_team', 'meciuri', 'xg_mediu',
                      'goluri_reale_mediu',
                      'overperformance']].to_string(index=False))
print('\nTop 5 echipe care SUB-performează xG (risipă de ocazii):')
print(xg_pd.tail(5)[['event_team', 'meciuri', 'xg_mediu',
                      'goluri_reale_mediu',
                      'overperformance']].to_string(index=False))

# Scatter: xG mediu vs goluri reale; linia y=x separă supra/sub-
performanța
fig, ax = plt.subplots(figsize=(10, 8))
ax.scatter(xg_pd['xg_mediu'], xg_pd['goluri_reale_mediu'],
           s=40, alpha=0.6, color='steelblue', edgecolor='white')
lim_min = min(xg_pd['xg_mediu'].min(),
              xg_pd['goluri_reale_mediu'].min()) - 0.05

```

```

lim_max = max(xg_pd['xg_mediu'].max(),
xg_pd['goluri_reale_mediu'].max()) + 0.05
ax.plot([lim_min, lim_max], [lim_min, lim_max], 'r--', alpha=0.7,
        label='xG = goluri reale')
# Adnotăm cele mai extreme echipe (supra + sub performanță)
for _, r in pd.concat([xg_pd.head(4), xg_pd.tail(4)]).iterrows():
    ax.annotate(r['event_team'], (r['xg_mediu'],
r['goluri_reale_mediu']),
                fontsize=8, alpha=0.85)
ax.set_xlabel('xG mediu per meci (estimat din locație)')
ax.set_ylabel('Goluri reale medii per meci')
ax.set_title('Goluri reale vs xG estimat – supra/sub-performanță per
echipă')
ax.legend()
plt.tight_layout()
plt.savefig('plots/fig_xg_vs_real.png', dpi=120,
bbox_inches='tight')
plt.show()

```

Top 10 echipe care SUPRA-performează xG (finalizare clinică / noroc):

	event_team	meciuri	xg_mediu	goluri_reale_mediu
overperformance				
0.857	Barcelona	201	1.885	2.741
0.704	Real Madrid	198	2.038	2.742
0.491	Bayern Munich	179	1.928	2.419
0.372	Borussia Dortmund	180	1.745	2.117
0.346	Napoli	207	1.524	1.870
0.302	Paris Saint-Germain	208	1.703	2.005
0.259	Borussia Monchengladbach	182	1.170	1.429
0.234	Lyon	206	1.392	1.626
0.213	AS Monaco	134	1.392	1.604
0.197	Manchester City	130	1.873	2.069

Top 5 echipe care SUB-performează xG (risipă de ocazii):

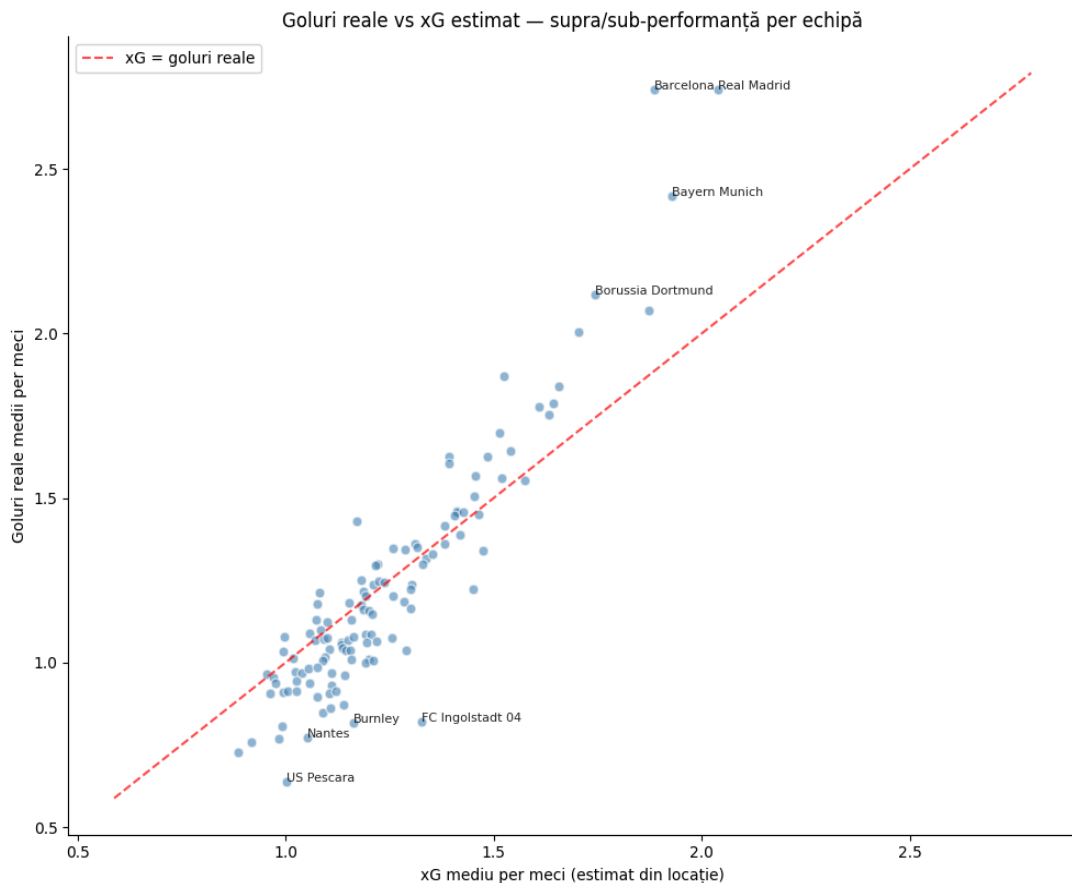
	event_team	meciuri	xg_mediu	goluri_reale_mediu
overperformance				
-0.268	Norwich City	70	1.140	0.871
-0.278	Nantes	133	1.052	0.774
-0.345	Burnley	60	1.162	0.817
-0.364	US Pescara	58	1.002	0.638

FC Ingolstadt 04
-0.506

50

1.326

0.820



<Figure size 1000x800 with 1 Axes>

2.11 Matrice de corelație între atributele șuturilor — Spark MLlib

Ce analizăm: Calculăm corelația Pearson între atributele numerice ale șuturilor (locație, parte corp, situație, metodă assist, fast break, plasament, minut) și ținta `is_goal`, folosind `Correlation.corr` din Spark MLlib (calcul distribuit pe vector de features, nu `pandas.corr`).

```
# Matrice de corelație Pearson între atributele șuturilor (Spark MLlib, calcul distribuit)
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.stat import Correlation
import pandas as pd

num_cols = ['is_goal', 'location', 'bodypart', 'situation',
            'assist_method', 'fast_break', 'shot_place', 'time']

# Asamblăm coloanele într-un singur vector; handleInvalid='skip'
# elimină rândurile cu null
```

```

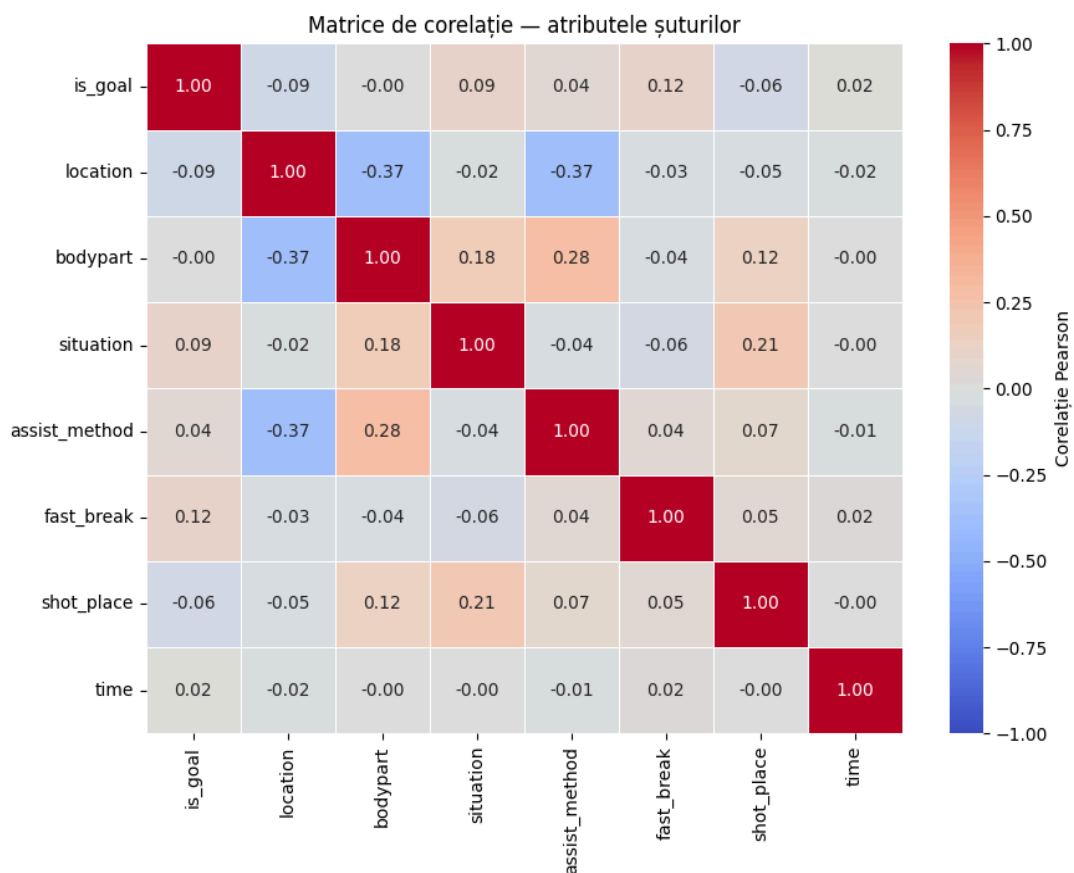
assembler = VectorAssembler(inputCols=num_cols,
                             outputCol='features',
                             handleInvalid='skip')
vec_df = assembler.transform(shots_df.select(*num_cols))

corr_mat = Correlation.corr(vec_df, 'features', 'pearson').head()[0]
corr_pd = pd.DataFrame(corr_mat.toArray(), index=num_cols,
                       columns=num_cols)

fig, ax = plt.subplots(figsize=(9, 7))
sns.heatmap(corr_pd, annot=True, fmt='.2f', cmap='coolwarm',
            center=0,
            vmin=-1, vmax=1, linewidths=0.5, ax=ax,
            cbar_kws={'label': 'Corelație Pearson'})
ax.set_title('Matrice de corelație - attributele șuturilor')
plt.tight_layout()
plt.savefig('plots/fig_correlation_matrix.png', dpi=120,
            bbox_inches='tight')
plt.show()

# Corelația fiecărui atribut cu is_goal, sortată după magnitudine
# (păstrând semnul)
goal_corr = corr_pd['is_goal'].drop('is_goal')
goal_corr =
goal_corr.reindex(goal_corr.abs().sort_values(ascending=False).index
)
print('Corelația fiecărui atribut cu is_goal (sortat după
|valoare|):')
print(goal_corr.round(3).to_string())
[Stage 114:>
(0 + 10) / 10]

```

<Figure size 900x700 with 2 Axes>

Corelația fiecărui atribut cu is_goal (sortat după |valoare|):

```
fast_break      0.116
location        -0.094
situation        0.089
shot_place      -0.064
assist_method    0.040
time             0.015
bodypart        -0.001
```

2.12 Evoluția golurilor pe sezoane și ligi — DataFrame API

Ce analizăm: Media de goluri per meci, grupată pe sezon și ligă, vizualizată ca serie temporală (o linie per ligă).

```
# Evoluția mediei de goluri per meci, pe sezoane și ligi (DataFrame API)
import pandas as pd

goals_evolution = ginf_df \
    .filter(col('season').isNotNull() & col('league').isNotNull() \
           & col('fthg').isNotNull() & col('ftag').isNotNull()) \
    .withColumn('total_goluri', col('fthg') + col('ftag')) \
    .groupBy('league', 'season') \
```

```

    .agg(spark_round(avg('total_goluri'), 3).alias('medie_goluri'),
         count('*').alias('meciuri')) \
    .orderBy('season', 'league')

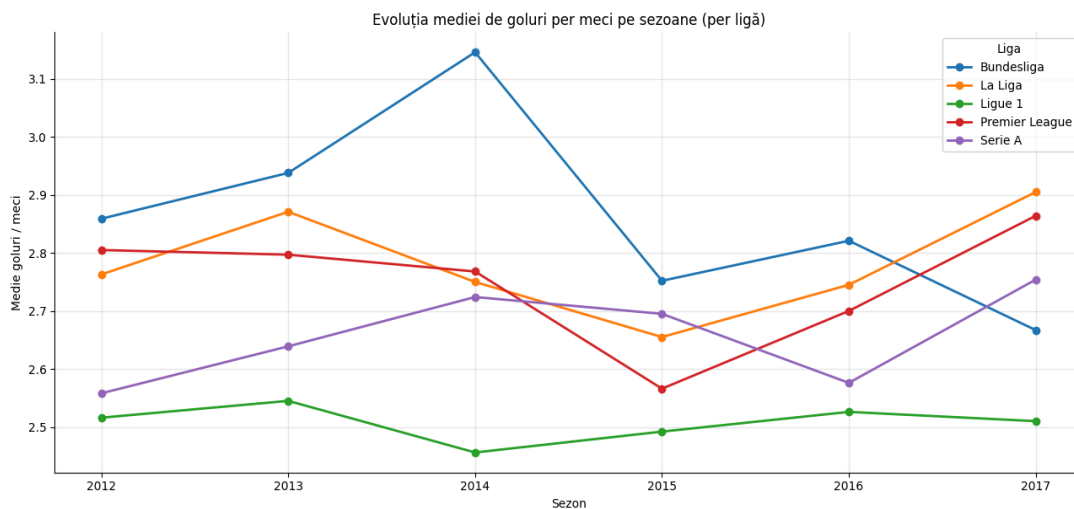
ge_pd = goals_evolution.toPandas()
ge_pd['liga'] =
ge_pd['league'].map(league_map).fillna(ge_pd['league'])

# Pivot „long” -> „wide”: un rând per sezon, o coloană per ligă
(echivalent SQL PIVOT)
pivot_ev = ge_pd.pivot_table(index='season', columns='liga',
                              values='medie_goluri',
                              aggfunc='first').sort_index()

fig, ax = plt.subplots(figsize=(13, 6))
for liga in pivot_ev.columns:
    ax.plot(pivot_ev.index.astype(str), pivot_ev[liga],
            marker='o', linewidth=2, label=liga)
ax.set_xlabel('Sezon')
ax.set_ylabel('Medie goluri / meci')
ax.set_title('Evoluția mediei de goluri per meci pe sezoane (per
liga)')
ax.legend(title='Liga')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('plots/fig_goals_evolution.png', dpi=120,
bbox_inches='tight')
plt.show()

print('Medie goluri/meci pe sezon x ligă:')
print(pivot_ev.round(3).to_string())

```



<Figure size 1300x600 with 1 Axes>

Medie goluri/meci pe sezon x ligă:

liga	Bundesliga	La Liga	Ligue 1	Premier League	Serie A
season					
2012	2.859	2.763	2.516	2.805	2.558
2013	2.938	2.871	2.545	2.797	2.639

2014	3.146	2.750	2.456	2.768	2.724
2015	2.752	2.655	2.492	2.566	2.695
2016	2.821	2.745	2.526	2.700	2.576
2017	2.667	2.905	2.510	2.864	2.754

2.13 Rezumat — statistici generale (Spark SQL)

O ultimă interogare SQL care rezumă datele cheie despre setul de date.

```
summary = spark.sql("""
    SELECT
        COUNT(DISTINCT m.id_odsp)           AS total_meciuri,
        COUNT(DISTINCT m.ht)                AS echipe_unice,
        COUNT(DISTINCT m.league)            AS ligi,
        COUNT(DISTINCT m.season)            AS sezoane,
        SUM(m.fthg + m.ftag)                AS total_goluri,
        ROUND(AVG(m.fthg + m.ftag), 3)      AS medie_goluri_per_meci,
        (SELECT COUNT(*) FROM events WHERE event_type = 1) AS
total_suturi,
        (SELECT COUNT(*) FROM events WHERE is_goal = 1)   AS
total_goluri_events
    FROM matches m
""")

print('=== Statistici generale dataset ===')
summary.show(1, vertical=True, truncate=False)

spark.stop()
print('\nSesiune Spark oprită.')
=== Statistici generale dataset ===
-RECORD 0-----
total_meciuri      | 10112
echipe_unice       | 147
ligi               | 5
sezoane            | 6
total_goluri       | 27333
medie_goluri_per_meci | 2.703
total_suturi       | 229135
total_goluri_events | 24446

Sesiune Spark oprită.
```

Cerința 3 — Metode ML cu Apache Spark MLlib

Aplicăm două metode de Machine Learning distincte folosind biblioteca **Spark MLlib** (`spark.ml`): clasificare binară (Random Forest — prezice dacă un șut devine gol) și regresie (GBT — prezice numărul total de goluri dintr-un meci).

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, sum as spark_sum, avg,
when, isnan
from pyspark.ml.feature import VectorAssembler, StringIndexer,
OneHotEncoder
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.regression import GBTRegressor
from pyspark.ml.evaluation import (
    BinaryClassificationEvaluator,
    MulticlassClassificationEvaluator,
    RegressionEvaluator
)
from pyspark.ml import Pipeline
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.figsize'] = (10, 5)
import seaborn as sns
import numpy as np
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder \
    .appName('FootballML') \
    .config('spark.driver.memory', '4g') \
    .config('spark.sql.shuffle.partitions', '8') \
    .getOrCreate()
spark.sparkContext.setLogLevel('ERROR')
print('Spark version:', spark.version)
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
26/06/11 20:06:40 WARN NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
Spark version: 4.1.2

```

Pregătirea datelor comune

Citim datele, aplicăm filtrele de bază și pregătim dataset-urile pentru cele două probleme ML.

```

events_df = spark.read.csv('data/events.csv', header=True,
inferSchema=True, nullValue='NA')
ginf_df = spark.read.csv('data/gin.csv', header=True,
inferSchema=True, nullValue='NA')

# Dataset șuturi curățat pentru Metoda 1
shots_df = events_df \
    .filter(

```

```

        (col('event_type') == 1) &
        col('is_goal').isNotNull() &
        col('location').isNotNull() &
        col('bodypart').isNotNull() &
        col('situation').isNotNull()
    ) \
    .fillna({'assist_method': 0, 'fast_break': 0, 'shot_place': 0,
'side': 1}) \
    .select('is_goal', 'location', 'bodypart', 'situation',
            'assist_method', 'fast_break', 'shot_place', 'side',
'time')

print(f'Șuturi: {shots_df.count():,} | Goluri:
{shots_df.filter(col("is_goal")==1).count():,}')
[Stage 1:=====>
(2 + 8) / 10]
Șuturi: 229,135 | Goluri: 24,441

```

Metoda 1 — Clasificare binară: Predicția golurilor

Enunțul problemei

Întrebare: Dat un șut cu caracteristicile sale (locație pe teren, parte a corpului folosită, situația de joc, tipul de assist, etc.), putem prezice dacă șutul va fi gol?

Justificarea alegerii — Random Forest Classifier

- **Robust** la outliers și valori lipsă
- Funcționează bine cu **date dezechilibrate** (~10% goluri vs 90% rateuri)
- Oferă **importanța variabilelor** — util pentru a înțelege ce caracteristici contează
- Nu necesită normalizare, se descurcă cu features categorice (după encoding)
- **Scalabil** distribuit în Spark MLlib

```

# Toate features sunt deja numerice (coduri întregi din
dictionary.txt)
feature_cols_clf = ['location', 'bodypart', 'situation',
                    'assist_method', 'fast_break', 'shot_place',
'side', 'time']

assembler_clf = VectorAssembler(
    inputCols=feature_cols_clf,
    outputCol='features',
    handleInvalid='skip'
)

rf = RandomForestClassifier(
    labelCol='is_goal',

```

```

        featuresCol='features',
        numTrees=100,
        maxDepth=8,
        seed=42
    )

    pipeline_clf = Pipeline(stages=[assembler_clf, rf])

    # Split 80/20 stratificat
    train_clf, test_clf = shots_df.randomSplit([0.8, 0.2], seed=42)
    print(f'Train: {train_clf.count():,} | Test: {test_clf.count():,}')
    Train: 183,555 | Test: 45,580

```

Antrenarea modelului

`pipeline_clf.fit(train_clf)` antrenează cei 100 de arbori pe datele de train și întoarce un `PipelineModel` (toate etapele devenite transformeri). `transform(test_clf)` adaugă coloanele `prediction` (clasa prezisă) și `probability` (vectorul de probabilități per clasă).

```

print('Antrenare Random Forest...')
model_clf = pipeline_clf.fit(train_clf)
predictions_clf = model_clf.transform(test_clf)
print('Antrenare completă.')
predictions_clf.select('is_goal', 'prediction',
                        'probability').show(5)
Antrenare Random Forest...
Antrenare completă.
+-----+-----+-----+
|is_goal|prediction|probability|
+-----+-----+-----+
|      0|      0.0|[0.95398155380278...|
|      0|      0.0|[0.95406178025158...|
|      0|      0.0|[0.95406178025158...|
|      0|      0.0|[0.95398155380278...|
|      0|      0.0|[0.95398155380278...|
+-----+-----+-----+
only showing top 5 rows

```

Evaluarea clasificatorului — mai multe metrice, nu doar Accuracy

Pe date dezechilibrate (~10% goluri) accuracy singură e înșelătoare: un model care prezice mereu „nu e gol” ar avea ~90% accuracy fiind inutil. De aceea raportăm AUC-ROC (calitatea ordonării probabilităților, independentă de prag), F1, Precision și Recall (weighted).

```

# Evaluare clasificare
bin_eval = BinaryClassificationEvaluator(labelCol='is_goal',
metricName='areaUnderROC')
mc_eval_acc = MulticlassClassificationEvaluator(labelCol='is_goal',
metricName='accuracy')

```

```

mc_eval_f1 = MulticlassClassificationEvaluator(labelCol='is_goal',
metricName='f1')
mc_eval_prec = MulticlassClassificationEvaluator(labelCol='is_goal',
metricName='weightedPrecision')
mc_eval_rec = MulticlassClassificationEvaluator(labelCol='is_goal',
metricName='weightedRecall')

auc = bin_eval.evaluate(predictions_clf)
accuracy = mc_eval_acc.evaluate(predictions_clf)
f1 = mc_eval_f1.evaluate(predictions_clf)
prec = mc_eval_prec.evaluate(predictions_clf)
rec = mc_eval_rec.evaluate(predictions_clf)

print('=== Metrici Clasificare – Random Forest ===')
print(f' AUC-ROC : {auc:.4f}')
print(f' Accuracy : {accuracy:.4f}')
print(f' F1-Score (wgt) : {f1:.4f}')
print(f' Precision (wgt) : {prec:.4f}')
print(f' Recall (wgt) : {rec:.4f}')
=== Metrici Clasificare – Random Forest ===
AUC-ROC : 0.9545
Accuracy : 0.9298
F1-Score (wgt) : 0.9159
Precision (wgt) : 0.9304
Recall (wgt) : 0.9298

```

Importanța variabilelor

Extragem `featureImportances` din modelul RF: cât a contribuit fiecare feature la reducerea totală a impurității (Gini), mediat peste toți arborii. Ne arată *pe ce se bazează* modelul când prezice golul.

```

# Importanța variabilelor
rf_model = model_clf.stages[-1]
importances = rf_model.featureImportances.toArray()

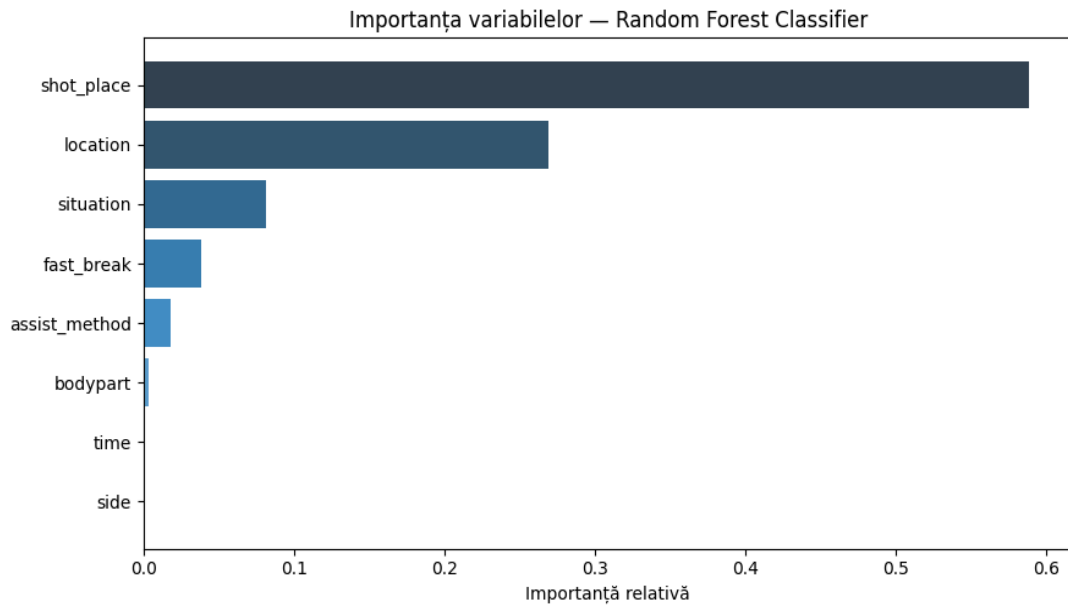
feat_imp = sorted(zip(feature_cols_clf, importances), key=lambda x:
x[1], reverse=True)

fig, ax = plt.subplots(figsize=(9, 5))
names, values = zip(*feat_imp)
ax.barh(names[:-1], values[:-1],
color=sns.color_palette('Blues_d', len(names)))
ax.set_title('Importanța variabilelor – Random Forest Classifier')
ax.set_xlabel('Importanță relativă')
plt.tight_layout()
plt.savefig('plots/fig_feature_importance_clf.png', dpi=120,
bbox_inches='tight')
plt.show()

print('\nTop features:')

```

```
for name, val in feat_imp:
    print(f' {name:<20}: {val:.4f}')
```



<Figure size 900x500 with 1 Axes>

Top features:

shot_place	: 0.5882
location	: 0.2688
situation	: 0.0818
fast_break	: 0.0381
assist_method	: 0.0180
bodypart	: 0.0036
time	: 0.0013
side	: 0.0002

Matricea de confuzie

Numărăm combinațiile (clasă reală × clasă prezisă) cu un `groupBy` distribuit și pivotăm rezultatul în pandas pentru vizualizare. Matricea descompune erorile în False Positives (șuturi prezise gol care nu au fost) și False Negatives (goluri ratate de model).

```
# Matricea de confuzie
cm_data = predictions_clf \
    .groupBy('is_goal', 'prediction') \
    .count() \
    .orderBy('is_goal', 'prediction') \
    .toPandas()

cm_pivot = cm_data.pivot(index='is_goal', columns='prediction',
                          values='count').fillna(0)

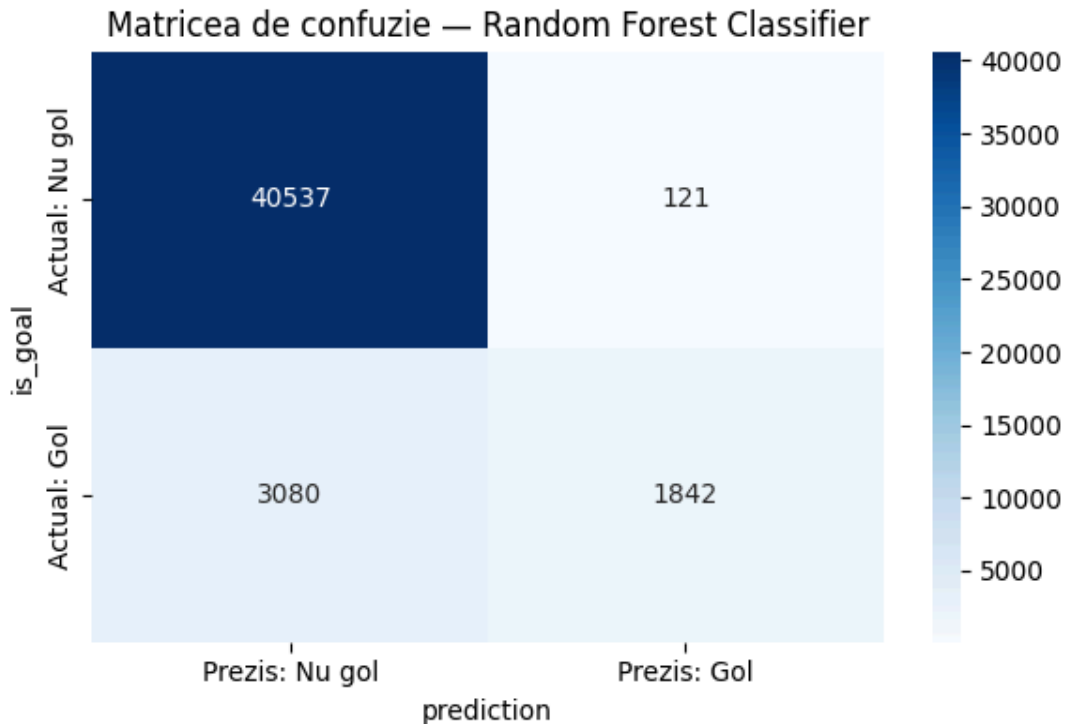
fig, ax = plt.subplots(figsize=(6, 4))
```



```

sns.heatmap(cm_pivot, annot=True, fmt='.0f', cmap='Blues',
            xticklabels=['Prezis: Nu gol', 'Prezis: Gol'],
            yticklabels=['Actual: Nu gol', 'Actual: Gol'], ax=ax)
ax.set_title('Matricea de confuzie – Random Forest Classifier')
plt.tight_layout()
plt.savefig('plots/fig_confusion_matrix_clf.png', dpi=120,
            bbox_inches='tight')
plt.show()
[Stage 59:=====>
(7 + 3) / 10]

```



<Figure size 600x400 with 2 Axes>

Diagnoză: overfitting (AUC train vs test)

Ce verificăm: Evaluăm același Random Forest pe setul de *train* și pe cel de *test*, comparând AUC-ROC. Dacă modelul memorează datele de antrenament în loc să învețe tipare generalizabile, AUC pe train va fi semnificativ mai mare decât pe test.

```

# Verificare overfitting: comparăm AUC-ROC pe train vs test pentru
Random Forest baseline
preds_train_clf = model_clf.transform(train_clf)
auc_train = bin_eval.evaluate(preds_train_clf)
auc_test = bin_eval.evaluate(predictions_clf) # același set test
ca mai sus
gap = auc_train - auc_test

print('=== Diagnoză overfitting – Random Forest (baseline) ===')

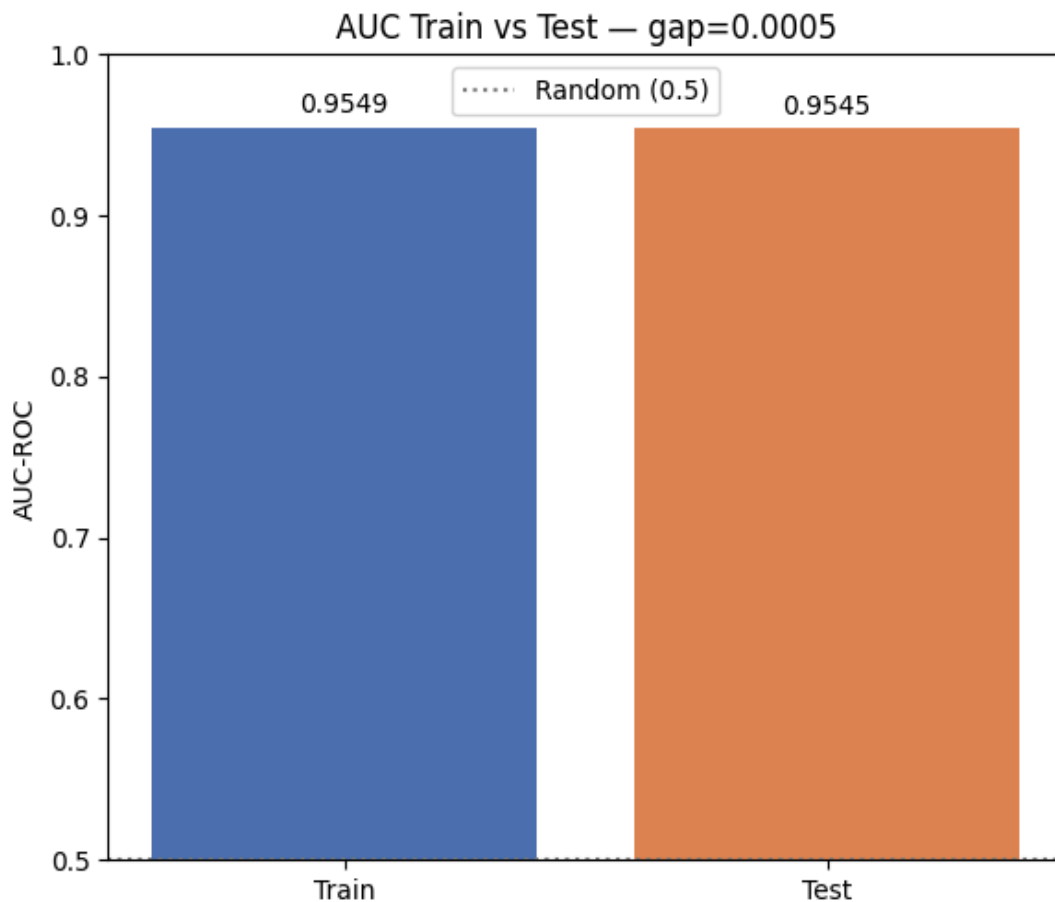
```

```

print(f' AUC train : {auc_train:.4f}')
print(f' AUC test  : {auc_test:.4f}')
print(f' Gap      : {gap:.4f}')
if gap < 0.02:
    verdict = 'NU overfit – generalizează bine (gap < 0.02)'
elif gap < 0.05:
    verdict = 'Overfitting UȘOR – acceptabil (gap 0.02–0.05)'
else:
    verdict = 'Overfitting SEMNIFICATIV – reduceți maxDepth / mai
multe date'
print(f' Verdict   : {verdict}')

fig, ax = plt.subplots(figsize=(6, 5))
bars = ax.bar(['Train', 'Test'], [auc_train, auc_test], color=
['#4c72b0', '#dd8452'])
ax.bar_label(bars, fmt='%.4f', padding=3)
ax.axhline(0.5, color='gray', linestyle=':', label='Random (0.5)')
ax.set_ylim(0.5, 1.0)
ax.set_ylabel('AUC-ROC')
ax.set_title(f'AUC Train vs Test – gap={gap:.4f}')
ax.legend()
plt.tight_layout()
plt.savefig('plots/fig_auc_train_vs_test.png', dpi=120,
bbox_inches='tight')
plt.show()
=== Diagnoză overfitting – Random Forest (baseline) ===
    AUC train : 0.9549
    AUC test  : 0.9545
    Gap      : 0.0005
    Verdict   : NU overfit – generalizează bine (gap < 0.02)

```



<Figure size 600x500 with 1 Axes>

Tratarea dezechilibrului de clase (`weightCol`)

Problema: Doar ~10-12% din șuturi sunt goluri. Un model antrenat naiv învață să prezică „nu e gol” aproape mereu — obține accuracy mare, dar **recall slab pe goluri** (ratează exact clasa care ne interesează).

```
from pyspark.sql.functions import lit

# Ponderi din distribuția claselor pe train (balanced weighting):
# fiecare clasă primește o pondere = fracția clasei OPUSE =>
# minoritarul (goluri) cântărește mai mult
n_total = train_clf.count()
n_pos   = train_clf.filter(col('is_goal') == 1).count()
n_neg   = n_total - n_pos
w_pos   = n_neg / n_total
w_neg   = n_pos / n_total
print(f'Distribuție train: goluri={n_pos:,}
      ({n_pos/n_total*100:.1f}%) | ne-goluri={n_neg:,}')
print(f'Pondere gol={w_pos:.3f} | pondere ne-gol={w_neg:.3f} (raport
      {w_pos/w_neg:.1f}x)')

train_w = train_clf.withColumn(
    'classWeight',
```

```

        when(col('is_goal') == 1, lit(w_pos)).otherwise(lit(w_neg))
    )

    rf_weighted = RandomForestClassifier(
        labelCol='is_goal', featuresCol='features',
        weightCol='classWeight',
        numTrees=100, maxDepth=8, seed=42
    )
    pipeline_clf_w = Pipeline(stages=[assembler_clf, rf_weighted])

    print('\nAntrenare Random Forest cu ponderi pe clase...')
    model_clf_w = pipeline_clf_w.fit(train_w)
    preds_w = model_clf_w.transform(test_clf)

def confusion_2x2(preds):
    """Matrice de confuzie 2x2 robustă (reindexare ca să existe
    mereu ambele clase)."""
    cm = preds.groupby('is_goal', 'prediction').count().toPandas()
    return (cm.pivot(index='is_goal', columns='prediction',
        values='count')
        .reindex(index=[0, 1], columns=[0.0, 1.0]).fillna(0))

def recall_goluri(cmp):
    tp, fn = cmp.loc[1, 1.0], cmp.loc[1, 0.0]
    return tp / (tp + fn) if (tp + fn) > 0 else 0.0

cm_base = confusion_2x2(predictions_clf)
cm_w = confusion_2x2(preds_w)
auc_w = bin_eval.evaluate(preds_w)
acc_w = mc_eval_acc.evaluate(preds_w)

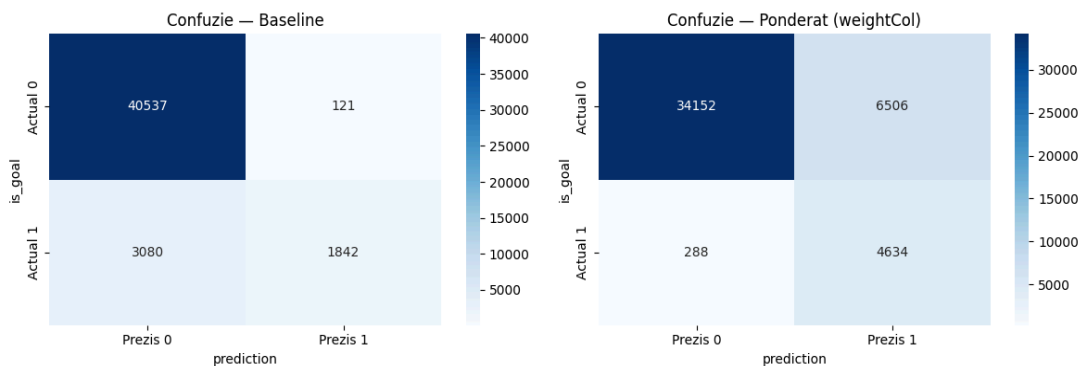
print('\n=== Baseline vs Ponderat (clasa minoritară = goluri) ===')
print(f' Recall goluri baseline : {recall_goluri(cm_base):.4f}')
print(f' Recall goluri ponderat : {recall_goluri(cm_w):.4f}')
print(f' AUC baseline / ponderat: {auc_test:.4f} / {auc_w:.4f}')
print(f' Accuracy ponderat : {acc_w:.4f}')

# Matrice de confuzie comparativă baseline vs ponderat
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, cmp, titlu in [(axes[0], cm_base, 'Baseline'),
    (axes[1], cm_w, 'Ponderat (weightCol)')]:
    sns.heatmap(cmp, annot=True, fmt='.0f', cmap='Blues', ax=ax,
        xticklabels=['Prezis 0', 'Prezis 1'],
        yticklabels=['Actual 0', 'Actual 1'])
    ax.set_title(f'Confuzie - {titlu}')
plt.tight_layout()
plt.savefig('plots/fig_confusion_weighted_vs_baseline.png', dpi=120,
    bbox_inches='tight')
plt.show()
Distribuție train: goluri=19,519 (10.6%) | ne-goluri=164,036
Pondere gol=0.894 | pondere ne-gol=0.106 (raport 8.4x)

```

```
Antrenare Random Forest cu ponderi pe clase...
[Stage 102:=====>
(1 + 9) / 10]
```

```
=== Baseline vs Ponderat (clasa minoritară = goluri) ===
Recall goluri baseline : 0.3742
Recall goluri ponderat : 0.9415
AUC baseline / ponderat: 0.9545 / 0.9587
Accuracy ponderat      : 0.8509
```



<Figure size 1200x400 with 4 Axes>

Metoda 2 — Regresie: Predicția numărului de goluri pe meci

Enunțul problemei

Întrebare: Pe baza statisticilor agregate dintr-un meci (număr de corneruri, fault-uri, șuturi, cartonașe per echipă), putem prezice **câte goluri** se vor marca în total?

Justificarea alegerii — GBT Regressor

- **Gradient-Boosted Trees** sunt robuste și performante pe date tabulare
- Capturează **relații non-liniare** complexe (ex: un meci cu 20 corneruri nu înseamnă neapărat mai multe goluri)
- Minimizează iterativ eroarea pe exemple dificile (**boosting**)
- Diferit metodologic față de Random Forest (bagging vs boosting) — demonstrăm diversitate de abordări

```
from pyspark.sql.functions import countDistinct

# Agregăm statistici per meci din events_df
match_stats = events_df.groupBy('id_odsp').agg(
    count(when(col('event_type') == 1, 1)).alias('total_suturi'),
```

```

    count(when((col('event_type') == 1) & (col('side') == 1),
1)).alias('suturi_acasa'),
    count(when((col('event_type') == 1) & (col('side') == 2),
1)).alias('suturi_deplasare'),
    count(when(col('event_type') == 2, 1)).alias('corneruri'),
    count(when(col('event_type') == 3, 1)).alias('falturi'),
    count(when(col('event_type') == 4, 1)).alias('galbene'),
    count(when(col('event_type') == 6, 1)).alias('rosii'),
    count(when(col('event_type') == 11, 1)).alias('penaltiuri'),
    count(when(col('fast_break') == 1, 1)).alias('contraatacuri')
)

# Join cu ginf pentru target (total goluri)
reg_df = match_stats.join(
    ginf_df.select('id_odsp', 'fthg', 'ftag'),
    on='id_odsp', how='inner'
).withColumn(
    'total_goals',
    (col('fthg') + col('ftag')).cast('double')
).drop('id_odsp', 'fthg', 'ftag') \
    .filter(col('total_goals').isNotNull())

print(f'Meciuri pentru regresie: {reg_df.count():,}')
reg_df.describe('total_goals', 'total_suturi', 'corneruri').show()
Meciuri pentru regresie: 9,074
+-----+-----+-----+-----+
|summary|      total_goals|      total_suturi|      corneruri|
+-----+-----+-----+-----+
|  count|           9074|           9074|           9074|
|   mean| 2.695724046726912|25.251818382190876|10.051135111307032|
| stddev|1.6819427293998597|5.7523718865185085|3.4086506561293533|
|    min|             0.0|             5|             1|
|    max|             12.0|             51|             26|
+-----+-----+-----+-----+

```

Asamblarea features și configurarea GBT

`VectorAssembler` împachetează cele 9 statistici agregate per meci într-o singură coloană `features` (formatul obligatoriu `MLlib`), apoi `GBRegressor` antrenează 50 de arbori *secvențial* — fiecare arbore nou corectează reziduurile celor anteriori, ponderat cu `stepSize=0.1` (learning rate).

```

feature_cols_reg = ['total_suturi', 'suturi_acasa',
'suturi_deplasare',
                    'corneruri', 'falturi', 'galbene', 'rosii',
                    'penaltiuri', 'contraatacuri']

assembler_reg = VectorAssembler(
    inputCols=feature_cols_reg,
    outputCol='features',
    handleInvalid='skip'
)

```

```

gbt = GBRegressor(
    labelCol='total_goals',
    featuresCol='features',
    maxIter=50,
    maxDepth=5,
    stepSize=0.1,
    seed=42
)

pipeline_reg = Pipeline(stages=[assembler_reg, gbt])

train_reg, test_reg = reg_df.randomSplit([0.8, 0.2], seed=42)
print(f'Train: {train_reg.count():,} | Test: {test_reg.count():,}')

print('Antrenare GBRegressor...')
model_reg = pipeline_reg.fit(train_reg)
preds_reg = model_reg.transform(test_reg)
print('Antrenare completă.')
Train: 7,333 | Test: 1,741
Antrenare GBRegressor...
Antrenare completă.

```

Evaluarea regresiei — RMSE, MAE, R²

RMSE penalizează pătratic erorile mari, MAE e eroarea medie „în goluri” (direct interpretabilă), iar R² spune ce proporție din varianța numărului de goluri e explicată de model.

```

rmse_eval = RegressionEvaluator(labelCol='total_goals',
    metricName='rmse')
mae_eval = RegressionEvaluator(labelCol='total_goals',
    metricName='mae')
r2_eval = RegressionEvaluator(labelCol='total_goals',
    metricName='r2')

rmse = rmse_eval.evaluate(preds_reg)
mae = mae_eval.evaluate(preds_reg)
r2 = r2_eval.evaluate(preds_reg)

print('=== Metrice Regresie - GBRegressor ===')
print(f' RMSE : {rmse:.4f} (eroare medie pătratică)')
print(f' MAE : {mae:.4f} (eroare medie absolută)')
print(f' R2 : {r2:.4f} (proporția varianței explicate)')
=== Metrice Regresie - GBRegressor ===
    RMSE : 1.5828 (eroare medie pătratică)
    MAE : 1.2618 (eroare medie absolută)
    R2 : 0.0717 (proporția varianței explicate)
# Vizualizare: valori reale vs prezise
sample_preds = preds_reg.select('total_goals',
    'prediction').limit(500).toPandas()

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

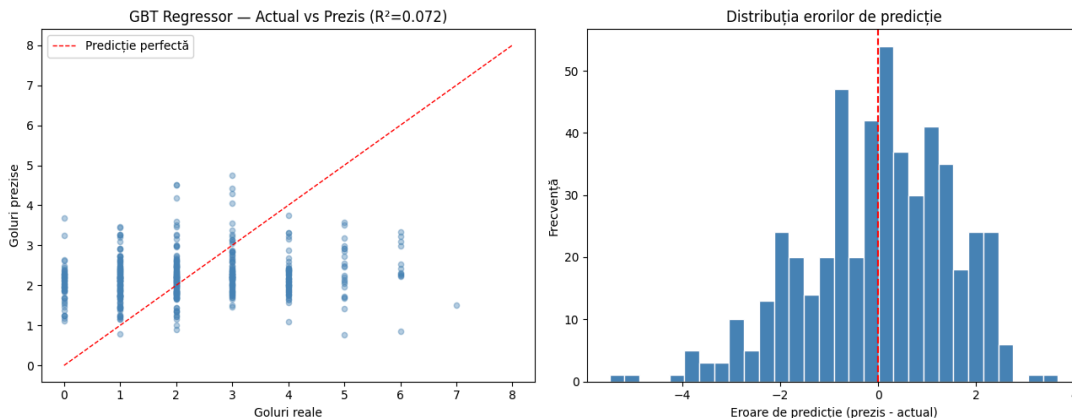
```

```
# Scatter actual vs predicted
axes[0].scatter(sample_preds['total_goals'],
                sample_preds['prediction'],
                alpha=0.4, color='steelblue', s=20)
lims = [0, max(sample_preds['total_goals'].max(),
                sample_preds['prediction'].max()) + 1]
axes[0].plot(lims, lims, 'r--', lw=1, label='Predicție perfectă')
axes[0].set_xlabel('Goluri reale')
axes[0].set_ylabel('Goluri prezise')
axes[0].set_title(f'GBT Regressor – Actual vs Prezis (R²={r2:.3f})')
axes[0].legend()

# Distribuția erorilor
errors = sample_preds['prediction'] - sample_preds['total_goals']
axes[1].hist(errors, bins=30, color='steelblue', edgecolor='white')
axes[1].axvline(0, color='red', linestyle='--')
axes[1].set_xlabel('Eroare de predicție (prezis - actual)')
axes[1].set_ylabel('Frecvență')
axes[1].set_title('Distribuția erorilor de predicție')

plt.tight_layout()
plt.savefig('plots/fig_regression_results.png', dpi=120,
            bbox_inches='tight')
plt.show()

spark.stop()
print('Sesiune Spark oprită.')
```



<Figure size 1300x500 with 2 Axes>
Sesiune Spark oprită.

Cerința 4 — Data Pipeline

Un **Data Pipeline** reprezintă un lanț automatizat de transformări care conduc datele dintr-o formă brută la o formă utilizabilă pentru analiză sau modelare.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
```



```

        col, when, count, isnan, floor, year, to_date,
        sum as spark_sum
    )
from pyspark.sql.types import IntegerType, DoubleType
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    VectorAssembler, StandardScaler, Imputer, Bucketizer
)
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
import os
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder \
    .appName('FootballPipeline') \
    .config('spark.driver.memory', '4g') \
    .config('spark.sql.shuffle.partitions', '8') \
    .getOrCreate()
spark.sparkContext.setLogLevel('ERROR')
print('Spark version:', spark.version)
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
26/06/11 20:06:15 WARN NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
Spark version: 4.1.2

```

4.1 Pipeline ETL — Citire, Curățare și Feature Engineering

Etapele pipeline-ului ETL:

```

CSV brut → [1] Citire cu schemă → [2] Filtrare → [3] Curățare null-
uri
        → [4] Cast tipuri → [5] Feature engineering → [6] Scriere
Parquet
# Pasul 1: Citire date brute
print('[ETL] Pasul 1: Citire date brute...')
raw_events = spark.read.csv('data/events.csv', header=True,
inferSchema=True, nullValue='NA')
raw_ginf    = spark.read.csv('data/ginf.csv',    header=True,
inferSchema=True, nullValue='NA')
print(f'  events: {raw_events.count():,} rânduri | ginf:
{raw_ginf.count():,} rânduri')
[ETL] Pasul 1: Citire date brute...
    events: 941,009 rânduri | ginf: 10,112 rânduri

```

```
# Pasul 2: Filtrare – păstrăm doar tentativele de șut cu date
complete
print('[ETL] Pasul 2: Filtrare tentative de șut...')
shots_raw = raw_events.filter(
    (col('event_type') == 1) &
    col('is_goal').isNotNull() &
    col('location').isNotNull() &
    col('bodypart').isNotNull() &
    col('situation').isNotNull() &
    col('time').isNotNull()
)
print(f' Rânduri după filtrare: {shots_raw.count():,}')
[ETL] Pasul 2: Filtrare tentative de șut...
Rânduri după filtrare: 229,135
# Pasul 3: Curățare – înlocuim null-urile din coloane opționale
print('[ETL] Pasul 3: Curățare valori lipsă...')
shots_clean = shots_raw.fillna({
    'assist_method': 0,
    'fast_break': 0,
    'shot_place': 0,
    'side': 1
})
[ETL] Pasul 3: Curățare valori lipsă...
```

Feature Engineering — coloane noi derivate

Pe lângă cast-ul explicit la tipuri întregi, construim trei features noi cu `withColumn: is_central_shot` (zonele centrale cu conversie mare), `time_phase` (început/mijloc/final de meci) și `is_set_piece` (fază fixă). Modelul primește astfel semnale pe care nu le-ar putea deduce ușor din codurile brute.

```
# Pasul 4 & 5: Cast tipuri corecte + Feature Engineering
print('[ETL] Pasul 4-5: Cast tipuri + Feature Engineering...')
shots_engineered = shots_clean \
    .withColumn('is_goal', col('is_goal').cast(IntegerType())) \
    .withColumn('location', col('location').cast(IntegerType())) \
    .withColumn('bodypart', col('bodypart').cast(IntegerType())) \
    .withColumn('situation', col('situation').cast(IntegerType())) \
    .withColumn('assist_method', col('assist_method').cast(IntegerType())) \
    .withColumn('fast_break', col('fast_break').cast(IntegerType())) \
    .withColumn('shot_place', col('shot_place').cast(IntegerType())) \
    .withColumn('time', col('time').cast(DoubleType())) \
    .withColumn('side', col('side').cast(IntegerType())) \
```

```

\
    .withColumn('is_central_shot',
                  when(col('location').isin(3, 5, 10, 12, 13, 14),
1).otherwise(0)) \
    \
    .withColumn('time_phase',
                  when(col('time') <= 45, 1)           # prima repriză
                  .when(col('time') <= 70, 2)         # a doua
repriză - început
                  .otherwise(3)) \
    \
    .withColumn('is_set_piece',
                  when(col('situation').isin(2, 3, 4),
1).otherwise(0)) \
    \
    .select('id_odsp', 'id_event', 'is_goal',
            'location', 'bodypart', 'situation', 'assist_method',
            'fast_break', 'shot_place', 'side', 'time',
            'is_central_shot', 'time_phase', 'is_set_piece')

print(f' Coloane după feature engineering:
{len(shots_engineered.columns)}')
shots_engineered.show(3)
[ETL] Pasul 4-5: Cast tipuri + Feature Engineering...
    Coloane după feature engineering: 14
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+
| id_odsp|
id_event|is_goal|location|bodypart|situation|assist_method|fast_brea
k|shot_place|side|time|is_central_shot|time_phase|is_set_piece|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+
|UFot0hit/|UFot0hit1|      0|      9|      2|      1|
1|      0|      6| 2| 2.0|      0|      1|
0|
|UFot0hit/|UFot0hit12|      0|     15|      1|      1|
1|      0|     13| 1|14.0|      0|      1|
0|
|UFot0hit/|UFot0hit14|      1|      9|      2|      1|
1|      0|      4| 1|17.0|      0|      1|
0|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+
only showing top 3 rows

```

Scrierea în Parquet partiționat — capătul pipeline-ului ETL

Parquet e format *columnar*: compresie mai bună (valori similare adiacente), citire doar a coloanelor cerute (column pruning) și filtrare la citire (predicate pushdown). `partitionBy('situation')` împarte fizic datele în subdirectoare

(`situation=1/`, `situation=2/` ...) — un filtru ulterior pe `situation` va sări complet peste partițiile irelevante (partition pruning).

```
# Pasul 6: Scriere în format Parquet (stocare optimizată)
print('[ETL] Pasul 6: Salvare în format Parquet...')
parquet_path = 'data/processed_shots'

shots_engineered.write \
    .mode('overwrite') \
    .partitionBy('situation') \
    .parquet(parquet_path)

print(f' Salvat în: {parquet_path}/')

# Verificare citire Parquet
shots_parquet = spark.read.parquet(parquet_path)
print(f' Citit din Parquet: {shots_parquet.count():,} rânduri')
print(f' Schema Parquet:')
shots_parquet.printSchema()
[ETL] Pasul 6: Salvare în format Parquet...
[Stage 14:>
(0 + 10) / 10]
Salvat în: data/processed_shots/
Citit din Parquet: 229,135 rânduri
Schema Parquet:
root
 |-- id_odsp: string (nullable = true)
 |-- id_event: string (nullable = true)
 |-- is_goal: integer (nullable = true)
 |-- location: integer (nullable = true)
 |-- bodypart: integer (nullable = true)
 |-- assist_method: integer (nullable = true)
 |-- fast_break: integer (nullable = true)
 |-- shot_place: integer (nullable = true)
 |-- side: integer (nullable = true)
 |-- time: double (nullable = true)
 |-- is_central_shot: integer (nullable = true)
 |-- time_phase: integer (nullable = true)
 |-- is_set_piece: integer (nullable = true)
 |-- situation: integer (nullable = true)
```

4.2 Pipeline MLlib — Preprocesare + Model ML

Construim un **Pipeline MLlib** complet care:

1. **Imputează** valorile lipsă rămase
2. **Asamblează** toate features într-un singur vector
3. **Scalează** features numerice (StandardScaler)
4. **Antrenează** un Random Forest Classifier

5. **Serializează** întregul pipeline pe disc — poate fi reîncărcat și aplicat pe date noi fără reantrenare

```
# Citim datele procesate din Parquet
data_df = spark.read.parquet('data/processed_shots')

feature_cols = ['location', 'bodypart', 'situation',
                'assist_method',
                'fast_break', 'shot_place', 'side', 'time',
                'is_central_shot', 'time_phase', 'is_set_piece']

# Etapa 1: Imputare (înlocuire null cu mediană)
imputer = Imputer(
    inputCols=feature_cols,
    outputCols=[f'{c}_imp' for c in feature_cols],
    strategy='median'
)

# Etapa 2: VectorAssembler
assembler = VectorAssembler(
    inputCols=[f'{c}_imp' for c in feature_cols],
    outputCol='features_raw',
    handleInvalid='skip'
)

# Etapa 3: StandardScaler
scaler = StandardScaler(
    inputCol='features_raw',
    outputCol='features',
    withMean=True,
    withStd=True
)

# Etapa 4: Model RF
rf_model = RandomForestClassifier(
    labelCol='is_goal',
    featuresCol='features',
    numTrees=100,
    maxDepth=8,
    seed=42
)

# Construim Pipeline-ul complet
full_pipeline = Pipeline(stages=[imputer, assembler, scaler,
rf_model])

print('Structura Pipeline-ului MLlib:')
for i, stage in enumerate(full_pipeline.getStages()):
    print(f' Etapa {i+1}: {type(stage).__name__}')
Structura Pipeline-ului MLlib:
    Etapa 1: Imputer
    Etapa 2: VectorAssembler
```

Etapa 3: StandardScaler
Etapa 4: RandomForestClassifier

Antrenarea și evaluarea pipeline-ului complet

`full_pipeline.fit(train_df)` rulează etapele în ordine: estimatorii (`Imputer`, `StandardScaler`, `RandomForestClassifier`) învață din date și devin transformeri; transformerii (`VectorAssembler`) se aplică direct. Rezultatul e un singur obiect `PipelineModel` care încapsulează tot lanțul de preprocesare + model.

```
train_df, test_df = data_df.randomSplit([0.8, 0.2], seed=42)

print('Antrenare Pipeline MLlib complet...')
pipeline_model = full_pipeline.fit(train_df)
print('Pipeline antrenat.')

predictions = pipeline_model.transform(test_df)

evaluator = BinaryClassificationEvaluator(labelCol='is_goal',
metricName='areaUnderROC')
auc = evaluator.evaluate(predictions)
print(f'\nAUC-ROC pe test set: {auc:.4f}')
Antrenare Pipeline MLlib complet...
[Stage 33:=====>
(6 + 3) / 9]
[Stage 33:=====>
(7 + 2) / 9]
[Stage 39:=====>
(7 + 2) / 9]
[Stage 41:=====>
(6 + 3) / 9] [Stage
41:=====> (7 + 2)
/ 9]

[Stage 43:=====>
(6 + 3) / 9] [Stage
43:=====> (7 + 2)
/ 9]

[Stage 45:=====>
(6 + 3) / 9] [Stage
45:=====> (7 + 2)
/ 9]

[Stage 47:=====>
(6 + 3) / 9]
[Stage 47:=====>
(7 + 2) / 9]

Pipeline antrenat.
```

AUC-ROC pe test set: 0.9433

Serializarea și reîncărcarea — dovada reproductibilității

Salvăm pipeline-ul întreg (nu doar modelul) pe disc și îl reîncărcăm cu `PipelineModel.load`. AUC-ul identic înainte/după demonstrează că artefactul conține *tot* ce e nevoie la inferență: mediana imputer-ului, media/deviația scaler-ului și toți arborii RF.

```
# Serializarea Pipeline-ului antrenat pe disc
pipeline_save_path = 'models/rf_pipeline_model'
os.makedirs('models', exist_ok=True)

pipeline_model.write().overwrite().save(pipeline_save_path)
print(f'Pipeline salvat în: {pipeline_save_path}')

# Reîncărcăm pipeline-ul pentru a demonstra reproductibilitatea
from pyspark.ml import PipelineModel
loaded_pipeline = PipelineModel.load(pipeline_save_path)

preds_loaded = loaded_pipeline.transform(test_df)
auc_loaded = evaluator.evaluate(preds_loaded)
print(f'AUC-ROC pipeline reîncărcat: {auc_loaded:.4f} (trebuie să
fie identic cu {auc:.4f})')

spark.stop()
print('\nSesiune Spark oprită.')
Pipeline salvat în: models/rf_pipeline_model
AUC-ROC pipeline reîncărcat: 0.9433 (trebuie să fie identic cu
0.9433)

Sesiune Spark oprită.
```

Cerința 5 — UDF și Optimizarea Hiperparametrilor

Această cerință demonstrează două tehnici în Spark: **UDF** (funcție Python personalizată, înregistrată și în Spark SQL) și **optimizarea hiperparametrilor** cu `CrossValidator` + `ParamGridBuilder` (validare încrucișată k-fold).

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when, udf, pandas_udf
from pyspark.sql.types import FloatType, StringType
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator,
```

```

MulticlassClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.figsize'] = (11, 5)
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder \
    .appName('FootballUDF_HyperTuning') \
    .config('spark.driver.memory', '4g') \
    .config('spark.sql.shuffle.partitions', '8') \
    .getOrCreate()
spark.sparkContext.setLogLevel('ERROR')
print('Spark version:', spark.version)
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
26/06/11 20:07:16 WARN NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
Spark version: 4.1.2

```

Citirea datelor procesate — dependență de Cerința 4

Citim Parquet-ul `data/processed_shots` scris de pipeline-ul ETL din Cerința 4 (cu fallback pe CSV brut dacă lipsește). Refolosirea artefactului procesat evită repetarea curățării și demonstrează lanțul firesc dintre notebook-uri.

```

# Citim datele procesate din Parquet (produse de Cerința 4)
# Dacă Parquet-ul nu există, citim din CSV
import os
if os.path.exists('data/processed_shots'):
    shots_df = spark.read.parquet('data/processed_shots')
    print('Date citite din Parquet (procesate).')
else:
    shots_df = spark.read.csv('data/events.csv', header=True,
inferSchema=True, nullValue='NA') \
        .filter((col('event_type') == 1) &
col('is_goal').isNotNull() &
        col('location').isNotNull() &
col('bodypart').isNotNull() &
        col('situation').isNotNull()) \
        .fillna({'assist_method': 0, 'fast_break': 0, 'shot_place':
0, 'side': 1})
    print('Date citite din CSV (brut).')

print(f'Rânduri disponibile: {shots_df.count():,}')

```


Date citite din Parquet (procesate).
Rânduri disponibile: 229,135

5.1 UDF — Funcție definită de utilizator:

shot_quality_score

Motivație

Nicio coloană din dataset nu cuantifică în mod direct „calitatea” unui șut. Creăm o funcție compozită care combină mai mulți factori într-un singur scor de calitate [0.0, 1.0].

Logica scorului

Scorul crește când:

- Șutul vine din zone **centrale** ale careului (locații 3, 5, 10, 12, 13, 14)
- Este executat cu **picioarul** (mai precis decât cu capul în general)
- Vine dintr-un **contraatac rapid** (apărarea dezorganizată)
- Situația este **joc deschis** (nu faze fixe întâmplătoare)

```
@udf(returnType=FloatType())
def shot_quality_score(location, bodypart, situation, fast_break):
    """
    Calculează un scor de calitate al șutului în intervalul [0.0,
    1.0].

    Factori luați în considerare:
    - Locația (zone centrale = scor mai mare)
    - Partea corpului (picior = bonus față de cap)
    - Situația de joc (joc deschis = mai mult control)
    - Contraatac rapid (apărare dezorganizată = avantaj)
    """
    if location is None:
        return 0.0

    score = 0.0

    # Factor 1: Locație (40% din scor)
    central_locations = {3, 5, 10, 12, 13, 14} # centru careu, 6
    good_locations = {4, 6, 9, 11} # unghiuri
    rezonabile
    if location in central_locations:
        score += 0.40
    elif location in good_locations:
        score += 0.20
    else:
        score += 0.05 # unghi dificil sau distanță mare
```

```

# Factor 2: Parte corp (25% din scor)
if bodypart is not None:
    if bodypart in (1, 2):    # picior drept / stâng
        score += 0.25
    else:                    # cap
        score += 0.15

# Factor 3: Situație de joc (20% din scor)
if situation is not None:
    if situation == 1:        # joc deschis
        score += 0.20
    elif situation == 4:      # lovitură liberă directă
        score += 0.15
    else:
        score += 0.08

# Factor 4: Contraatac rapid (15% din scor)
if fast_break is not None and fast_break == 1:
    score += 0.15

return float(min(score, 1.0))

@udf(returnType=StringType())
def shot_quality_label(score):
    """Convertește scorul numeric într-o etichetă categorică."""
    if score is None:
        return 'unknown'
    if score >= 0.75:
        return 'excelent'
    elif score >= 0.55:
        return 'bun'
    elif score >= 0.35:
        return 'mediu'
    else:
        return 'slab'

print('UDF-uri înregistrate: shot_quality_score,
      shot_quality_label')
UDF-uri înregistrate: shot_quality_score, shot_quality_label

```

Aplicarea UDF-urilor pe DataFrame

`withColumn` + UDF: pentru fiecare rând, Spark serializează valorile din JVM către un proces Python, execută funcția și serializează rezultatul înapoi. De aceea UDF-urile standard sunt lente — overhead per rând — dar permit logică arbitrară Python pe care funcțiile native nu o exprimă.

```

# Aplicăm UDF-ul pe DataFrame
shots_with_quality = shots_df.withColumn(
    'shot_quality',

```

```

    shot_quality_score(col('location'), col('bodypart'),
col('situation'), col('fast_break'))
).withColumn(
    'quality_label',
    shot_quality_label(col('shot_quality'))
)

print('Primele 10 rânduri cu shot_quality calculat:')
shots_with_quality.select(
    'location', 'bodypart', 'situation', 'fast_break',
    'shot_quality', 'quality_label', 'is_goal'
).show(10)
Primele 10 rânduri cu shot_quality calculat:
+-----+-----+-----+-----+-----+-----+-----+
-----+
|location|bodypart|situation|fast_break|shot_quality|quality_label|is_goal|
+-----+-----+-----+-----+-----+-----+-----+
-----+
|      15|      1|      1|      0|      0.5|      mediu|0|
|      9|      1|      1|      0|      0.65|      bun|0|
|      15|      2|      1|      0|      0.5|      mediu|0|
|      9|      1|      1|      0|      0.65|      bun|0|
|      3|      2|      1|      0|      0.85|    excelent|0|
|      11|      1|      1|      0|      0.65|      bun|0|
|      3|      1|      1|      0|      0.85|    excelent|0|
|      15|      1|      1|      0|      0.5|      mediu|0|
|      3|      1|      1|      0|      0.85|    excelent|0|
|      15|      2|      1|      0|      0.5|      mediu|0|
+-----+-----+-----+-----+-----+-----+-----+
-----+
only showing top 10 rows

```

Înregistrarea UDF-ului în Spark SQL

`spark.udf.register` face funcțiile apelabile direct din interogări SQL, nu doar din DataFrame API. Analizăm apoi rata reală de conversie per categorie de calitate — validarea empirică a scorului: dacă șuturile „excelente” convertesc mult peste cele „slabe”, UDF-ul capturează un semnal real.

```

# Înregistrăm UDF-ul în Spark SQL pentru utilizare în interogări
spark.udf.register('shot_quality_score', shot_quality_score)
spark.udf.register('shot_quality_label', shot_quality_label)

```

```
shots_with_quality.createOrReplaceTempView('shots_quality')

# Analiză prin Spark SQL: rata de conversie pe categoria de calitate a șutului
quality_analysis = spark.sql("""
    SELECT
        quality_label,
        COUNT(*) AS total_suturi,
        SUM(is_goal) AS goluri,
        ROUND(AVG(shot_quality), 3) AS scor_mediu,
        ROUND(SUM(is_goal)*100.0/COUNT(*), 2) AS rata_conversie_pct
    FROM shots_quality
    GROUP BY quality_label
    ORDER BY scor_mediu DESC
""")

print('Analiza ratei de conversie pe categoria de calitate:')
quality_analysis.show()
Analiza ratei de conversie pe categoria de calitate:
+-----+-----+-----+-----+-----+
|quality_label|total_suturi|goluri|scor_mediu|rata_conversie_pct|
+-----+-----+-----+-----+-----+
|      excelent|      65980|  11831|      0.822|      17.93|
|         bun|      57958|   8239|      0.654|      14.22|
|        mediu|     104923|   4361|      0.49|       4.16|
|         slab|       274|    10|      0.28|       3.65|
+-----+-----+-----+-----+-----+

import matplotlib
import matplotlib.pyplot as plt

qa_pd = quality_analysis.toPandas()
# ROUND() din Spark SQL întoarce decimal.Decimal – îl convertim la float
# pentru aritmetica cu float din adnotările matplotlib (val + 0.3)
qa_pd['rata_conversie_pct'] =
qa_pd['rata_conversie_pct'].astype(float)
order = ['excelent', 'bun', 'mediu', 'slab']
qa_pd =
qa_pd.set_index('quality_label').reindex(order).reset_index()

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

colors = ['#1a9641', '#a6d96a', '#fdae61', '#d7191c']

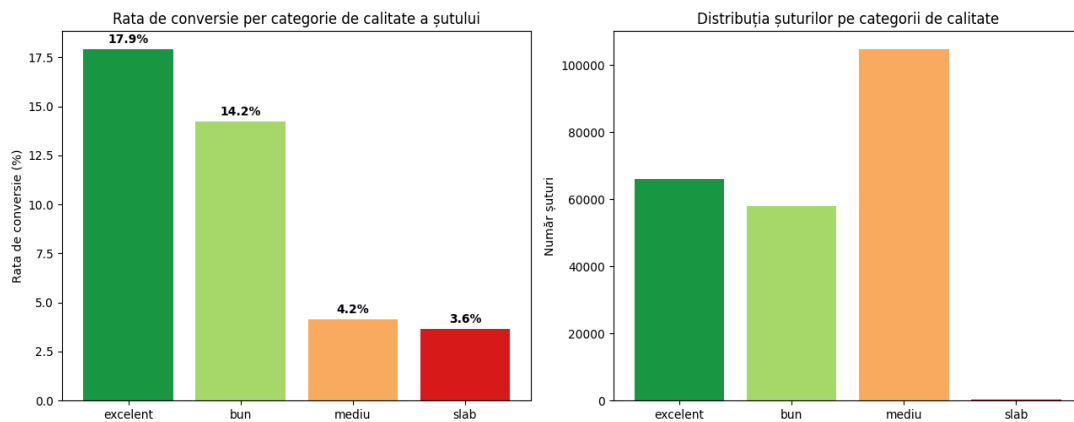
axes[0].bar(qa_pd['quality_label'], qa_pd['rata_conversie_pct'],
color=colors)
for i, (label, val) in enumerate(zip(qa_pd['quality_label'],
qa_pd['rata_conversie_pct'])):
    axes[0].text(i, val + 0.3, f'{val:.1f}%', ha='center',
fontweight='bold')
axes[0].set_ylabel('Rata de conversie (%)')
axes[0].set_title('Rata de conversie per categorie de calitate a
șutului')
```

```

axes[1].bar(qa_pd['quality_label'], qa_pd['total_suturi'],
color=colors)
axes[1].set_ylabel('Număr şuturi')
axes[1].set_title('Distribuția şuturilor pe categorii de calitate')

plt.tight_layout()
plt.savefig('plots/fig_shot_quality_udf.png', dpi=120,
bbox_inches='tight')
plt.show()

```



<Figure size 1300x500 with 2 Axes>

5.2 Pandas UDF vectorizat (@pandas_udf) + benchmark

Rescriem aceeaşi logică de scor sub forma unui **Pandas UDF** (vectorizat) şi o comparăm cu UDF-ul standard de mai sus.

```

# Aceeaşi logică de scor, dar VECTORIZATĂ ca Pandas UDF (procesează
un batch întreg, nu un rând)
@pandas_udf(FloatType())
def shot_quality_score_vec(location: pd.Series, bodypart: pd.Series,
                           situation: pd.Series, fast_break:
pd.Series) -> pd.Series:
    # Toate argumentele sosesc ca pandas.Series (un lot de rânduri),
    prin Apache Arrow
    loc = location.to_numpy()
    bp = bodypart.to_numpy()
    sit = situation.to_numpy()
    fb = fast_break.to_numpy()

    score = np.zeros(len(loc), dtype='float64')

    # Factor 1: locație (40%)
    central = np.isin(loc, [3, 5, 10, 12, 13, 14])
    good = np.isin(loc, [4, 6, 9, 11])
    score += np.where(central, 0.40, np.where(good, 0.20, 0.05))

```

```

    # Factor 2: parte corp (25%) – doar dacă nu e null
    bp_ok = ~pd.isna(bp)
    score += np.where(bp_ok, np.where(np.isin(bp, [1, 2]), 0.25,
0.15), 0.0)

    # Factor 3: situație (20%) – doar dacă nu e null
    sit_ok = ~pd.isna(sit)
    score += np.where(sit_ok, np.where(sit == 1, 0.20, np.where(sit
== 4, 0.15, 0.08)), 0.0)

    # Factor 4: contraatac rapid (15%)
    fb_ok = ~pd.isna(fb)
    score += np.where(fb_ok & (fb == 1), 0.15, 0.0)

    score = np.minimum(score, 1.0)
    # Locație null => scor 0.0 (identic cu comportamentul UDF-ului
standard)
    score = np.where(~pd.isna(loc), score, 0.0)
    return pd.Series(score, dtype='float32')

print('Pandas UDF înregistrat: shot_quality_score_vec')
Pandas UDF înregistrat: shot_quality_score_vec
import time
from pyspark.sql.functions import avg as spark_avg

args = (col('location'), col('bodypart'), col('situation'),
col('fast_break'))

# 1) Verificare corectitudine: ambele variante trebuie să dea EXACT
același scor
check = (shots_df.limit(3000)
        .withColumn('q_udf', shot_quality_score(*args))
        .withColumn('q_pudf', shot_quality_score_vec(*args))
        .select('q_udf', 'q_pudf').toPandas())
max_diff = (check['q_udf'] - check['q_pudf']).abs().max()
print(f'Diferența maximă UDF vs Pandas UDF (3000 rânduri):
{max_diff:.6f}')
print('=> Rezultate identice.' if max_diff < 1e-5 else '=> ATENȚIE:
diferențe între variante!')

# 2) Warmup pentru ambele căi (pornirea workerilor Python + codegen)
– benchmark corect
shots_df.withColumn('q',
shot_quality_score(*args)).agg(spark_avg('q')).collect()
shots_df.withColumn('q',
shot_quality_score_vec(*args)).agg(spark_avg('q')).collect()

def timp(expr_udf):
    t0 = time.time()
    val = shots_df.withColumn('q',
expr_udf).agg(spark_avg('q')).collect()[0][0]
    return time.time() - t0, val

```

```

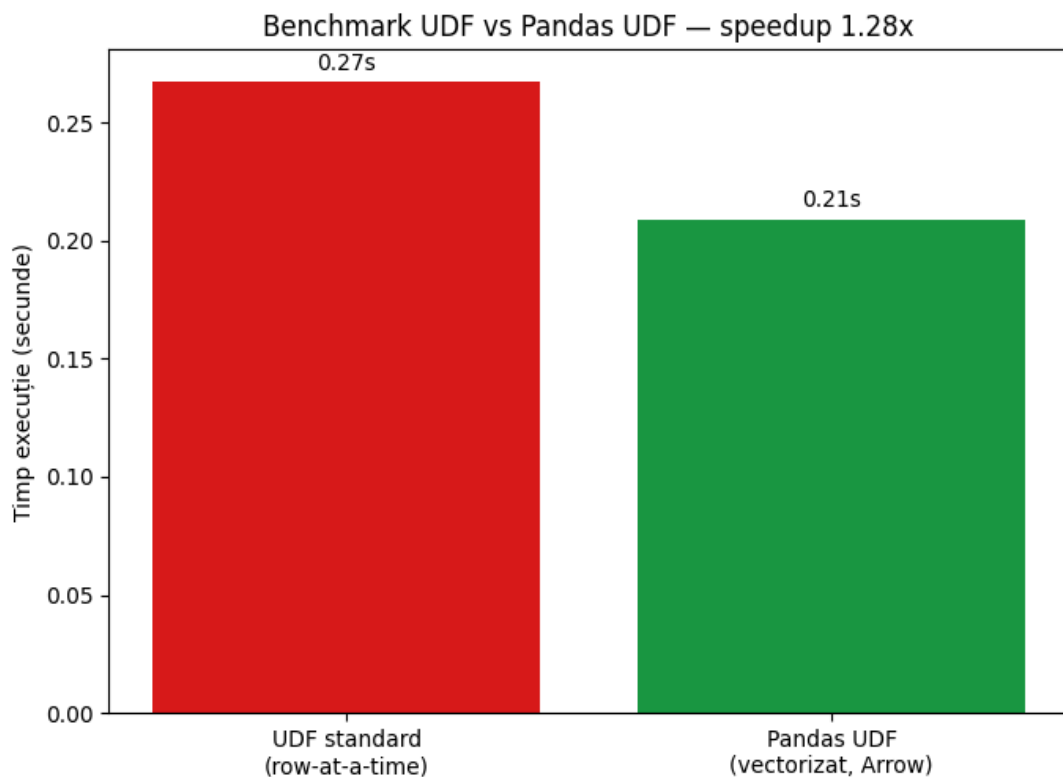
t_udf, v_udf = timp(shot_quality_score(*args))
t_pudf, v_pudf = timp(shot_quality_score_vec(*args))

print(f'\nUDF standard (row-at-a-time): {t_udf:.2f}s | scor mediu={v_udf:.4f}')
print(f'Pandas UDF (vectorizat)      : {t_pudf:.2f}s | scor mediu={v_pudf:.4f}')
speedup = t_udf / t_pudf if t_pudf > 0 else float('nan')
print(f'Speedup Pandas UDF: {speedup:.2f}x')

fig, ax = plt.subplots(figsize=(7, 5))
bars = ax.bar(['UDF standard\n(row-at-a-time)', 'Pandas UDF\n(vectorizat, Arrow)'],
              [t_udf, t_pudf], color=['#d7191c', '#1a9641'])
ax.bar_label(bars, fmt='%.2fs', padding=3)
ax.set_ylabel('Timp execuție (secunde)')
ax.set_title(f'Benchmark UDF vs Pandas UDF — speedup {speedup:.2f}x')
plt.tight_layout()
plt.savefig('plots/fig_udf_benchmark.png', dpi=120,
           bbox_inches='tight')
plt.show()
Diferența maximă UDF vs Pandas UDF (3000 rânduri): 0.000000
=> Rezultate identice.

UDF standard (row-at-a-time): 0.27s | scor mediu=0.6270
Pandas UDF (vectorizat)      : 0.21s | scor mediu=0.6270
Speedup Pandas UDF: 1.28x

```



<Figure size 700x500 with 1 Axes>

5.3 Optimizarea Hiperparametrilor — CrossValidator

Motivație

Parametrii implicați ai unui model sunt rareori optimi pentru un dataset specific. `CrossValidator` cu `ParamGridBuilder` caută sistematic cea mai bună combinație de hiperparametri prin **k-fold cross-validation**, evitând overfitting-ul față de setul de test.

Parametrii explorați pentru Random Forest

- `numTrees` — numărul de arbori din pădure
- `maxDepth` — adâncimea maximă a fiecărui arbore
- `minInstancesPerNode` — numărul minim de exemple dintr-un nod frunză

```
# Adăugăm shot_quality ca feature suplimentar (beneficiind de UDF)
feature_cols = ['location', 'bodypart', 'situation',
               'assist_method',
               'fast_break', 'shot_place', 'side', 'time',
               'shot_quality']

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol='features',
    handleInvalid='skip'
)

rf = RandomForestClassifier(
    labelCol='is_goal',
    featuresCol='features',
    seed=42
)

pipeline = Pipeline(stages=[assembler, rf])

# Grid de hiperparametri
param_grid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [50, 100]) \
    .addGrid(rf.maxDepth, [5, 10]) \
    .addGrid(rf.minInstancesPerNode, [1, 5]) \
    .build()

evaluator = BinaryClassificationEvaluator(
    labelCol='is_goal',
    metricName='areaUnderROC'
)
```



```

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=evaluator,
    numFolds=3,
    parallelism=2,
    seed=42
)

n_combinations = len(param_grid)
print(f'Grila de parametri: {n_combinations} combinații × 3 folds = {n_combinations*3} antrenări')
Grila de parametri: 8 combinații × 3 folds = 24 antrenări

```

Rularea CrossValidator — 24 de antrenări

Grid-ul are 8 combinații ($2 \times 2 \times 2$), fiecare evaluată cu 3-fold cross-validation → 24 de modele antrenate. De aceea folosim un sample de 30% din date: scopul e *compararea* hiperparametrilor, nu modelul final de producție, iar pe sample comparația rămâne validă.

```

# Folosim un sample pentru a reduce timpul de rulare la ~10 min
sample_df = shots_with_quality.sample(fraction=0.3, seed=42)
train_df, test_df = sample_df.randomSplit([0.8, 0.2], seed=42)

print(f'Train: {train_df.count():,} | Test: {test_df.count():,}')
print('Rulare CrossValidator (poate dura câteva minute)...')

cv_model = cv.fit(train_df)
print('CrossValidator finalizat.')
Train: 55,117 | Test: 13,566
Rulare CrossValidator (poate dura câteva minute)...
CrossValidator finalizat.

```

Extragerea celui mai bun model și analiza grid-ului

`cv_model.bestModel` e pipeline-ul reantrenat automat pe *tot* train-ul cu combinația câștigătoare. Afișăm hiperparametrii câștigători, evaluăm pe test (date nevăzute de CV) și listăm AUC-ul mediu per combinație din `avgMetrics` — ca să vedem nu doar cine a câștigat, ci și cât de sensibil e modelul la fiecare hiperparametru.

```

# Cel mai bun model
best_rf = cv_model.bestModel.stages[-1]

print('=== Cel mai bun model găsit ===')
print(f' numTrees           : {best_rf.getNumTrees()}')
print(f' maxDepth           : {best_rf.getOrDefault("maxDepth")}')
print(f' minInstancesPerNode : {best_rf.getOrDefault("minInstancesPerNode")}')

```

```

best_preds = cv_model.transform(test_df)
best_auc = evaluator.evaluate(best_preds)

mc_eval = MulticlassClassificationEvaluator(labelCol='is_goal',
metricName='f1')
best_f1 = mc_eval.evaluate(best_preds)

print(f'\n AUC-ROC pe test : {best_auc:.4f}')
print(f' F1-Score pe test: {best_f1:.4f}')
=== Cel mai bun model găsit ===
    numTrees          : 50
    maxDepth          : 10
    minInstancesPerNode : 1

    AUC-ROC pe test : 0.9576
    F1-Score pe test: 0.9254
# Compararea scorurilor AUC pentru toate combinațiile din grid
avg_metrics = cv_model.avgMetrics

results = []
for params, score in zip(param_grid, avg_metrics):
    row = {p.name: v for p, v in params.items()}
    row['auc_cv'] = round(score, 4)
    results.append(row)

results_df = pd.DataFrame(results).sort_values('auc_cv',
ascending=False)
print('Toate combinațiile testate (ordonate după AUC-ROC):')
print(results_df.to_string(index=False))
Toate combinațiile testate (ordonate după AUC-ROC):
  numTrees  maxDepth  minInstancesPerNode  auc_cv
        50         10                   1  0.9557
       100         10                   1  0.9544
       100         10                   5  0.9541
        50         10                   5  0.9536
        50          5                   5  0.9129
        50          5                   1  0.9123
       100          5                   1  0.9059
       100          5                   5  0.9043

fig, ax = plt.subplots(figsize=(11, 5))
labels = [f'nT={r["numTrees"]}', mD={r["maxDepth"]}, mI=
{r["minInstancesPerNode"]}']
    for _, r in results_df.iterrows()
colors_cv = ['#d73027' if i == 0 else '#91bfdb' for i in
range(len(results_df))]

bars = ax.barh(labels[:-1], results_df['auc_cv'][:-1],
color=colors_cv[:-1])
ax.bar_label(bars, fmt='{:.4f}', padding=3, fontsize=9)
ax.set_xlabel('AUC-ROC (cross-validation, 3 folds)')
ax.set_title('Compararea combinațiilor de hiperparametri – Random
Forest')
ax.set_xlim(results_df['auc_cv'].min() - 0.01,

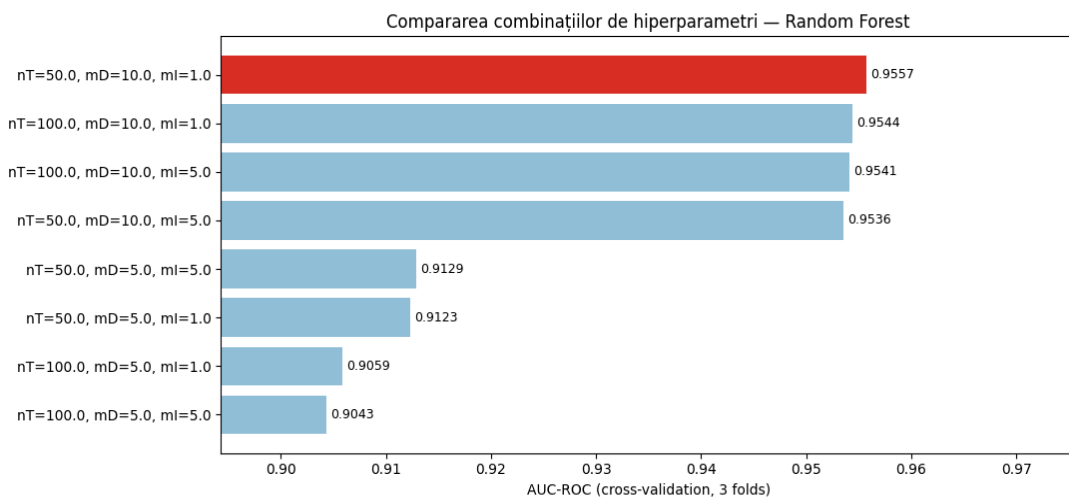
```

```

results_df['auc_cv'].max() + 0.02)

plt.tight_layout()
plt.savefig('plots/fig_hyperparameter_tuning.png', dpi=120,
bbox_inches='tight')
plt.show()

spark.stop()
print('Sesiune Spark oprită.')
```



<Figure size 1100x500 with 1 Axes>
Sesiune Spark oprită.

Cerința 6 — Deep Learning cu TensorFlow

Enunțul problemei

Construim o **rețea neurală feed-forward** (Fully Connected / Dense) pentru clasificarea binară a tentativelor de șut:

- **Input:** vectorul de caracteristici al unui șut (locație, parte corp, situație, assist, etc.)
- **Output:** probabilitatea că șutul este gol

Justificarea alegerii — Rețea neurală densă

- Rețelele neuronale pot **captura interacțiuni non-liniare complexe** între features (ex: combinația „colțul careului + contraatac + picior stâng” poate fi mult mai mult decât suma componentelor)
- Spre deosebire de Random Forest care combină arbori independenți, rețeaua neurală **învață reprezentări ierarhice** ale datelor

- Permite **compararea directă** cu modelul MLlib din Cerința 3 pe aceleași date
- **Batch Normalization** și **Dropout** combat overfitting-ul pe dataset-ul dezechilibrat

Workflow

```
Spark DataFrame → toPandas() → Preprocessing (sklearn) → TF/Keras
model → Evaluare → Comparație cu RF MLlib

import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3' # supress TF warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.figsize'] = (12, 5)
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve, f1_score, accuracy_score
)

print(f'TensorFlow: {tf.__version__}')
print(f'Keras: {keras.__version__}')
print(f'GPU disponibil: {len(tf.config.list_physical_devices("GPU"))
> 0}')
TensorFlow: 2.16.2
Keras: 3.12.2
GPU disponibil: False
```

6.1 Pregătirea datelor

Folosim Spark doar pentru citirea și preprocesarea inițială, apoi transferăm în Pandas pentru antrenarea cu TensorFlow.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName('FootballDL') \
    .config('spark.driver.memory', '4g') \
```

```

        .config('spark.sql.shuffle.partitions', '8') \
        .getOrCreate()
spark.sparkContext.setLogLevel('ERROR')

# Citim datele cu Spark
if os.path.exists('data/processed_shots'):
    shots_spark = spark.read.parquet('data/processed_shots')
else:
    shots_spark = spark.read.csv('data/events.csv', header=True,
    inferSchema=True, nullValue='NA') \
        .filter((col('event_type') == 1) &
col('is_goal').isNotNull() &
                col('location').isNotNull() &
col('bodypart').isNotNull() &
                col('situation').isNotNull()) \
        .fillna({'assist_method': 0, 'fast_break': 0, 'shot_place':
0, 'side': 1})

feature_cols = ['location', 'bodypart', 'situation',
'assist_method',
                'fast_break', 'shot_place', 'side', 'time']

# Selectăm coloanele necesare și transferăm în Pandas
shots_pd = shots_spark.select(feature_cols +
['is_goal']).dropna().toPandas()
spark.stop()

print(f'Dataset: {len(shots_pd):,} rânduri, {shots_pd.shape[1]}
coloane')
print(f'Distribuție target: gol={shots_pd["is_goal"].sum():,}
({shots_pd["is_goal"].mean()*100:.1f}%), '
      f'non-gol={({shots_pd["is_goal"]==0).sum():,}')
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
26/06/11 21:35:27 WARN NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
Dataset: 229,135 rânduri, 9 coloane
Distribuție target: gol=24,441 (10.7%), non-gol=204,694

```

Split stratificat, normalizare și ponderi de clasă

Trei pregătiri esențiale pentru o rețea neuronală: **(1)** split 70/15/15 *stratificat* — păstrăm proporția de ~10% goluri în toate seturile; **(2)** `StandardScaler` (medie 0, deviație 1) — gradient descent converge prost pe features cu scale diferite, spre deosebire de arborii din C3 care sunt invarianți la scală; **(3)** `class_weight` — erorile pe goluri (clasa minoritară) cântăresc de ~9x mai mult în funcția de pierdere.

```

X = shots_pd[feature_cols].values.astype(np.float32)
y = shots_pd['is_goal'].values.astype(np.float32)

# Split: 70% train, 15% val, 15% test
X_train, X_temp, y_train, y_temp = train_test_split(X, y,
test_size=0.30, random_state=42, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
test_size=0.50, random_state=42, stratify=y_temp)

# Normalizare
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

# Pondere pentru clase dezechilibrate
neg, pos = (y_train == 0).sum(), (y_train == 1).sum()
class_weight = {0: 1.0, 1: neg / pos}

print(f'Train: {len(X_train):,} | Val: {len(X_val):,} | Test: {len(X_test):,}')
print(f'Class weight - 0: {class_weight[0]:.1f}, 1: {class_weight[1]:.1f} (compensare dezechilibru)')
Train: 160,394 | Val: 34,370 | Test: 34,371
Class weight - 0: 1.0, 1: 8.4 (compensare dezechilibru)

```

6.2 Arhitectura rețelei neuronale

```

Input (8 features)
↓
Dense(256, ReLU) + BatchNorm + Dropout(0.4)
↓
Dense(128, ReLU) + BatchNorm + Dropout(0.3)
↓
Dense(64, ReLU) + Dropout(0.2)
↓
Dense(1, Sigmoid) → P(gol)
tf.random.set_seed(42)
np.random.seed(42)

n_features = X_train.shape[1]

model = keras.Sequential([
    layers.Input(shape=(n_features,)),

    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.4),

    layers.Dense(128, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),

```

```

layers.Dense(64, activation='relu'),
layers.Dropout(0.2),

layers.Dense(1, activation='sigmoid')
], name='FootballGoalPredictor')

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=[
        'accuracy',
        keras.metrics.AUC(name='auc'),
        keras.metrics.Precision(name='precision'),
        keras.metrics.Recall(name='recall')
    ]
)

```

```
model.summary()
```

```
[1mModel: "FootballGoalPredictor"[0m
```

[1m [0m[1mLayer (type)	[0m[1m [0m[1m [1m		
[0m[1mOutput Shape	[0m[1m [0m[1m [1m [0m[1m	Param # [0m[1m	
[0m[1m			
dense ([38;5;33mDense[0m)	([38;5;45mNone[0m,		
[38;5;34m256[0m)	[38;5;34m2,304[0m		
batch_normalization	([38;5;45mNone[0m,		
[38;5;34m256[0m)	[38;5;34m1,024[0m		
([38;5;33mBatchNormalization[0m)			
dropout ([38;5;33mDropout[0m)	([38;5;45mNone[0m,		
[38;5;34m256[0m)	[38;5;34m0[0m		
dense_1 ([38;5;33mDense[0m)	([38;5;45mNone[0m,		
[38;5;34m128[0m)	[38;5;34m32,896[0m		
batch_normalization_1	([38;5;45mNone[0m,		
[38;5;34m128[0m)	[38;5;34m512[0m		
([38;5;33mBatchNormalization[0m)			
dropout_1 ([38;5;33mDropout[0m)	([38;5;45mNone[0m,		
[38;5;34m128[0m)	[38;5;34m0[0m		

```

| dense_2 ([38;5;33mDense[0m) | ([38;5;45mNone[0m,
[38;5;34m64[0m) | [38;5;34m8,256[0m |
| dropout_2 ([38;5;33mDropout[0m) | ([38;5;45mNone[0m,
[38;5;34m64[0m) | [38;5;34m0[0m |
| dense_3 ([38;5;33mDense[0m) | ([38;5;45mNone[0m,
[38;5;34m1[0m) | [38;5;34m65[0m |

[1m Total params: [0m[38;5;34m45,057[0m (176.00 KB)
[1m Trainable params: [0m[38;5;34m44,289[0m (173.00 KB)
[1m Non-trainable params: [0m[38;5;34m768[0m (3.00 KB)

```

6.3 Antrenarea modelului

```

callbacks = [
    keras.callbacks.EarlyStopping(
        monitor='val_auc',
        patience=6,
        restore_best_weights=True,
        mode='max',
        verbose=1
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        min_lr=1e-6,
        verbose=1
    )
]

history = model.fit(
    X_train, y_train,
    epochs=40,
    batch_size=512,
    validation_data=(X_val, y_val),
    class_weight=class_weight,
    callbacks=callbacks,
    verbose=1
)

print(f'\nAntrenare finalizată după {len(history.history["loss"])}
epoci.')
Epoch 1/40
[1m 1/314[0m [37m[0m [1m3:35[0m 688ms/step -
accuracy: 0.6348 - auc: 0.5089 - loss: 1.6513 - precision: 0.0964 -
recall: 0.3019

```



```

[1m 22/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6485 - auc: 0.5638 - loss: 1.4976 - precision: 0.1303
- recall: 0.4291
[1m 42/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6437 - auc: 0.5894 - loss: 1.4365 - precision: 0.1405
- recall: 0.4794
[1m 63/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6434 - auc: 0.6043 - loss: 1.3952 - precision: 0.1466
- recall: 0.5060
[1m 84/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6437 - auc: 0.6163 - loss: 1.3660 - precision: 0.1518
- recall: 0.5263
[1m106/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6443 - auc: 0.6265 - loss: 1.3418 - precision: 0.1561
- recall: 0.5418
[1m128/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6458 - auc: 0.6354 - loss: 1.3206 - precision: 0.1595
- recall: 0.5537
[1m149/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6476 - auc: 0.6430 - loss: 1.3032 - precision: 0.1626
- recall: 0.5630
[1m173/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6497 - auc: 0.6511 - loss: 1.2854 - precision: 0.1659
- recall: 0.5727
[1m197/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6522 - auc: 0.6589 - loss: 1.2687 - precision: 0.1690
- recall: 0.5813
[1m221/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6551 - auc: 0.6662 - loss: 1.2528 - precision: 0.1720
- recall: 0.5889
[1m245/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6582 - auc: 0.6734 - loss: 1.2378 - precision: 0.1751
- recall: 0.5962
[1m269/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6612 - auc: 0.6802 - loss: 1.2236 - precision: 0.1780
- recall: 0.6031
[1m293/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.6643 - auc: 0.6867 - loss: 1.2102 - precision: 0.1809
- recall: 0.6095
[1m314/314[0m [32m[0m[37m[0m[1m2s[0m 3ms/step
- accuracy: 0.7054 - auc: 0.7719 - loss: 1.0368 - precision: 0.2205
- recall: 0.6947 - val_accuracy: 0.7742 - val_auc: 0.8814 -
val_loss: 0.4354 - val_precision: 0.2989 - val_recall: 0.8301 -
learning_rate: 0.0010
Epoch 2/40
[1m 1/314[0m [37m[0m[1m3s[0m 10ms/step -
accuracy: 0.7793 - auc: 0.8563 - loss: 0.8245 - precision: 0.2917 -
recall: 0.7925
[1m 19/314[0m [32m[0m[37m[0m[1m0s[0m 3ms/step
- accuracy: 0.7802 - auc: 0.8676 - loss: 0.7944 - precision: 0.2917
- recall: 0.8022
[1m 33/314[0m [32m[0m[37m[0m[1m0s[0m 3ms/step
- accuracy: 0.7797 - auc: 0.8678 - loss: 0.7977 - precision: 0.2935
- recall: 0.8019

```

```
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.7792 - auc: 0.8690 - loss: 0.7951 - precision: 0.2938
- recall: 0.8045
[1m 71/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.7807 - auc: 0.8709 - loss: 0.7921 - precision: 0.2970
- recall: 0.8066
[1m 93/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.7812 - auc: 0.8727 - loss: 0.7890 - precision: 0.2994
- recall: 0.8086
[1m116/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.7822 - auc: 0.8746 - loss: 0.7848 - precision: 0.3019
- recall: 0.8111
[1m140/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.7835 - auc: 0.8765 - loss: 0.7797 - precision: 0.3043
- recall: 0.8137
[1m164/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.7848 - auc: 0.8783 - loss: 0.7748 - precision: 0.3067
- recall: 0.8164
[1m189/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.7862 - auc: 0.8802 - loss: 0.7697 - precision: 0.3090
- recall: 0.8190
[1m214/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.7876 - auc: 0.8819 - loss: 0.7644 - precision: 0.3111
- recall: 0.8212
[1m238/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.7889 - auc: 0.8834 - loss: 0.7595 - precision: 0.3129
- recall: 0.8232
[1m262/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.7901 - auc: 0.8849 - loss: 0.7547 - precision: 0.3147
- recall: 0.8251
[1m287/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.7913 - auc: 0.8863 - loss: 0.7499 - precision: 0.3163
- recall: 0.8268
[1m311/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.7923 - auc: 0.8876 - loss: 0.7455 - precision: 0.3178
- recall: 0.8284
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 3ms/step
- accuracy: 0.8050 - auc: 0.9038 - loss: 0.6902 - precision: 0.3362
- recall: 0.8496 - val_accuracy: 0.8089 - val_auc: 0.9381 -
val_loss: 0.3718 - val_precision: 0.3545 - val_recall: 0.9643 -
learning_rate: 0.0010
Epoch 3/40
[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.7988 - auc: 0.9126 - loss: 0.6622 - precision: 0.3288 -
recall: 0.9057
[1m 24/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8250 - auc: 0.9282 - loss: 0.5820 - precision: 0.3595
- recall: 0.8967
[1m 46/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8232 - auc: 0.9271 - loss: 0.5856 - precision: 0.3584
- recall: 0.8927
[1m 54/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8231 - auc: 0.9269 - loss: 0.5865 - precision: 0.3584
- recall: 0.8911
```

```

[1m 69/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8227 - auc: 0.9266 - loss: 0.5881 - precision: 0.3588
- recall: 0.8899
[1m 90/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8218 - auc: 0.9263 - loss: 0.5903 - precision: 0.3591
- recall: 0.8894
[1m 111/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8214 - auc: 0.9262 - loss: 0.5911 - precision: 0.3597
- recall: 0.8894
[1m 133/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8212 - auc: 0.9262 - loss: 0.5912 - precision: 0.3599
- recall: 0.8894
[1m 155/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8210 - auc: 0.9263 - loss: 0.5910 - precision: 0.3603
- recall: 0.8898
[1m 177/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8208 - auc: 0.9264 - loss: 0.5905 - precision: 0.3606
- recall: 0.8902
[1m 200/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8208 - auc: 0.9265 - loss: 0.5897 - precision: 0.3609
- recall: 0.8905
[1m 221/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8207 - auc: 0.9266 - loss: 0.5888 - precision: 0.3609
- recall: 0.8908
[1m 243/314[0m [32m[0m[37m[0m [1m0s[0m 3ms/step
- accuracy: 0.8207 - auc: 0.9268 - loss: 0.5879 - precision: 0.3610
- recall: 0.8910
[1m 267/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8207 - auc: 0.9269 - loss: 0.5868 - precision: 0.3611
- recall: 0.8914
[1m 291/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8207 - auc: 0.9271 - loss: 0.5856 - precision: 0.3612
- recall: 0.8918
[1m 314/314[0m [32m[0m[37m[0m [1m1s[0m 3ms/step
- accuracy: 0.8205 - auc: 0.9293 - loss: 0.5718 - precision: 0.3623
- recall: 0.8980 - val_accuracy: 0.8126 - val_auc: 0.9531 -
val_loss: 0.3145 - val_precision: 0.3615 - val_recall: 0.9880 -
learning_rate: 0.0010
Epoch 4/40
[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8281 - auc: 0.9299 - loss: 0.5724 - precision: 0.3684 -
recall: 0.9245
[1m 24/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8334 - auc: 0.9384 - loss: 0.5269 - precision: 0.3726
- recall: 0.9063
[1m 47/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8308 - auc: 0.9380 - loss: 0.5274 - precision: 0.3705
- recall: 0.9055
[1m 71/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8289 - auc: 0.9373 - loss: 0.5303 - precision: 0.3695
- recall: 0.9059
[1m 95/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8268 - auc: 0.9368 - loss: 0.5326 - precision: 0.3685
- recall: 0.9079

```

```
[1m119/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8257 - auc: 0.9366 - loss: 0.5335 - precision: 0.3681
- recall: 0.9096
[1m143/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8250 - auc: 0.9366 - loss: 0.5334 - precision: 0.3680
- recall: 0.9110
[1m167/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8246 - auc: 0.9366 - loss: 0.5333 - precision: 0.3681
- recall: 0.9122
[1m192/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8243 - auc: 0.9367 - loss: 0.5329 - precision: 0.3682
- recall: 0.9133
[1m217/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8240 - auc: 0.9367 - loss: 0.5323 - precision: 0.3680
- recall: 0.9143
[1m242/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8238 - auc: 0.9368 - loss: 0.5316 - precision: 0.3680
- recall: 0.9151
[1m267/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8237 - auc: 0.9369 - loss: 0.5308 - precision: 0.3680
- recall: 0.9159
[1m291/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8237 - auc: 0.9371 - loss: 0.5301 - precision: 0.3680
- recall: 0.9165
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8230 - auc: 0.9384 - loss: 0.5206 - precision: 0.3686
- recall: 0.9250 - val_accuracy: 0.8159 - val_auc: 0.9550 -
val_loss: 0.2898 - val_precision: 0.3650 - val_recall: 0.9809 -
learning_rate: 0.0010
Epoch 5/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8203 - auc: 0.9471 - loss: 0.4712 - precision: 0.3636 -
recall: 0.9811
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8303 - auc: 0.9468 - loss: 0.4799 - precision: 0.3707
- recall: 0.9303
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8290 - auc: 0.9453 - loss: 0.4851 - precision: 0.3708
- recall: 0.9308
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8282 - auc: 0.9446 - loss: 0.4885 - precision: 0.3714
- recall: 0.9309
[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8269 - auc: 0.9441 - loss: 0.4913 - precision: 0.3712
- recall: 0.9318
[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8261 - auc: 0.9437 - loss: 0.4926 - precision: 0.3712
- recall: 0.9330
[1m147/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8256 - auc: 0.9436 - loss: 0.4932 - precision: 0.3712
- recall: 0.9339
[1m171/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8252 - auc: 0.9435 - loss: 0.4937 - precision: 0.3714
- recall: 0.9347
```

```
[1m196/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8250 - auc: 0.9434 - loss: 0.4938 - precision: 0.3715
- recall: 0.9355
[1m221/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8249 - auc: 0.9434 - loss: 0.4938 - precision: 0.3715
- recall: 0.9360
[1m246/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8247 - auc: 0.9434 - loss: 0.4936 - precision: 0.3714
- recall: 0.9364
[1m271/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8246 - auc: 0.9434 - loss: 0.4933 - precision: 0.3714
- recall: 0.9369
[1m296/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8246 - auc: 0.9435 - loss: 0.4931 - precision: 0.3714
- recall: 0.9372
[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8240 - auc: 0.9437 - loss: 0.4902 - precision: 0.3717
- recall: 0.9414 - val_accuracy: 0.8210 - val_auc: 0.9566 -
val_loss: 0.2788 - val_precision: 0.3711 - val_recall: 0.9763 -
learning_rate: 0.0010
Epoch 6/40
[1m 1/314[0m [37m-----[0m [1m3s[0m 10ms/step -
accuracy: 0.8223 - auc: 0.9481 - loss: 0.4698 - precision: 0.3623 -
recall: 0.9434
[1m 25/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8264 - auc: 0.9483 - loss: 0.4670 - precision: 0.3665
- recall: 0.9437
[1m 49/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8259 - auc: 0.9471 - loss: 0.4739 - precision: 0.3675
- recall: 0.9419
[1m 74/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8255 - auc: 0.9468 - loss: 0.4753 - precision: 0.3688
- recall: 0.9426
[1m 98/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8249 - auc: 0.9467 - loss: 0.4762 - precision: 0.3696
- recall: 0.9437
[1m122/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8246 - auc: 0.9466 - loss: 0.4764 - precision: 0.3703
- recall: 0.9447
[1m147/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8245 - auc: 0.9465 - loss: 0.4764 - precision: 0.3708
- recall: 0.9456
[1m172/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8244 - auc: 0.9465 - loss: 0.4765 - precision: 0.3714
- recall: 0.9462
[1m197/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8243 - auc: 0.9466 - loss: 0.4764 - precision: 0.3717
- recall: 0.9467
[1m221/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8243 - auc: 0.9466 - loss: 0.4762 - precision: 0.3718
- recall: 0.9471
[1m245/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8243 - auc: 0.9467 - loss: 0.4758 - precision: 0.3719
- recall: 0.9475
```

```

[1m270/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8243 - auc: 0.9467 - loss: 0.4755 - precision: 0.3720
- recall: 0.9477
[1m294/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8243 - auc: 0.9468 - loss: 0.4753 - precision: 0.3721
- recall: 0.9479
[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8246 - auc: 0.9473 - loss: 0.4726 - precision: 0.3735
- recall: 0.9512 - val_accuracy: 0.8093 - val_auc: 0.9584 -
val_loss: 0.2806 - val_precision: 0.3578 - val_recall: 0.9918 -
learning_rate: 0.0010
Epoch 7/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8164 - auc: 0.9479 - loss: 0.5022 - precision: 0.3566 -
recall: 0.9623
[1m 25/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8277 - auc: 0.9538 - loss: 0.4474 - precision: 0.3696
- recall: 0.9561
[1m 49/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8273 - auc: 0.9523 - loss: 0.4520 - precision: 0.3703
- recall: 0.9506
[1m 73/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8270 - auc: 0.9514 - loss: 0.4558 - precision: 0.3713
- recall: 0.9481
[1m 97/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8262 - auc: 0.9509 - loss: 0.4585 - precision: 0.3719
- recall: 0.9481
[1m122/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8260 - auc: 0.9506 - loss: 0.4598 - precision: 0.3726
- recall: 0.9487
[1m146/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8260 - auc: 0.9505 - loss: 0.4602 - precision: 0.3733
- recall: 0.9492
[1m170/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8260 - auc: 0.9504 - loss: 0.4603 - precision: 0.3739
- recall: 0.9497
[1m194/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8261 - auc: 0.9504 - loss: 0.4601 - precision: 0.3745
- recall: 0.9501
[1m219/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8262 - auc: 0.9504 - loss: 0.4600 - precision: 0.3747
- recall: 0.9504
[1m243/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8262 - auc: 0.9504 - loss: 0.4598 - precision: 0.3749
- recall: 0.9508
[1m265/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8263 - auc: 0.9504 - loss: 0.4596 - precision: 0.3751
- recall: 0.9511
[1m289/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8264 - auc: 0.9505 - loss: 0.4593 - precision: 0.3753
- recall: 0.9514
[1m314/314[0m [32m-[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8265 - auc: 0.9505 - loss: 0.4591 - precision: 0.3756
- recall: 0.9517

```

[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8282 - auc: 0.9506 - loss: 0.4572 - precision: 0.3788
- recall: 0.9550 - val_accuracy: 0.8269 - val_auc: 0.9594 -
val_loss: 0.2731 - val_precision: 0.3786 - val_recall: 0.9703 -
learning_rate: 0.0010

Epoch 8/40

[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8125 - auc: 0.9486 - loss: 0.4590 - precision: 0.3497 -
recall: 0.9434

[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8294 - auc: 0.9564 - loss: 0.4293 - precision: 0.3722
- recall: 0.9561

[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8291 - auc: 0.9556 - loss: 0.4334 - precision: 0.3735
- recall: 0.9565

[1m 73/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8291 - auc: 0.9551 - loss: 0.4365 - precision: 0.3751
- recall: 0.9564

[1m 97/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8285 - auc: 0.9546 - loss: 0.4393 - precision: 0.3759
- recall: 0.9565

[1m122/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8284 - auc: 0.9543 - loss: 0.4409 - precision: 0.3768
- recall: 0.9566

[1m146/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8285 - auc: 0.9541 - loss: 0.4418 - precision: 0.3776
- recall: 0.9568

[1m171/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8286 - auc: 0.9539 - loss: 0.4426 - precision: 0.3783
- recall: 0.9570

[1m195/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8287 - auc: 0.9539 - loss: 0.4430 - precision: 0.3788
- recall: 0.9570

[1m219/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8287 - auc: 0.9538 - loss: 0.4433 - precision: 0.3789
- recall: 0.9570

[1m243/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8288 - auc: 0.9537 - loss: 0.4434 - precision: 0.3792
- recall: 0.9569

[1m267/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8290 - auc: 0.9537 - loss: 0.4433 - precision: 0.3794
- recall: 0.9569

[1m292/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8291 - auc: 0.9537 - loss: 0.4432 - precision: 0.3797
- recall: 0.9568

[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8308 - auc: 0.9532 - loss: 0.4440 - precision: 0.3827
- recall: 0.9556 - val_accuracy: 0.8345 - val_auc: 0.9606 -
val_loss: 0.2645 - val_precision: 0.3888 - val_recall: 0.9637 -
learning_rate: 0.0010

Epoch 9/40

[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8184 - auc: 0.9560 - loss: 0.4344 - precision: 0.3611 -
recall: 0.9811


```
[1m 25/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8319 - auc: 0.9579 - loss: 0.4249 - precision: 0.3766
- recall: 0.9664
[1m 49/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8325 - auc: 0.9564 - loss: 0.4294 - precision: 0.3789
- recall: 0.9629
[1m 73/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8324 - auc: 0.9558 - loss: 0.4320 - precision: 0.3802
- recall: 0.9606
[1m 98/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8318 - auc: 0.9552 - loss: 0.4344 - precision: 0.3809
- recall: 0.9596
[1m122/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8316 - auc: 0.9550 - loss: 0.4357 - precision: 0.3815
- recall: 0.9592
[1m146/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8315 - auc: 0.9549 - loss: 0.4363 - precision: 0.3821
- recall: 0.9591
[1m172/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8315 - auc: 0.9549 - loss: 0.4366 - precision: 0.3827
- recall: 0.9590
[1m198/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8316 - auc: 0.9549 - loss: 0.4367 - precision: 0.3832
- recall: 0.9589
[1m222/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8317 - auc: 0.9549 - loss: 0.4366 - precision: 0.3834
- recall: 0.9588
[1m247/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8319 - auc: 0.9549 - loss: 0.4364 - precision: 0.3838
- recall: 0.9587
[1m272/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8321 - auc: 0.9549 - loss: 0.4362 - precision: 0.3841
- recall: 0.9586
[1m297/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8323 - auc: 0.9550 - loss: 0.4360 - precision: 0.3844
- recall: 0.9585
[1m314/314[0m [32m[0m[37m[0m[1m1s[0m 2ms/step
- accuracy: 0.8345 - auc: 0.9551 - loss: 0.4349 - precision: 0.3882
- recall: 0.9577 - val_accuracy: 0.8259 - val_auc: 0.9615 -
val_loss: 0.2650 - val_precision: 0.3778 - val_recall: 0.9774 -
learning_rate: 0.0010
Epoch 10/40
[1m 1/314[0m [37m[0m[1m2s[0m 10ms/step -
accuracy: 0.8203 - auc: 0.9620 - loss: 0.4226 - precision: 0.3617 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8320 - auc: 0.9609 - loss: 0.4115 - precision: 0.3762
- recall: 0.9622
[1m 49/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8316 - auc: 0.9585 - loss: 0.4209 - precision: 0.3771
- recall: 0.9584
[1m 73/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8312 - auc: 0.9573 - loss: 0.4265 - precision: 0.3781
- recall: 0.9569
```



```
[1m 98/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8307 - auc: 0.9566 - loss: 0.4304 - precision: 0.3791
- recall: 0.9571
[1m123/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8307 - auc: 0.9562 - loss: 0.4325 - precision: 0.3801
- recall: 0.9575
[1m147/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8308 - auc: 0.9560 - loss: 0.4335 - precision: 0.3809
- recall: 0.9579
[1m172/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8309 - auc: 0.9558 - loss: 0.4344 - precision: 0.3816
- recall: 0.9581
[1m197/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8310 - auc: 0.9557 - loss: 0.4350 - precision: 0.3821
- recall: 0.9582
[1m222/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8311 - auc: 0.9556 - loss: 0.4353 - precision: 0.3824
- recall: 0.9583
[1m246/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8312 - auc: 0.9556 - loss: 0.4354 - precision: 0.3827
- recall: 0.9584
[1m270/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8313 - auc: 0.9556 - loss: 0.4354 - precision: 0.3829
- recall: 0.9585
[1m295/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8315 - auc: 0.9556 - loss: 0.4354 - precision: 0.3832
- recall: 0.9585
[1m314/314[0m [32m-----[0m[37m-----[0m [1m1s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9552 - loss: 0.4358 - precision: 0.3865
- recall: 0.9587 - val_accuracy: 0.8332 - val_auc: 0.9615 -
val_loss: 0.2591 - val_precision: 0.3870 - val_recall: 0.9656 -
learning_rate: 0.0010
Epoch 11/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8242 - auc: 0.9595 - loss: 0.4202 - precision: 0.3669 -
recall: 0.9623
[1m 25/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8369 - auc: 0.9621 - loss: 0.4013 - precision: 0.3827
- recall: 0.9538
[1m 49/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8366 - auc: 0.9602 - loss: 0.4101 - precision: 0.3837
- recall: 0.9496
[1m 73/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8360 - auc: 0.9593 - loss: 0.4146 - precision: 0.3846
- recall: 0.9500
[1m 97/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9587 - loss: 0.4180 - precision: 0.3847
- recall: 0.9514
[1m121/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8344 - auc: 0.9583 - loss: 0.4200 - precision: 0.3850
- recall: 0.9528
[1m145/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8341 - auc: 0.9581 - loss: 0.4211 - precision: 0.3853
- recall: 0.9538
```

```
[1m169/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8340 - auc: 0.9580 - loss: 0.4219 - precision: 0.3858
- recall: 0.9547
[1m193/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8340 - auc: 0.9579 - loss: 0.4224 - precision: 0.3862
- recall: 0.9554
[1m217/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8339 - auc: 0.9578 - loss: 0.4228 - precision: 0.3863
- recall: 0.9560
[1m241/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8338 - auc: 0.9577 - loss: 0.4233 - precision: 0.3863
- recall: 0.9564
[1m266/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8338 - auc: 0.9576 - loss: 0.4237 - precision: 0.3864
- recall: 0.9569
[1m290/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8337 - auc: 0.9576 - loss: 0.4240 - precision: 0.3864
- recall: 0.9572
[1m313/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8337 - auc: 0.9575 - loss: 0.4243 - precision: 0.3864
- recall: 0.9575
[1m314/314[0m [32m-----[0m[37m-----[0m [1m1s[0m 2ms/step
- accuracy: 0.8338 - auc: 0.9568 - loss: 0.4283 - precision: 0.3875
- recall: 0.9608 - val_accuracy: 0.8402 - val_auc: 0.9621 -
val_loss: 0.2555 - val_precision: 0.3969 - val_recall: 0.9588 -
learning_rate: 0.0010
Epoch 12/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 9ms/step -
accuracy: 0.8184 - auc: 0.9608 - loss: 0.4104 - precision: 0.3611 -
recall: 0.9811
[1m 25/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8378 - auc: 0.9620 - loss: 0.4071 - precision: 0.3845
- recall: 0.9592
[1m 49/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8377 - auc: 0.9603 - loss: 0.4123 - precision: 0.3859
- recall: 0.9556
[1m 73/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8371 - auc: 0.9595 - loss: 0.4152 - precision: 0.3867
- recall: 0.9550
[1m 98/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8360 - auc: 0.9590 - loss: 0.4176 - precision: 0.3868
- recall: 0.9559
[1m123/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8356 - auc: 0.9588 - loss: 0.4186 - precision: 0.3873
- recall: 0.9570
[1m148/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8355 - auc: 0.9587 - loss: 0.4193 - precision: 0.3878
- recall: 0.9578
[1m172/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8353 - auc: 0.9585 - loss: 0.4201 - precision: 0.3881
- recall: 0.9583
[1m197/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8352 - auc: 0.9584 - loss: 0.4206 - precision: 0.3884
- recall: 0.9586
```

```
[1m222/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8351 - auc: 0.9584 - loss: 0.4209 - precision: 0.3883
- recall: 0.9589
[1m247/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9583 - loss: 0.4211 - precision: 0.3883
- recall: 0.9591
[1m272/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8349 - auc: 0.9583 - loss: 0.4212 - precision: 0.3883
- recall: 0.9592
[1m297/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8349 - auc: 0.9583 - loss: 0.4213 - precision: 0.3883
- recall: 0.9592
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8348 - auc: 0.9577 - loss: 0.4238 - precision: 0.3889
- recall: 0.9597 - val_accuracy: 0.8305 - val_auc: 0.9619 -
val_loss: 0.2585 - val_precision: 0.3839 - val_recall: 0.9738 -
learning_rate: 0.0010
Epoch 13/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8164 - auc: 0.9680 - loss: 0.3871 - precision: 0.3586 -
recall: 0.9811
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8342 - auc: 0.9645 - loss: 0.3942 - precision: 0.3804
- recall: 0.9715
[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8355 - auc: 0.9624 - loss: 0.4025 - precision: 0.3836
- recall: 0.9659
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8354 - auc: 0.9613 - loss: 0.4076 - precision: 0.3850
- recall: 0.9632
[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8349 - auc: 0.9606 - loss: 0.4111 - precision: 0.3858
- recall: 0.9625
[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9602 - loss: 0.4130 - precision: 0.3868
- recall: 0.9623
[1m148/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8352 - auc: 0.9600 - loss: 0.4142 - precision: 0.3877
- recall: 0.9621
[1m172/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8353 - auc: 0.9598 - loss: 0.4151 - precision: 0.3884
- recall: 0.9619
[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8353 - auc: 0.9596 - loss: 0.4157 - precision: 0.3887
- recall: 0.9619
[1m221/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8352 - auc: 0.9595 - loss: 0.4162 - precision: 0.3887
- recall: 0.9619
[1m245/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9594 - loss: 0.4165 - precision: 0.3886
- recall: 0.9619
[1m269/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9593 - loss: 0.4167 - precision: 0.3886
- recall: 0.9620
```

```
[1m293/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9592 - loss: 0.4170 - precision: 0.3886
- recall: 0.9620
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8351 - auc: 0.9582 - loss: 0.4207 - precision: 0.3895
- recall: 0.9618 - val_accuracy: 0.8291 - val_auc: 0.9622 -
val_loss: 0.2573 - val_precision: 0.3815 - val_recall: 0.9692 -
learning_rate: 0.0010
Epoch 14/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8203 - auc: 0.9644 - loss: 0.4036 - precision: 0.3617 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8327 - auc: 0.9629 - loss: 0.3979 - precision: 0.3775
- recall: 0.9641
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8334 - auc: 0.9616 - loss: 0.4034 - precision: 0.3802
- recall: 0.9617
[1m 75/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9610 - loss: 0.4063 - precision: 0.3818
- recall: 0.9624
[1m100/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8328 - auc: 0.9607 - loss: 0.4084 - precision: 0.3828
- recall: 0.9633
[1m124/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8328 - auc: 0.9606 - loss: 0.4094 - precision: 0.3837
- recall: 0.9639
[1m148/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8329 - auc: 0.9605 - loss: 0.4100 - precision: 0.3846
- recall: 0.9641
[1m173/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9604 - loss: 0.4106 - precision: 0.3854
- recall: 0.9641
[1m198/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8332 - auc: 0.9604 - loss: 0.4110 - precision: 0.3860
- recall: 0.9640
[1m222/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9603 - loss: 0.4113 - precision: 0.3861
- recall: 0.9640
[1m247/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9602 - loss: 0.4116 - precision: 0.3862
- recall: 0.9640
[1m272/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9602 - loss: 0.4118 - precision: 0.3863
- recall: 0.9641
[1m296/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9601 - loss: 0.4119 - precision: 0.3864
- recall: 0.9641
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8340 - auc: 0.9594 - loss: 0.4153 - precision: 0.3881
- recall: 0.9638 - val_accuracy: 0.8332 - val_auc: 0.9623 -
val_loss: 0.2534 - val_precision: 0.3875 - val_recall: 0.9716 -
learning_rate: 0.0010
Epoch 15/40
```

```
[1m 1/314[0m [37m-----[0m [1m2s[0m 9ms/step -
accuracy: 0.8301 - auc: 0.9672 - loss: 0.3782 - precision: 0.3768 -
recall: 0.9811
[1m 25/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8407 - auc: 0.9643 - loss: 0.3887 - precision: 0.3898
- recall: 0.9704
[1m 49/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9623 - loss: 0.3986 - precision: 0.3907
- recall: 0.9655
[1m 73/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9612 - loss: 0.4047 - precision: 0.3902
- recall: 0.9633
[1m 97/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8374 - auc: 0.9606 - loss: 0.4091 - precision: 0.3893
- recall: 0.9627
[1m122/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8366 - auc: 0.9603 - loss: 0.4114 - precision: 0.3892
- recall: 0.9626
[1m147/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8362 - auc: 0.9601 - loss: 0.4126 - precision: 0.3893
- recall: 0.9627
[1m172/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8361 - auc: 0.9600 - loss: 0.4136 - precision: 0.3897
- recall: 0.9628
[1m197/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8360 - auc: 0.9599 - loss: 0.4143 - precision: 0.3900
- recall: 0.9630
[1m222/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8359 - auc: 0.9598 - loss: 0.4146 - precision: 0.3900
- recall: 0.9631
[1m247/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8359 - auc: 0.9598 - loss: 0.4148 - precision: 0.3900
- recall: 0.9633
[1m272/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8359 - auc: 0.9597 - loss: 0.4149 - precision: 0.3901
- recall: 0.9634
[1m296/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8360 - auc: 0.9597 - loss: 0.4150 - precision: 0.3902
- recall: 0.9634
[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8368 - auc: 0.9593 - loss: 0.4169 - precision: 0.3921
- recall: 0.9628 - val_accuracy: 0.8373 - val_auc: 0.9627 -
val_loss: 0.2526 - val_precision: 0.3933 - val_recall: 0.9681 -
learning_rate: 0.0010
Epoch 16/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 9ms/step -
accuracy: 0.8145 - auc: 0.9693 - loss: 0.3839 - precision: 0.3562 -
recall: 0.9811
[1m 24/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8352 - auc: 0.9651 - loss: 0.3887 - precision: 0.3809
- recall: 0.9633
[1m 48/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8347 - auc: 0.9627 - loss: 0.3998 - precision: 0.3819
- recall: 0.9604
```

```

[1m 72/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8342 - auc: 0.9615 - loss: 0.4054 - precision: 0.3827
- recall: 0.9604
[1m 97/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9609 - loss: 0.4089 - precision: 0.3832
- recall: 0.9611
[1m 121/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9606 - loss: 0.4106 - precision: 0.3839
- recall: 0.9617
[1m 146/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8332 - auc: 0.9604 - loss: 0.4116 - precision: 0.3848
- recall: 0.9621
[1m 170/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8334 - auc: 0.9603 - loss: 0.4123 - precision: 0.3857
- recall: 0.9624
[1m 195/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8336 - auc: 0.9602 - loss: 0.4128 - precision: 0.3864
- recall: 0.9625
[1m 220/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8338 - auc: 0.9602 - loss: 0.4130 - precision: 0.3867
- recall: 0.9625
[1m 244/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8339 - auc: 0.9601 - loss: 0.4132 - precision: 0.3870
- recall: 0.9626
[1m 269/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8340 - auc: 0.9601 - loss: 0.4133 - precision: 0.3872
- recall: 0.9627
[1m 293/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8341 - auc: 0.9601 - loss: 0.4133 - precision: 0.3874
- recall: 0.9626
[1m 314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8362 - auc: 0.9596 - loss: 0.4145 - precision: 0.3911
- recall: 0.9621 - val_accuracy: 0.8284 - val_auc: 0.9627 -
val_loss: 0.2567 - val_precision: 0.3813 - val_recall: 0.9782 -
learning_rate: 0.0010
Epoch 17/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8203 - auc: 0.9598 - loss: 0.4128 - precision: 0.3597 -
recall: 0.9434
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8342 - auc: 0.9651 - loss: 0.3851 - precision: 0.3798
- recall: 0.9654
[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8346 - auc: 0.9634 - loss: 0.3928 - precision: 0.3819
- recall: 0.9629
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8343 - auc: 0.9624 - loss: 0.3981 - precision: 0.3832
- recall: 0.9623
[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8335 - auc: 0.9618 - loss: 0.4014 - precision: 0.3837
- recall: 0.9630
[1m 122/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8332 - auc: 0.9615 - loss: 0.4034 - precision: 0.3843
- recall: 0.9635

```

```

[1m146/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9613 - loss: 0.4046 - precision: 0.3848
- recall: 0.9639
[1m171/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9611 - loss: 0.4057 - precision: 0.3853
- recall: 0.9643
[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9611 - loss: 0.4064 - precision: 0.3858
- recall: 0.9644
[1m220/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9610 - loss: 0.4068 - precision: 0.3860
- recall: 0.9646
[1m245/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9609 - loss: 0.4071 - precision: 0.3863
- recall: 0.9647
[1m270/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8335 - auc: 0.9609 - loss: 0.4073 - precision: 0.3866
- recall: 0.9648
[1m295/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8337 - auc: 0.9609 - loss: 0.4074 - precision: 0.3870
- recall: 0.9648
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8362 - auc: 0.9604 - loss: 0.4101 - precision: 0.3913
- recall: 0.9645 - val_accuracy: 0.8306 - val_auc: 0.9629 -
val_loss: 0.2594 - val_precision: 0.3840 - val_recall: 0.9733 -
learning_rate: 0.0010
Epoch 18/40
[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8125 - auc: 0.9582 - loss: 0.4175 - precision: 0.3517 -
recall: 0.9623
[1m 26/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8309 - auc: 0.9639 - loss: 0.3957 - precision: 0.3750
- recall: 0.9624
[1m 51/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8342 - auc: 0.9628 - loss: 0.4004 - precision: 0.3813
- recall: 0.9597
[1m 76/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8355 - auc: 0.9622 - loss: 0.4036 - precision: 0.3849
- recall: 0.9581
[1m100/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8357 - auc: 0.9618 - loss: 0.4058 - precision: 0.3867
- recall: 0.9585
[1m124/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8360 - auc: 0.9616 - loss: 0.4074 - precision: 0.3880
- recall: 0.9591
[1m148/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8362 - auc: 0.9614 - loss: 0.4086 - precision: 0.3890
- recall: 0.9593
[1m172/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8363 - auc: 0.9613 - loss: 0.4095 - precision: 0.3898
- recall: 0.9596
[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8364 - auc: 0.9612 - loss: 0.4100 - precision: 0.3903
- recall: 0.9599

```



```
[1m221/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8364 - auc: 0.9611 - loss: 0.4105 - precision: 0.3904
- recall: 0.9602
[1m246/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8363 - auc: 0.9610 - loss: 0.4108 - precision: 0.3904
- recall: 0.9606
[1m269/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8363 - auc: 0.9609 - loss: 0.4110 - precision: 0.3904
- recall: 0.9609
[1m284/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8363 - auc: 0.9609 - loss: 0.4111 - precision: 0.3905
- recall: 0.9610
[1m308/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8363 - auc: 0.9609 - loss: 0.4112 - precision: 0.3905
- recall: 0.9612
```

Epoch 18: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

```
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8362 - auc: 0.9603 - loss: 0.4130 - precision: 0.3913
- recall: 0.9639 - val_accuracy: 0.8292 - val_auc: 0.9629 -
val_loss: 0.2544 - val_precision: 0.3827 - val_recall: 0.9806 -
learning_rate: 0.0010
```

Epoch 19/40

```
[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8203 - auc: 0.9678 - loss: 0.3871 - precision: 0.3617 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8318 - auc: 0.9651 - loss: 0.3853 - precision: 0.3764
- recall: 0.9672
[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8323 - auc: 0.9632 - loss: 0.3941 - precision: 0.3788
- recall: 0.9656
[1m 73/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8323 - auc: 0.9623 - loss: 0.3987 - precision: 0.3805
- recall: 0.9656
[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8323 - auc: 0.9619 - loss: 0.4016 - precision: 0.3822
- recall: 0.9655
[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8326 - auc: 0.9617 - loss: 0.4030 - precision: 0.3836
- recall: 0.9657
[1m148/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8329 - auc: 0.9616 - loss: 0.4037 - precision: 0.3847
- recall: 0.9659
[1m172/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8331 - auc: 0.9615 - loss: 0.4042 - precision: 0.3856
- recall: 0.9660
[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8333 - auc: 0.9615 - loss: 0.4043 - precision: 0.3863
- recall: 0.9661
[1m221/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8335 - auc: 0.9615 - loss: 0.4044 - precision: 0.3867
- recall: 0.9662
```



```

[1m245/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8336 - auc: 0.9614 - loss: 0.4045 - precision: 0.3870
- recall: 0.9663
[1m269/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8338 - auc: 0.9615 - loss: 0.4045 - precision: 0.3872
- recall: 0.9664
[1m294/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8339 - auc: 0.9615 - loss: 0.4044 - precision: 0.3875
- recall: 0.9665
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8367 - auc: 0.9614 - loss: 0.4048 - precision: 0.3923
- recall: 0.9658 - val_accuracy: 0.8374 - val_auc: 0.9631 -
val_loss: 0.2566 - val_precision: 0.3933 - val_recall: 0.9670 -
learning_rate: 5.0000e-04
Epoch 20/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8301 - auc: 0.9651 - loss: 0.4087 - precision: 0.3750 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8415 - auc: 0.9653 - loss: 0.3861 - precision: 0.3907
- recall: 0.9665
[1m 48/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8418 - auc: 0.9639 - loss: 0.3916 - precision: 0.3925
- recall: 0.9622
[1m 73/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8410 - auc: 0.9632 - loss: 0.3955 - precision: 0.3930
- recall: 0.9610
[1m 97/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8401 - auc: 0.9627 - loss: 0.3984 - precision: 0.3933
- recall: 0.9609
[1m121/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9625 - loss: 0.3999 - precision: 0.3938
- recall: 0.9612
[1m146/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9624 - loss: 0.4008 - precision: 0.3940
- recall: 0.9616
[1m171/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9623 - loss: 0.4014 - precision: 0.3943
- recall: 0.9619
[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9622 - loss: 0.4018 - precision: 0.3945
- recall: 0.9623
[1m220/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9622 - loss: 0.4020 - precision: 0.3945
- recall: 0.9625
[1m244/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8389 - auc: 0.9621 - loss: 0.4022 - precision: 0.3944
- recall: 0.9628
[1m269/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8388 - auc: 0.9621 - loss: 0.4022 - precision: 0.3944
- recall: 0.9630
[1m294/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9621 - loss: 0.4023 - precision: 0.3943
- recall: 0.9632

```

[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8384 - auc: 0.9616 - loss: 0.4040 - precision: 0.3947
- recall: 0.9645 - val_accuracy: 0.8308 - val_auc: 0.9632 -
val_loss: 0.2564 - val_precision: 0.3848 - val_recall: 0.9790 -
learning_rate: 5.0000e-04

Epoch 21/40

[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8262 - auc: 0.9702 - loss: 0.3707 - precision: 0.3714 -
recall: 0.9811

[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8359 - auc: 0.9675 - loss: 0.3758 - precision: 0.3828
- recall: 0.9726

[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8355 - auc: 0.9656 - loss: 0.3844 - precision: 0.3839
- recall: 0.9692

[1m 75/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9645 - loss: 0.3897 - precision: 0.3847
- recall: 0.9675

[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8346 - auc: 0.9639 - loss: 0.3930 - precision: 0.3857
- recall: 0.9671

[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8348 - auc: 0.9635 - loss: 0.3948 - precision: 0.3869
- recall: 0.9670

[1m149/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8350 - auc: 0.9633 - loss: 0.3960 - precision: 0.3879
- recall: 0.9671

[1m172/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8352 - auc: 0.9631 - loss: 0.3969 - precision: 0.3887
- recall: 0.9671

[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8354 - auc: 0.9630 - loss: 0.3975 - precision: 0.3894
- recall: 0.9671

[1m221/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8355 - auc: 0.9629 - loss: 0.3979 - precision: 0.3896
- recall: 0.9671

[1m246/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8356 - auc: 0.9628 - loss: 0.3984 - precision: 0.3899
- recall: 0.9670

[1m270/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8357 - auc: 0.9628 - loss: 0.3987 - precision: 0.3901
- recall: 0.9670

[1m295/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8358 - auc: 0.9627 - loss: 0.3990 - precision: 0.3902
- recall: 0.9669

Epoch 21: ReduceLROnPlateau reducing learning rate to
0.0002500000118743628.

[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8373 - auc: 0.9619 - loss: 0.4030 - precision: 0.3930
- recall: 0.9658 - val_accuracy: 0.8309 - val_auc: 0.9632 -
val_loss: 0.2591 - val_precision: 0.3848 - val_recall: 0.9776 -
learning_rate: 5.0000e-04

Epoch 22/40

```
[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8242 - auc: 0.9676 - loss: 0.3835 - precision: 0.3688 -
recall: 0.9811
[1m 25/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8348 - auc: 0.9669 - loss: 0.3785 - precision: 0.3817
- recall: 0.9767
[1m 50/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8354 - auc: 0.9649 - loss: 0.3879 - precision: 0.3840
- recall: 0.9728
[1m 75/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8355 - auc: 0.9639 - loss: 0.3929 - precision: 0.3858
- recall: 0.9710
[1m100/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8354 - auc: 0.9634 - loss: 0.3957 - precision: 0.3872
- recall: 0.9700
[1m125/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8356 - auc: 0.9632 - loss: 0.3969 - precision: 0.3884
- recall: 0.9696
[1m150/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8359 - auc: 0.9631 - loss: 0.3975 - precision: 0.3894
- recall: 0.9693
[1m176/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8362 - auc: 0.9630 - loss: 0.3981 - precision: 0.3904
- recall: 0.9690
[1m201/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8364 - auc: 0.9629 - loss: 0.3984 - precision: 0.3911
- recall: 0.9687
[1m225/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8366 - auc: 0.9628 - loss: 0.3987 - precision: 0.3914
- recall: 0.9685
[1m248/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8367 - auc: 0.9628 - loss: 0.3989 - precision: 0.3917
- recall: 0.9684
[1m272/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8368 - auc: 0.9627 - loss: 0.3990 - precision: 0.3919
- recall: 0.9683
[1m296/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8369 - auc: 0.9627 - loss: 0.3991 - precision: 0.3920
- recall: 0.9683
[1m314/314[0m [32m-----[0m [37m-----[0m [1m1s[0m 2ms/step
- accuracy: 0.8385 - auc: 0.9621 - loss: 0.4015 - precision: 0.3951
- recall: 0.9668 - val_accuracy: 0.8381 - val_auc: 0.9633 -
val_loss: 0.2558 - val_precision: 0.3945 - val_recall: 0.9675 -
learning_rate: 2.5000e-04
Epoch 23/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8223 - auc: 0.9697 - loss: 0.3816 - precision: 0.3643 -
recall: 0.9623
[1m 25/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8377 - auc: 0.9670 - loss: 0.3788 - precision: 0.3849
- recall: 0.9661
[1m 49/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8382 - auc: 0.9652 - loss: 0.3859 - precision: 0.3874
- recall: 0.9644
```

```

[1m 74/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8383 - auc: 0.9642 - loss: 0.3909 - precision: 0.3894
- recall: 0.9641
[1m 98/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8383 - auc: 0.9636 - loss: 0.3941 - precision: 0.3908
- recall: 0.9640
[1m122/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8384 - auc: 0.9633 - loss: 0.3956 - precision: 0.3920
- recall: 0.9644
[1m146/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8386 - auc: 0.9632 - loss: 0.3963 - precision: 0.3929
- recall: 0.9648
[1m171/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9631 - loss: 0.3969 - precision: 0.3938
- recall: 0.9649
[1m196/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8389 - auc: 0.9630 - loss: 0.3972 - precision: 0.3945
- recall: 0.9650
[1m221/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9630 - loss: 0.3975 - precision: 0.3948
- recall: 0.9650
[1m245/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9629 - loss: 0.3976 - precision: 0.3951
- recall: 0.9651
[1m270/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9629 - loss: 0.3978 - precision: 0.3952
- recall: 0.9652
[1m294/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8393 - auc: 0.9628 - loss: 0.3979 - precision: 0.3953
- recall: 0.9653
[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8404 - auc: 0.9623 - loss: 0.4007 - precision: 0.3978
- recall: 0.9653 - val_accuracy: 0.8416 - val_auc: 0.9634 -
val_loss: 0.2558 - val_precision: 0.3993 - val_recall: 0.9624 -
learning_rate: 2.5000e-04
Epoch 24/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 9ms/step -
accuracy: 0.8242 - auc: 0.9728 - loss: 0.3644 - precision: 0.3669 -
recall: 0.9623
[1m 26/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8405 - auc: 0.9670 - loss: 0.3754 - precision: 0.3892
- recall: 0.9644
[1m 50/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8410 - auc: 0.9653 - loss: 0.3830 - precision: 0.3915
- recall: 0.9632
[1m 74/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8411 - auc: 0.9643 - loss: 0.3886 - precision: 0.3934
- recall: 0.9625
[1m 98/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8410 - auc: 0.9638 - loss: 0.3921 - precision: 0.3948
- recall: 0.9619
[1m122/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8412 - auc: 0.9636 - loss: 0.3937 - precision: 0.3960
- recall: 0.9620

```

```

[1m146/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8413 - auc: 0.9634 - loss: 0.3946 - precision: 0.3968
- recall: 0.9621
[1m170/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8413 - auc: 0.9633 - loss: 0.3953 - precision: 0.3975
- recall: 0.9623
[1m195/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8414 - auc: 0.9632 - loss: 0.3958 - precision: 0.3980
- recall: 0.9624
[1m220/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8414 - auc: 0.9632 - loss: 0.3961 - precision: 0.3982
- recall: 0.9624
[1m244/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8414 - auc: 0.9631 - loss: 0.3964 - precision: 0.3983
- recall: 0.9625
[1m268/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8414 - auc: 0.9631 - loss: 0.3966 - precision: 0.3984
- recall: 0.9626
[1m292/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8414 - auc: 0.9630 - loss: 0.3967 - precision: 0.3984
- recall: 0.9626

```

Epoch 24: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

```

[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8420 - auc: 0.9625 - loss: 0.3996 - precision: 0.4000
- recall: 0.9622 - val_accuracy: 0.8404 - val_auc: 0.9634 -
val_loss: 0.2543 - val_precision: 0.3976 - val_recall: 0.9640 -
learning_rate: 2.5000e-04

```

Epoch 25/40

```

[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8320 - auc: 0.9713 - loss: 0.3638 - precision: 0.3778 -
recall: 0.9623
[1m 25/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8415 - auc: 0.9674 - loss: 0.3725 - precision: 0.3908
- recall: 0.9680
[1m 49/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8409 - auc: 0.9655 - loss: 0.3815 - precision: 0.3915
- recall: 0.9644
[1m 74/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8405 - auc: 0.9646 - loss: 0.3865 - precision: 0.3926
- recall: 0.9629
[1m 98/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9641 - loss: 0.3900 - precision: 0.3937
- recall: 0.9621
[1m123/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9637 - loss: 0.3919 - precision: 0.3947
- recall: 0.9618
[1m147/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8404 - auc: 0.9636 - loss: 0.3930 - precision: 0.3955
- recall: 0.9619
[1m171/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8405 - auc: 0.9634 - loss: 0.3939 - precision: 0.3962
- recall: 0.9620

```

```
[1m195/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8407 - auc: 0.9633 - loss: 0.3945 - precision: 0.3968
- recall: 0.9620
[1m219/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8407 - auc: 0.9633 - loss: 0.3949 - precision: 0.3971
- recall: 0.9619
[1m243/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9632 - loss: 0.3952 - precision: 0.3973
- recall: 0.9619
[1m267/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8409 - auc: 0.9631 - loss: 0.3955 - precision: 0.3975
- recall: 0.9620
[1m291/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8409 - auc: 0.9631 - loss: 0.3957 - precision: 0.3975
- recall: 0.9621
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8413 - auc: 0.9624 - loss: 0.3995 - precision: 0.3990
- recall: 0.9624 - val_accuracy: 0.8408 - val_auc: 0.9634 -
val_loss: 0.2513 - val_precision: 0.3980 - val_recall: 0.9615 -
learning_rate: 1.2500e-04
Epoch 26/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8301 - auc: 0.9715 - loss: 0.3654 - precision: 0.3750 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9675 - loss: 0.3759 - precision: 0.3898
- recall: 0.9691
[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9651 - loss: 0.3870 - precision: 0.3915
- recall: 0.9660
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9642 - loss: 0.3917 - precision: 0.3931
- recall: 0.9646
[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8407 - auc: 0.9638 - loss: 0.3944 - precision: 0.3945
- recall: 0.9640
[1m122/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9636 - loss: 0.3955 - precision: 0.3957
- recall: 0.9640
[1m146/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8410 - auc: 0.9635 - loss: 0.3960 - precision: 0.3966
- recall: 0.9640
[1m170/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8412 - auc: 0.9634 - loss: 0.3965 - precision: 0.3974
- recall: 0.9639
[1m194/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8413 - auc: 0.9634 - loss: 0.3967 - precision: 0.3981
- recall: 0.9639
[1m218/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8414 - auc: 0.9633 - loss: 0.3969 - precision: 0.3983
- recall: 0.9638
[1m242/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9632 - loss: 0.3970 - precision: 0.3986
- recall: 0.9638
```

```

[1m266/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9632 - loss: 0.3971 - precision: 0.3989
- recall: 0.9639
[1m290/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8417 - auc: 0.9632 - loss: 0.3971 - precision: 0.3990
- recall: 0.9640
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8429 - auc: 0.9627 - loss: 0.3981 - precision: 0.4015
- recall: 0.9638 - val_accuracy: 0.8415 - val_auc: 0.9634 -
val_loss: 0.2526 - val_precision: 0.3990 - val_recall: 0.9607 -
learning_rate: 1.2500e-04
Epoch 27/40
[1m 1/314[0m [32m[0m[37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8398 - auc: 0.9698 - loss: 0.3706 - precision: 0.3893 -
recall: 0.9623
[1m 24/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8444 - auc: 0.9681 - loss: 0.3705 - precision: 0.3951
- recall: 0.9677
[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8431 - auc: 0.9659 - loss: 0.3801 - precision: 0.3949
- recall: 0.9648
[1m 73/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8423 - auc: 0.9650 - loss: 0.3854 - precision: 0.3953
- recall: 0.9638
[1m 97/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8418 - auc: 0.9643 - loss: 0.3893 - precision: 0.3960
- recall: 0.9631
[1m121/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9640 - loss: 0.3916 - precision: 0.3968
- recall: 0.9631
[1m145/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9638 - loss: 0.3929 - precision: 0.3974
- recall: 0.9631
[1m169/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9636 - loss: 0.3941 - precision: 0.3979
- recall: 0.9630
[1m193/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9635 - loss: 0.3949 - precision: 0.3984
- recall: 0.9629
[1m217/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9634 - loss: 0.3955 - precision: 0.3985
- recall: 0.9628
[1m241/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9633 - loss: 0.3959 - precision: 0.3986
- recall: 0.9628
[1m265/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8416 - auc: 0.9633 - loss: 0.3963 - precision: 0.3986
- recall: 0.9628
[1m290/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8415 - auc: 0.9632 - loss: 0.3966 - precision: 0.3985
- recall: 0.9629
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8410 - auc: 0.9624 - loss: 0.4011 - precision: 0.3985
- recall: 0.9632 - val_accuracy: 0.8396 - val_auc: 0.9635 -

```



```
val_loss: 0.2529 - val_precision: 0.3965 - val_recall: 0.9651 -  
learning_rate: 1.2500e-04  
Epoch 28/40  
[1m 1/314[0m [37m-----[0m [1m3s[0m 10ms/step -  
accuracy: 0.8184 - auc: 0.9660 - loss: 0.3949 - precision: 0.3592 -  
recall: 0.9623  
[1m 25/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8411 - auc: 0.9666 - loss: 0.3758 - precision: 0.3909  
- recall: 0.9734  
[1m 49/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8407 - auc: 0.9652 - loss: 0.3823 - precision: 0.3917  
- recall: 0.9700  
[1m 74/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8403 - auc: 0.9644 - loss: 0.3868 - precision: 0.3927  
- recall: 0.9683  
[1m 98/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8399 - auc: 0.9639 - loss: 0.3899 - precision: 0.3936  
- recall: 0.9674  
[1m122/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8399 - auc: 0.9637 - loss: 0.3915 - precision: 0.3946  
- recall: 0.9668  
[1m146/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8400 - auc: 0.9635 - loss: 0.3924 - precision: 0.3952  
- recall: 0.9663  
[1m170/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8400 - auc: 0.9634 - loss: 0.3932 - precision: 0.3958  
- recall: 0.9660  
[1m194/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8401 - auc: 0.9633 - loss: 0.3938 - precision: 0.3963  
- recall: 0.9658  
[1m218/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8401 - auc: 0.9632 - loss: 0.3942 - precision: 0.3964  
- recall: 0.9655  
[1m243/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8401 - auc: 0.9632 - loss: 0.3946 - precision: 0.3965  
- recall: 0.9654  
[1m268/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8401 - auc: 0.9631 - loss: 0.3948 - precision: 0.3966  
- recall: 0.9653  
[1m292/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step  
- accuracy: 0.8401 - auc: 0.9631 - loss: 0.3950 - precision: 0.3966  
- recall: 0.9652
```

Epoch 28: ReduceLROnPlateau reducing learning rate to
6.25000029685907e-05.

```
[1m314/314[0m [32m-----[0m [37m-----[0m [1m1s[0m 2ms/step  
- accuracy: 0.8404 - auc: 0.9624 - loss: 0.3984 - precision: 0.3976  
- recall: 0.9643 - val_accuracy: 0.8397 - val_auc: 0.9635 -  
val_loss: 0.2519 - val_precision: 0.3966 - val_recall: 0.9637 -  
learning_rate: 1.2500e-04  
Epoch 29/40  
[1m 1/314[0m [37m-----[0m [1m3s[0m 10ms/step -  
accuracy: 0.8281 - auc: 0.9733 - loss: 0.3591 - precision: 0.3723 -  
recall: 0.9623
```



```

[1m 25/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8401 - auc: 0.9686 - loss: 0.3700 - precision: 0.3888
- recall: 0.9695
[1m 50/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9665 - loss: 0.3793 - precision: 0.3899
- recall: 0.9683
[1m 74/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8393 - auc: 0.9655 - loss: 0.3843 - precision: 0.3911
- recall: 0.9680
[1m 98/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9648 - loss: 0.3879 - precision: 0.3922
- recall: 0.9673
[1m123/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9645 - loss: 0.3899 - precision: 0.3932
- recall: 0.9670
[1m148/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9642 - loss: 0.3910 - precision: 0.3941
- recall: 0.9668
[1m172/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8393 - auc: 0.9641 - loss: 0.3919 - precision: 0.3947
- recall: 0.9667
[1m197/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8393 - auc: 0.9639 - loss: 0.3926 - precision: 0.3952
- recall: 0.9664
[1m222/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8394 - auc: 0.9638 - loss: 0.3931 - precision: 0.3954
- recall: 0.9662
[1m246/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8394 - auc: 0.9637 - loss: 0.3935 - precision: 0.3955
- recall: 0.9661
[1m270/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8394 - auc: 0.9637 - loss: 0.3938 - precision: 0.3956
- recall: 0.9661
[1m294/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8394 - auc: 0.9636 - loss: 0.3940 - precision: 0.3956
- recall: 0.9661
[1m314/314[0m [32m[0m[37m[0m[1m1s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9628 - loss: 0.3978 - precision: 0.3967
- recall: 0.9657 - val_accuracy: 0.8381 - val_auc: 0.9635 -
val_loss: 0.2504 - val_precision: 0.3944 - val_recall: 0.9678 -
learning_rate: 6.2500e-05
Epoch 30/40
[1m 1/314[0m [37m[0m[1m3s[0m 10ms/step -
accuracy: 0.8262 - auc: 0.9712 - loss: 0.3658 - precision: 0.3696 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8409 - auc: 0.9684 - loss: 0.3700 - precision: 0.3904
- recall: 0.9744
[1m 50/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8404 - auc: 0.9663 - loss: 0.3788 - precision: 0.3912
- recall: 0.9703
[1m 74/314[0m [32m[0m[37m[0m[1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9652 - loss: 0.3843 - precision: 0.3917
- recall: 0.9682

```

```
[1m 99/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8393 - auc: 0.9646 - loss: 0.3880 - precision: 0.3927
- recall: 0.9671
[1m124/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8394 - auc: 0.9642 - loss: 0.3899 - precision: 0.3937
- recall: 0.9665
[1m149/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9640 - loss: 0.3910 - precision: 0.3945
- recall: 0.9663
[1m174/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9639 - loss: 0.3918 - precision: 0.3953
- recall: 0.9662
[1m199/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9638 - loss: 0.3924 - precision: 0.3958
- recall: 0.9660
[1m223/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9637 - loss: 0.3928 - precision: 0.3961
- recall: 0.9658
[1m248/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8399 - auc: 0.9636 - loss: 0.3932 - precision: 0.3963
- recall: 0.9658
[1m272/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9636 - loss: 0.3934 - precision: 0.3964
- recall: 0.9658
[1m296/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9635 - loss: 0.3937 - precision: 0.3965
- recall: 0.9658
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8405 - auc: 0.9629 - loss: 0.3973 - precision: 0.3980
- recall: 0.9659 - val_accuracy: 0.8386 - val_auc: 0.9635 -
val_loss: 0.2511 - val_precision: 0.3951 - val_recall: 0.9654 -
learning_rate: 6.2500e-05
Epoch 31/40
[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8320 - auc: 0.9694 - loss: 0.3680 - precision: 0.3778 -
recall: 0.9623
[1m 24/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8412 - auc: 0.9673 - loss: 0.3744 - precision: 0.3905
- recall: 0.9713
[1m 48/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9655 - loss: 0.3823 - precision: 0.3909
- recall: 0.9694
[1m 72/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9647 - loss: 0.3866 - precision: 0.3918
- recall: 0.9691
[1m 96/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9642 - loss: 0.3900 - precision: 0.3931
- recall: 0.9686
[1m120/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9640 - loss: 0.3917 - precision: 0.3943
- recall: 0.9684
[1m144/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9639 - loss: 0.3926 - precision: 0.3953
- recall: 0.9682
```

```
[1m169/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8402 - auc: 0.9638 - loss: 0.3933 - precision: 0.3963
- recall: 0.9683
[1m193/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8405 - auc: 0.9637 - loss: 0.3938 - precision: 0.3971
- recall: 0.9683
[1m217/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8406 - auc: 0.9636 - loss: 0.3941 - precision: 0.3975
- recall: 0.9682
[1m242/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9636 - loss: 0.3944 - precision: 0.3978
- recall: 0.9681
[1m267/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8409 - auc: 0.9635 - loss: 0.3945 - precision: 0.3981
- recall: 0.9681
[1m291/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8410 - auc: 0.9635 - loss: 0.3946 - precision: 0.3982
- recall: 0.9681
[1m314/314[0m [32m-----[0m[37m-----[0m [1m1s[0m 2ms/step
- accuracy: 0.8422 - auc: 0.9629 - loss: 0.3974 - precision: 0.4007
- recall: 0.9674 - val_accuracy: 0.8404 - val_auc: 0.9635 -
val_loss: 0.2508 - val_precision: 0.3976 - val_recall: 0.9629 -
learning_rate: 6.2500e-05
Epoch 32/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8320 - auc: 0.9730 - loss: 0.3615 - precision: 0.3796 -
recall: 0.9811
[1m 25/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8399 - auc: 0.9681 - loss: 0.3709 - precision: 0.3881
- recall: 0.9652
[1m 50/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9663 - loss: 0.3792 - precision: 0.3893
- recall: 0.9624
[1m 75/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9654 - loss: 0.3842 - precision: 0.3906
- recall: 0.9622
[1m100/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9648 - loss: 0.3875 - precision: 0.3919
- recall: 0.9622
[1m125/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9645 - loss: 0.3892 - precision: 0.3931
- recall: 0.9622
[1m150/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9643 - loss: 0.3902 - precision: 0.3942
- recall: 0.9623
[1m174/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9642 - loss: 0.3912 - precision: 0.3951
- recall: 0.9624
[1m198/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8399 - auc: 0.9641 - loss: 0.3918 - precision: 0.3958
- recall: 0.9625
[1m222/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9639 - loss: 0.3924 - precision: 0.3961
- recall: 0.9625
```

[1m246/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8402 - auc: 0.9638 - loss: 0.3929 - precision: 0.3964
- recall: 0.9626
[1m271/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9638 - loss: 0.3932 - precision: 0.3966
- recall: 0.9627
[1m296/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9637 - loss: 0.3936 - precision: 0.3967
- recall: 0.9628

Epoch 32: ReduceLROnPlateau reducing learning rate to
3.125000148429535e-05.

[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8412 - auc: 0.9627 - loss: 0.3987 - precision: 0.3988
- recall: 0.9633 - val_accuracy: 0.8388 - val_auc: 0.9635 -
val_loss: 0.2511 - val_precision: 0.3953 - val_recall: 0.9664 -
learning_rate: 6.2500e-05

Epoch 33/40

[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8398 - auc: 0.9721 - loss: 0.3573 - precision: 0.3910 -
recall: 0.9811
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8409 - auc: 0.9689 - loss: 0.3653 - precision: 0.3900
- recall: 0.9691
[1m 49/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9669 - loss: 0.3748 - precision: 0.3908
- recall: 0.9676
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9657 - loss: 0.3813 - precision: 0.3916
- recall: 0.9669
[1m 98/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9650 - loss: 0.3854 - precision: 0.3928
- recall: 0.9666
[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9647 - loss: 0.3876 - precision: 0.3940
- recall: 0.9663
[1m148/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9645 - loss: 0.3890 - precision: 0.3948
- recall: 0.9661
[1m172/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9643 - loss: 0.3901 - precision: 0.3956
- recall: 0.9660
[1m197/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9642 - loss: 0.3909 - precision: 0.3962
- recall: 0.9658
[1m221/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9640 - loss: 0.3915 - precision: 0.3964
- recall: 0.9657
[1m246/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8401 - auc: 0.9639 - loss: 0.3920 - precision: 0.3966
- recall: 0.9656
[1m271/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8402 - auc: 0.9639 - loss: 0.3924 - precision: 0.3968
- recall: 0.9656

```
[1m295/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8402 - auc: 0.9638 - loss: 0.3927 - precision: 0.3968
- recall: 0.9656
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8406 - auc: 0.9628 - loss: 0.3974 - precision: 0.3981
- recall: 0.9655 - val_accuracy: 0.8381 - val_auc: 0.9635 -
val_loss: 0.2502 - val_precision: 0.3944 - val_recall: 0.9681 -
learning_rate: 3.1250e-05
Epoch 34/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8301 - auc: 0.9682 - loss: 0.3756 - precision: 0.3750 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8386 - auc: 0.9671 - loss: 0.3740 - precision: 0.3862
- recall: 0.9656
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8376 - auc: 0.9654 - loss: 0.3824 - precision: 0.3865
- recall: 0.9645
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8371 - auc: 0.9646 - loss: 0.3869 - precision: 0.3875
- recall: 0.9640
[1m 99/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8369 - auc: 0.9642 - loss: 0.3901 - precision: 0.3887
- recall: 0.9638
[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8371 - auc: 0.9640 - loss: 0.3914 - precision: 0.3900
- recall: 0.9640
[1m147/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8374 - auc: 0.9639 - loss: 0.3921 - precision: 0.3911
- recall: 0.9643
[1m172/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8377 - auc: 0.9638 - loss: 0.3927 - precision: 0.3922
- recall: 0.9645
[1m197/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8380 - auc: 0.9637 - loss: 0.3931 - precision: 0.3931
- recall: 0.9647
[1m222/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8382 - auc: 0.9637 - loss: 0.3933 - precision: 0.3935
- recall: 0.9648
[1m247/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8384 - auc: 0.9636 - loss: 0.3936 - precision: 0.3939
- recall: 0.9648
[1m272/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8386 - auc: 0.9636 - loss: 0.3936 - precision: 0.3943
- recall: 0.9649
[1m297/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9636 - loss: 0.3938 - precision: 0.3944
- recall: 0.9650
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8401 - auc: 0.9630 - loss: 0.3965 - precision: 0.3973
- recall: 0.9653 - val_accuracy: 0.8379 - val_auc: 0.9636 -
val_loss: 0.2503 - val_precision: 0.3941 - val_recall: 0.9678 -
learning_rate: 3.1250e-05
Epoch 35/40
```

```

[1m 1/314[0m [37m-----[0m [1m3s[0m 10ms/step -
accuracy: 0.8301 - auc: 0.9681 - loss: 0.3781 - precision: 0.3750 -
recall: 0.9623
[1m 25/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9666 - loss: 0.3791 - precision: 0.3876
- recall: 0.9655
[1m 49/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8388 - auc: 0.9649 - loss: 0.3867 - precision: 0.3883
- recall: 0.9638
[1m 74/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8383 - auc: 0.9641 - loss: 0.3905 - precision: 0.3892
- recall: 0.9637
[1m 98/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8380 - auc: 0.9636 - loss: 0.3932 - precision: 0.3904
- recall: 0.9636
[1m123/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8381 - auc: 0.9634 - loss: 0.3944 - precision: 0.3916
- recall: 0.9638
[1m148/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8383 - auc: 0.9633 - loss: 0.3950 - precision: 0.3925
- recall: 0.9640
[1m173/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8385 - auc: 0.9632 - loss: 0.3954 - precision: 0.3934
- recall: 0.9642
[1m198/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9631 - loss: 0.3957 - precision: 0.3940
- recall: 0.9642
[1m223/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8388 - auc: 0.9631 - loss: 0.3959 - precision: 0.3943
- recall: 0.9642
[1m247/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8389 - auc: 0.9630 - loss: 0.3961 - precision: 0.3946
- recall: 0.9642
[1m271/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9630 - loss: 0.3961 - precision: 0.3948
- recall: 0.9643
[1m296/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9630 - loss: 0.3962 - precision: 0.3949
- recall: 0.9645
[1m314/314[0m [32m-----[0m [37m-----[0m [1m1s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9627 - loss: 0.3978 - precision: 0.3965
- recall: 0.9653 - val_accuracy: 0.8381 - val_auc: 0.9635 -
val_loss: 0.2506 - val_precision: 0.3945 - val_recall: 0.9675 -
learning_rate: 3.1250e-05
Epoch 36/40
[1m 1/314[0m [37m-----[0m [1m3s[0m 10ms/step -
accuracy: 0.8281 - auc: 0.9723 - loss: 0.3615 - precision: 0.3723 -
recall: 0.9623
[1m 25/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9686 - loss: 0.3682 - precision: 0.3882
- recall: 0.9646
[1m 50/314[0m [32m-----[0m [37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9666 - loss: 0.3774 - precision: 0.3889
- recall: 0.9651

```

```

[1m 75/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8385 - auc: 0.9656 - loss: 0.3830 - precision: 0.3898
- recall: 0.9649
[1m100/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8383 - auc: 0.9650 - loss: 0.3867 - precision: 0.3910
- recall: 0.9647
[1m125/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8383 - auc: 0.9646 - loss: 0.3886 - precision: 0.3920
- recall: 0.9648
[1m150/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8385 - auc: 0.9644 - loss: 0.3898 - precision: 0.3929
- recall: 0.9649
[1m175/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9642 - loss: 0.3909 - precision: 0.3938
- recall: 0.9650
[1m200/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8388 - auc: 0.9641 - loss: 0.3915 - precision: 0.3944
- recall: 0.9651
[1m224/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8389 - auc: 0.9640 - loss: 0.3920 - precision: 0.3946
- recall: 0.9651
[1m248/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9639 - loss: 0.3924 - precision: 0.3949
- recall: 0.9652
[1m273/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9638 - loss: 0.3926 - precision: 0.3950
- recall: 0.9653
[1m298/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9638 - loss: 0.3929 - precision: 0.3951
- recall: 0.9654

```

Epoch 36: ReduceLROnPlateau reducing learning rate to 1.5625000742147677e-05.

```

[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9630 - loss: 0.3970 - precision: 0.3967
- recall: 0.9657 - val_accuracy: 0.8379 - val_auc: 0.9636 -
val_loss: 0.2508 - val_precision: 0.3941 - val_recall: 0.9684 -
learning_rate: 3.1250e-05

```

Epoch 37/40

```

[1m 1/314[0m [37m[0m [1m3s[0m 10ms/step -
accuracy: 0.8359 - auc: 0.9683 - loss: 0.3777 - precision: 0.3835 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8386 - auc: 0.9677 - loss: 0.3713 - precision: 0.3862
- recall: 0.9656
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8380 - auc: 0.9658 - loss: 0.3796 - precision: 0.3872
- recall: 0.9651
[1m 75/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8375 - auc: 0.9649 - loss: 0.3845 - precision: 0.3883
- recall: 0.9652
[1m100/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8374 - auc: 0.9645 - loss: 0.3876 - precision: 0.3897
- recall: 0.9655

```



```

[1m125/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8378 - auc: 0.9643 - loss: 0.3889 - precision: 0.3912
- recall: 0.9659
[1m150/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8381 - auc: 0.9642 - loss: 0.3898 - precision: 0.3925
- recall: 0.9662
[1m175/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8384 - auc: 0.9641 - loss: 0.3906 - precision: 0.3935
- recall: 0.9662
[1m200/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9640 - loss: 0.3911 - precision: 0.3943
- recall: 0.9662
[1m225/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8388 - auc: 0.9639 - loss: 0.3916 - precision: 0.3946
- recall: 0.9661
[1m250/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9638 - loss: 0.3919 - precision: 0.3949
- recall: 0.9661
[1m275/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9638 - loss: 0.3921 - precision: 0.3951
- recall: 0.9661
[1m300/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9637 - loss: 0.3924 - precision: 0.3953
- recall: 0.9661
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8404 - auc: 0.9631 - loss: 0.3959 - precision: 0.3978
- recall: 0.9665 - val_accuracy: 0.8390 - val_auc: 0.9635 -
val_loss: 0.2499 - val_precision: 0.3956 - val_recall: 0.9651 -
learning_rate: 1.5625e-05
Epoch 38/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8262 - auc: 0.9672 - loss: 0.3824 - precision: 0.3696 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8408 - auc: 0.9679 - loss: 0.3697 - precision: 0.3895
- recall: 0.9644
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8404 - auc: 0.9660 - loss: 0.3786 - precision: 0.3906
- recall: 0.9632
[1m 75/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9651 - loss: 0.3840 - precision: 0.3915
- recall: 0.9628
[1m100/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8394 - auc: 0.9646 - loss: 0.3876 - precision: 0.3925
- recall: 0.9626
[1m125/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9643 - loss: 0.3893 - precision: 0.3936
- recall: 0.9629
[1m150/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9641 - loss: 0.3905 - precision: 0.3945
- recall: 0.9631
[1m175/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9640 - loss: 0.3914 - precision: 0.3954
- recall: 0.9631

```



```
[1m200/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9638 - loss: 0.3921 - precision: 0.3960
- recall: 0.9630
[1m225/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8402 - auc: 0.9637 - loss: 0.3927 - precision: 0.3964
- recall: 0.9629
[1m250/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8403 - auc: 0.9637 - loss: 0.3931 - precision: 0.3967
- recall: 0.9629
[1m275/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8404 - auc: 0.9636 - loss: 0.3933 - precision: 0.3969
- recall: 0.9629
[1m300/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8405 - auc: 0.9636 - loss: 0.3937 - precision: 0.3970
- recall: 0.9630
[1m314/314[0m [32m[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8415 - auc: 0.9628 - loss: 0.3979 - precision: 0.3993
- recall: 0.9632 - val_accuracy: 0.8396 - val_auc: 0.9635 -
val_loss: 0.2499 - val_precision: 0.3965 - val_recall: 0.9640 -
learning_rate: 1.5625e-05
Epoch 39/40
[1m 1/314[0m [37m[0m [1m2s[0m 9ms/step -
accuracy: 0.8301 - auc: 0.9672 - loss: 0.3831 - precision: 0.3750 -
recall: 0.9623
[1m 25/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9674 - loss: 0.3732 - precision: 0.3882
- recall: 0.9684
[1m 50/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9658 - loss: 0.3807 - precision: 0.3895
- recall: 0.9654
[1m 74/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9650 - loss: 0.3849 - precision: 0.3906
- recall: 0.9642
[1m 99/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9645 - loss: 0.3881 - precision: 0.3919
- recall: 0.9637
[1m123/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9642 - loss: 0.3899 - precision: 0.3929
- recall: 0.9635
[1m147/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8391 - auc: 0.9640 - loss: 0.3910 - precision: 0.3937
- recall: 0.9635
[1m171/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8393 - auc: 0.9638 - loss: 0.3919 - precision: 0.3945
- recall: 0.9635
[1m196/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9637 - loss: 0.3924 - precision: 0.3952
- recall: 0.9636
[1m220/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9637 - loss: 0.3928 - precision: 0.3955
- recall: 0.9637
[1m244/314[0m [32m[0m[37m[0m [1m0s[0m 2ms/step
- accuracy: 0.8397 - auc: 0.9636 - loss: 0.3931 - precision: 0.3958
- recall: 0.9638
```

```

[1m269/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9636 - loss: 0.3933 - precision: 0.3960
- recall: 0.9640
[1m293/314[0m [32m-----[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8399 - auc: 0.9635 - loss: 0.3934 - precision: 0.3961
- recall: 0.9641
[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step
- accuracy: 0.8406 - auc: 0.9630 - loss: 0.3965 - precision: 0.3981
- recall: 0.9653 - val_accuracy: 0.8388 - val_auc: 0.9636 -
val_loss: 0.2500 - val_precision: 0.3953 - val_recall: 0.9656 -
learning_rate: 1.5625e-05
Epoch 40/40
[1m 1/314[0m [37m-----[0m [1m2s[0m 10ms/step -
accuracy: 0.8281 - auc: 0.9697 - loss: 0.3683 - precision: 0.3723 -
recall: 0.9623
[1m 25/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9683 - loss: 0.3697 - precision: 0.3873
- recall: 0.9667
[1m 49/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9663 - loss: 0.3790 - precision: 0.3889
- recall: 0.9653
[1m 74/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8388 - auc: 0.9654 - loss: 0.3849 - precision: 0.3901
- recall: 0.9651
[1m 99/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8385 - auc: 0.9648 - loss: 0.3888 - precision: 0.3914
- recall: 0.9649
[1m124/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8387 - auc: 0.9644 - loss: 0.3908 - precision: 0.3926
- recall: 0.9651
[1m149/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8390 - auc: 0.9642 - loss: 0.3921 - precision: 0.3937
- recall: 0.9653
[1m174/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8392 - auc: 0.9641 - loss: 0.3931 - precision: 0.3947
- recall: 0.9655
[1m199/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8395 - auc: 0.9640 - loss: 0.3938 - precision: 0.3954
- recall: 0.9655
[1m224/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8396 - auc: 0.9639 - loss: 0.3943 - precision: 0.3958
- recall: 0.9656
[1m249/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8398 - auc: 0.9638 - loss: 0.3946 - precision: 0.3961
- recall: 0.9656
[1m274/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8399 - auc: 0.9637 - loss: 0.3948 - precision: 0.3963
- recall: 0.9656
[1m299/314[0m [32m-[0m[37m-----[0m [1m0s[0m 2ms/step
- accuracy: 0.8400 - auc: 0.9637 - loss: 0.3950 - precision: 0.3964
- recall: 0.9656

```

Epoch 40: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.

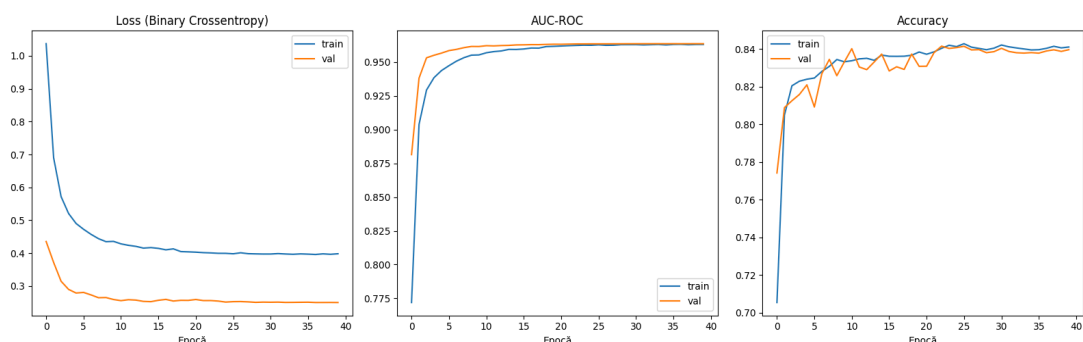
```
[1m314/314[0m [32m-----[0m[37m[0m [1m1s[0m 2ms/step  
- accuracy: 0.8411 - auc: 0.9630 - loss: 0.3982 - precision: 0.3989  
- recall: 0.9655 - val_accuracy: 0.8397 - val_auc: 0.9635 -  
val_loss: 0.2498 - val_precision: 0.3965 - val_recall: 0.9640 -  
learning_rate: 1.5625e-05  
Restoring model weights from the end of the best epoch: 39.
```

Antrenare finalizată după 40 epoci.

Curbele de antrenare — diagnoza vizuală a antrenării

Urmărim loss/AUC/accuracy pe train vs validare, epocă cu epocă. Curbele apropiate = model sănătos; val-loss care crește în timp ce train-loss scade = overfitting (moment în care EarlyStopping oprește antrenarea și restaurează cele mai bune ponderi).

```
# Curbe de antrenare  
fig, axes = plt.subplots(1, 3, figsize=(16, 5))  
  
axes[0].plot(history.history['loss'], label='train')  
axes[0].plot(history.history['val_loss'], label='val')  
axes[0].set_title('Loss (Binary Crossentropy)')  
axes[0].set_xlabel('Epocă')  
axes[0].legend()  
  
axes[1].plot(history.history['auc'], label='train')  
axes[1].plot(history.history['val_auc'], label='val')  
axes[1].set_title('AUC-ROC')  
axes[1].set_xlabel('Epocă')  
axes[1].legend()  
  
axes[2].plot(history.history['accuracy'], label='train')  
axes[2].plot(history.history['val_accuracy'], label='val')  
axes[2].set_title('Accuracy')  
axes[2].set_xlabel('Epocă')  
axes[2].legend()  
  
plt.tight_layout()  
plt.savefig('plots/fig_tf_training_curves.png', dpi=120,  
bbox_inches='tight')  
plt.show()
```



<Figure size 1600x500 with 3 Axes>

6.4 Evaluarea modelului pe setul de test

```
# Predicții pe test set
y_prob = model.predict(X_test, verbose=0).ravel()
y_pred = (y_prob >= 0.5).astype(int)

test_loss, test_acc, test_auc, test_prec, test_rec =
model.evaluate(X_test, y_test, verbose=0)
test_f1 = f1_score(y_test, y_pred, average='weighted')

print('=== Metrici TensorFlow Neural Network (Test Set) ===')
print(f' Loss (BCE) : {test_loss:.4f}')
print(f' AUC-ROC    : {test_auc:.4f}')
print(f' Accuracy   : {test_acc:.4f}')
print(f' Precision  : {test_prec:.4f}')
print(f' Recall     : {test_rec:.4f}')
print(f' F1-Score   : {test_f1:.4f}')
print()
print(classification_report(y_test, y_pred, target_names=['Nu gol',
'Gol']))
=== Metrici TensorFlow Neural Network (Test Set) ===
 Loss (BCE) : 0.2477
 AUC-ROC    : 0.9640
 Accuracy   : 0.8417
 Precision  : 0.3994
 Recall     : 0.9621
 F1-Score   : 0.8671
```

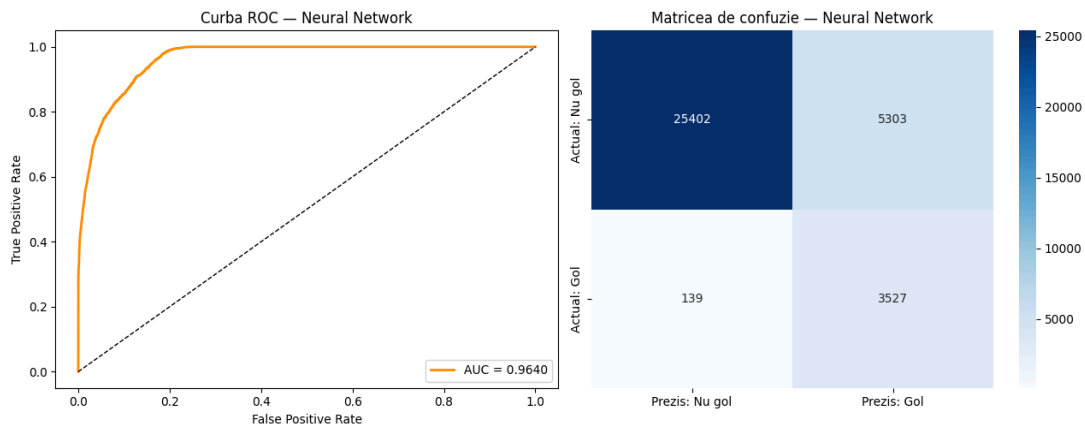
	precision	recall	f1-score	support
Nu gol	0.99	0.83	0.90	30705
Gol	0.40	0.96	0.56	3666
accuracy			0.84	34371
macro avg	0.70	0.89	0.73	34371
weighted avg	0.93	0.84	0.87	34371

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

# Curba ROC
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_val = roc_auc_score(y_test, y_prob)
axes[0].plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC =
{auc_val:.4f}')
axes[0].plot([0,1],[0,1], 'k--', lw=1)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('Curba ROC - Neural Network')
axes[0].legend()
```

```
# Matricea de confuzie
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Prezis: Nu gol', 'Prezis: Gol'],
            yticklabels=['Actual: Nu gol', 'Actual: Gol'],
            ax=axes[1])
axes[1].set_title('Matricea de confuzie – Neural Network')

plt.tight_layout()
plt.savefig('plots/fig_tf_roc_cm.png', dpi=120, bbox_inches='tight')
plt.show()
```



<Figure size 1300x500 with 3 Axes>

Diagnoză pe clase: distribuția probabilităților + Precision-Recall

De ce aceste grafice: Pe un dataset dezechilibrat (~10-12% goluri), **AUC-ROC și accuracy pot fi înșelător de optimiste** — un model care prezice mereu „nu e gol” obține deja ~90% accuracy. Folosim două instrumente mai oneste: distribuția probabilităților prezise pe clase și curba Precision-Recall.

```
from sklearn.metrics import precision_recall_curve,
average_precision_score

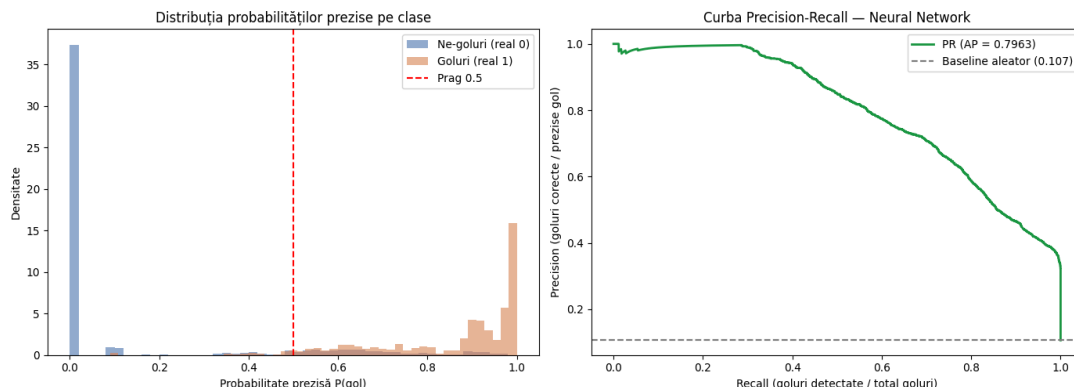
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# (1) Distribuția probabilităților prezise, separat pe cele două
clase reale
axes[0].hist(y_prob[y_test == 0], bins=50, alpha=0.6, density=True,
            label='Ne-goluri (real 0)', color='#4c72b0')
axes[0].hist(y_prob[y_test == 1], bins=50, alpha=0.6, density=True,
            label='Goluri (real 1)', color='#dd8452')
axes[0].axvline(0.5, color='red', linestyle='--', label='Prag 0.5')
axes[0].set_xlabel('Probabilitate prezisă P(gol)')
axes[0].set_ylabel('Densitate')
axes[0].set_title('Distribuția probabilităților prezise pe clase')
axes[0].legend()
```

```
# (2) Curba Precision-Recall (mai informativă decât ROC pe date
dezechilibrate)
prec_curve, rec_curve, _ = precision_recall_curve(y_test, y_prob)
ap = average_precision_score(y_test, y_prob)
baseline = y_test.mean() # precision a unui clasificator aleator =
proportia pozitivelor
axes[1].plot(rec_curve, prec_curve, color='#1a9641', lw=2,
label=f'PR (AP = {ap:.4f})')
axes[1].axhline(baseline, color='gray', linestyle='--',
label=f'Baseline aleator ({baseline:.3f})')
axes[1].set_xlabel('Recall (goluri detectate / total goluri)')
axes[1].set_ylabel('Precision (goluri corecte / prezise gol)')
axes[1].set_title('Curba Precision-Recall – Neural Network')
axes[1].legend()

plt.tight_layout()
plt.savefig('plots/fig_tf_prob_dist_pr_curve.png', dpi=120,
bbox_inches='tight')
plt.show()

print(f'Average Precision (AP): {ap:.4f} | baseline aleator:
{baseline:.4f}')
```



<Figure size 1400x500 with 2 Axes>
Average Precision (AP): 0.7963 | baseline aleator: 0.1067

6.5 Compararea Neural Network vs Random Forest (MLlib)

Comparăm metricile obținute de cele două abordări pe același task de clasificare.

```
import joblib

# Valorile RF măsurate în Cerința 3 (rularea finală, test set,
seed=42)
comparison = pd.DataFrame({
    'Metrică': ['AUC-ROC', 'Accuracy', 'F1-Score (wgt)'],
```

```

    'Random Forest (MLlib)': [0.9545, 0.9298, 0.9159],
    'Neural Network (TF)': [test_auc, test_acc, test_f1]
})

print(comparison.to_string(index=False))

# Salvăm artefactele necesare pentru inferență
os.makedirs('models', exist_ok=True)
model.save('models/tf_goal_predictor.keras')
# Scaler-ul TREBUIE salvat împreună cu modelul: la inferență, datele
noi trebuie
# normalizate cu EXACT aceiași parametri (medie/deviație standard)
învățați pe train.
# Altfel modelul primește input pe altă scară decât cea pe care a
fost antrenat.
joblib.dump scaler, 'models/tf_scaler.joblib')

print('\nModel TensorFlow salvat în :
models/tf_goal_predictor.keras')
print('Scaler salvat în          : models/tf_scaler.joblib')
    Metrică Random Forest (MLlib) Neural Network (TF)
    AUC-ROC                0.9545          0.963996
    Accuracy                0.9298          0.841669
    F1-Score (wgt)          0.9159          0.867116

Model TensorFlow salvat în : models/tf_goal_predictor.keras
Scaler salvat în          : models/tf_scaler.joblib
metrics = comparison['Metrică'].tolist()
rf_vals = comparison['Random Forest (MLlib)'].tolist()
nn_vals = comparison['Neural Network (TF)'].tolist()

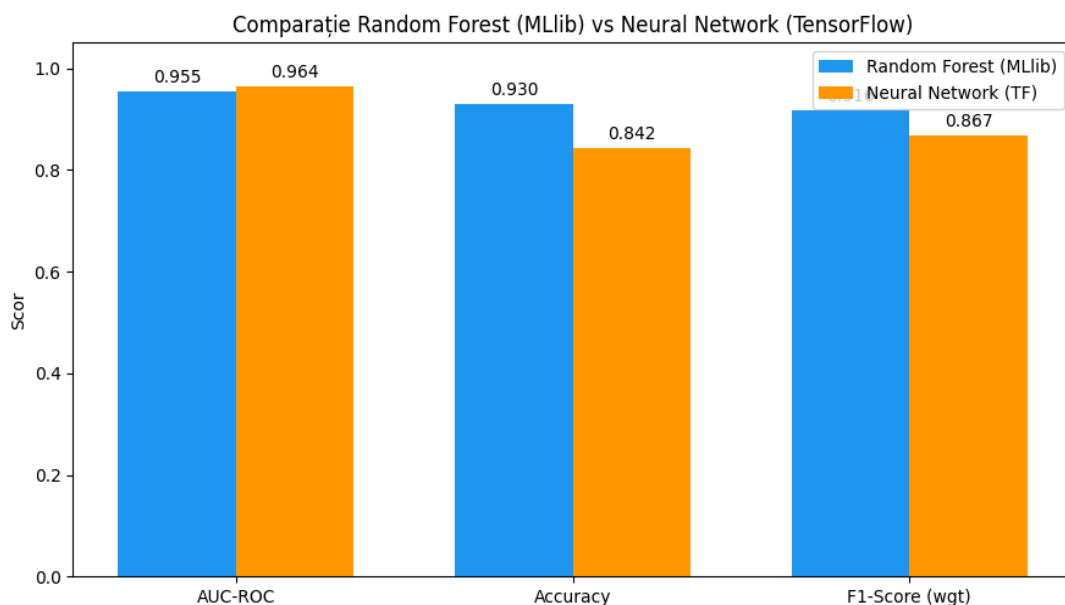
x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(9, 5))
bars1 = ax.bar(x - width/2, rf_vals, width, label='Random Forest
(MLlib)', color='#2196F3')
bars2 = ax.bar(x + width/2, nn_vals, width, label='Neural Network
(TF)', color='#FF9800')

ax.bar_label(bars1, fmt='{:.3f}', padding=3)
ax.bar_label(bars2, fmt='{:.3f}', padding=3)
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.set_ylim(0, 1.05)
ax.set_ylabel('Scor')
ax.set_title('Comparație Random Forest (MLlib) vs Neural Network
(TensorFlow)')
ax.legend()

plt.tight_layout()
plt.savefig('plots/fig_model_comparison.png', dpi=120,
bbox_inches='tight')
plt.show()

```



<Figure size 900x500 with 1 Axes>

Cerința 7 — Spark Structured Streaming + Inferență în Timp Real

Scenariu

Simulăm un **flux de date live de la un meci de fotbal**: evenimentele (tentative de șut) sosesc în timp real și sistemul nostru aplică **modelul ML antrenat** (pipeline-ul de la Cerința 4) pentru a prezice probabilitatea de gol pentru fiecare șut, în timp ce produce statistici agregate în ferestre de timp.

Arhitectura pipeline-ului de streaming

```

Fișiere CSV simulate → Spark readStream → Preprocesare →
ML inference          (trigger 5s)         (fillna, cast)
(meci live events)
(pipeline model)

↓

writeStream
(console output)
  
```


Abordarea: Structured Streaming din fișiere

Simulăm stream-ul prin scrierea treptată a fișierelor CSV într-un director de intrare — Spark monitorizează directorul și procesează fișierele noi ca micro-batch-uri.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, count, sum as spark_sum, avg, round as spark_round,
    current_timestamp, window, when, lit
)
from pyspark.sql.types import (
    StructType, StructField, IntegerType, DoubleType, StringType
)
from pyspark.ml import PipelineModel
import pandas as pd
import numpy as np
import time
import os
import shutil
import warnings
warnings.filterwarnings('ignore')

spark = SparkSession.builder \
    .appName('FootballStreaming') \
    .config('spark.driver.memory', '4g') \
    .config('spark.sql.shuffle.partitions', '4') \
    .config('spark.sql.streaming.schemaInference', 'true') \
    .getOrCreate()

spark.sparkContext.setLogLevel('ERROR')
print('Spark version:', spark.version)
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-
defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,
use setLogLevel(newLevel).
26/06/11 20:08:55 WARN NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
Spark version: 4.1.2
```

7.1 Generarea datelor simulate (stream de intrare)

Împărțim dataset-ul real în "batch-uri" care simulează evenimentele unui meci ce sosesc pe rând — câte unul la fiecare câteva secunde.

```
# Directoare pentru stream
STREAM_INPUT_DIR = 'data/stream_input'
STREAM_OUTPUT_DIR = 'data/stream_output'
CHECKPOINT_DIR = 'data/stream_checkpoint'
```

```

for d in [STREAM_INPUT_DIR, STREAM_OUTPUT_DIR, CHECKPOINT_DIR]:
    if os.path.exists(d):
        shutil.rmtree(d)
    os.makedirs(d)

# Citim câteva sute de șuturi reale ca bază pentru simulare
events_real = spark.read.csv('data/events.csv', header=True,
inferSchema=True, nullValue='NA') \
    .filter(
        (col('event_type') == 1) &
        col('is_goal').isNotNull() &
        col('location').isNotNull() &
        col('bodypart').isNotNull() &
        col('situation').isNotNull()
    ) \
    .fillna({'assist_method': 0, 'fast_break': 0, 'shot_place': 0,
'side': 1}) \
    .select('id_odsp', 'time', 'event_team', 'opponent',
        'is_goal', 'location', 'bodypart', 'situation',
        'assist_method', 'fast_break', 'shot_place', 'side') \
    .limit(500)

# Convertim în pandas pentru a scrie batch-uri simulate
events_pd = events_real.toPandas()
print(f'Date simulate: {len(events_pd)} tentative de șut')

# Împărțim în 5 batch-uri de ~100 rânduri fiecare
BATCH_SIZE = 100
batches = [events_pd.iloc[i:i+BATCH_SIZE] for i in range(0,
len(events_pd), BATCH_SIZE)]
print(f'Batches pregătite: {len(batches)} × ~{BATCH_SIZE} rânduri')
Date simulate: 500 tentative de șut
Batches pregătite: 5 × ~100 rânduri

```

7.2 Definirea schemei și configurarea stream-ului de intrare

Definim explicit schema pentru a nu pierde timp la inferare și pentru a asigura consistența tipurilor.

```

stream_schema = StructType([
    StructField('id_odsp',      StringType(), True),
    StructField('time',         DoubleType(), True),
    StructField('event_team',   StringType(), True),
    StructField('opponent',     StringType(), True),
    StructField('is_goal',      IntegerType(), True),
    StructField('location',     IntegerType(), True),
    StructField('bodypart',     IntegerType(), True),
    StructField('situation',    IntegerType(), True),
    StructField('assist_method', IntegerType(), True),

```

```

        StructField('fast_break', IntegerType(), True),
        StructField('shot_place', IntegerType(), True),
        StructField('side', IntegerType(), True)
    ])

# Scriem primul batch pentru a inițializa stream-ul
batch0_path = os.path.join(STREAM_INPUT_DIR, 'batch_0.csv')
batches[0].to_csv(batch0_path, index=False)
print(f'Batch 0 scris: {batch0_path} ({len(batches[0])} rânduri)')

# Citim stream-ul din director
stream_df = spark.readStream \
    .schema(stream_schema) \
    .option('header', 'true') \
    .option('maxFilesPerTrigger', 1) \
    .csv(STREAM_INPUT_DIR)

print(f'Stream isStreaming: {stream_df.isStreaming}')
Batch 0 scris: data/stream_input/batch_0.csv (100 rânduri)
Stream isStreaming: True

```

7.3 Aplicarea modelului ML în timp real

Aplicăm **pipeline-ul antrenat** (de la Cerința 4) pe fiecare micro-batch de date primite în stream.

```

# Încărcăm pipeline-ul ML antrenat anterior (Cerința 4)
# Dacă nu există, antrenăm un model simplu
pipeline_path = 'models/rf_pipeline_model'

if os.path.exists(pipeline_path):
    pipeline_model = PipelineModel.load(pipeline_path)
    print(f'Pipeline MLlib încărcat din: {pipeline_path}')

    # Aplicăm modelul pe stream
    # NOTĂ: VectorAssembler necesită coloanele corecte
    # Pipeline-ul din cerința 4 folosea și coloane inginerite
    (is_central_shot etc.)
    # Facem o preprocesare minimă compatibilă
    from pyspark.sql.functions import floor
    stream_enriched = stream_df \
        .withColumn('is_central_shot',
                    when(col('location').isin(3, 5, 10, 12, 13, 14),
1).otherwise(0)) \
        .withColumn('time_phase',
                    when(col('time') <= 45, 1).when(col('time') <=
70, 2).otherwise(3)) \
        .withColumn('is_set_piece',
                    when(col('situation').isin(2, 3, 4),
1).otherwise(0))

    stream_with_predictions =

```

```

pipeline_model.transform(stream_enriched)

    use_ml_predictions = True
    print('Inferență ML activată pe stream.')
else:
    # Fallback: rulăm stream-ul fără inferență ML
    stream_with_predictions = stream_df
    use_ml_predictions = False
    print('ATENȚIE: Pipeline ML negăsit. Rulăm stream-ul fără
inferență ML.')
    print('Rulați mai întâi Cerința 4 pentru a genera modelul.')
Pipeline MLlib încărcat din: models/rf_pipeline_model
Inferență ML activată pe stream.

```

7.4 Query 1 — Statistici în timp real pe echipă

Calculăm per fiecare echipă: numărul de șuturi, goluri marcate și rata de conversie curentă.

```

# Statistici agregate pe echipă
team_stats_stream = stream_df \
    .groupBy('event_team') \
    .agg(
        count('*').alias('suturi'),
        spark_sum('is_goal').alias('goluri')
    ) \
    .withColumn(
        'rata_conversie_pct',
        spark_round(col('goluri') * 100.0 / col('suturi'), 1)
    ) \
    .orderBy('goluri', ascending=False)

# Query Streaming cu outputMode 'complete' (pentru agregări)
query_team_stats = team_stats_stream.writeStream \
    .outputMode('complete') \
    .format('console') \
    .option('truncate', False) \
    .option('numRows', 10) \
    .trigger(processingTime='5 seconds') \
    .start()

print('Query 1 pornit: statistici per echipă (outputMode=complete)')
print('Adăugăm batch-uri noi la fiecare 6 secunde...')

# Simulăm sosirea datelor noi
for i, batch in enumerate(batches[1:], start=1):
    time.sleep(6)
    batch_path = os.path.join(STREAM_INPUT_DIR, f'batch_{i}.csv')
    batch.to_csv(batch_path, index=False)
    print(f' Batch {i} adăugat: {len(batch)} rânduri noi')

# Așteptăm ultimul trigger

```

```
time.sleep(8)
query_team_stats.stop()
print('Query 1 oprit.')
Query 1 pornit: statistici per echipă (outputMode=complete)
Adăugăm batch-uri noi la fiecare 6 secunde...
```

Batch: 0

event_team	suturi	goluri	rata_conversie_pct
Borussia Dortmund	16	3	18.8
SC Freiburg	9	2	22.2
Werder Bremen	19	2	10.5
FC Augsburg	11	2	18.2
Lorient	9	1	11.1
Hamburg SV	8	1	12.5
Valenciennes	4	0	0.0
Paris Saint-Germain	12	0	0.0
Caen	3	0	0.0
Kaiserslautern	9	0	0.0

Batch 1 adăugat: 100 rânduri noi

Batch: 1

event_team	suturi	goluri	rata_conversie_pct
Borussia Dortmund	16	3	18.8
SC Freiburg	9	2	22.2
Werder Bremen	19	2	10.5
Evian Thonon Gaillard	9	2	22.2
Brest	23	2	8.7
Toulouse	13	2	15.4
FC Augsburg	11	2	18.2
Nice	3	1	33.3
Nurnberg	13	1	7.7
Caen	11	1	9.1

only showing top 10 rows

Batch 2 adăugat: 100 rânduri noi

Batch: 2

event_team	suturi	goluri	rata_conversie_pct
Borussia Dortmund	16	3	18.8
Lyon	22	3	13.6
VfL Wolfsburg	18	3	16.7
Werder Bremen	19	2	10.5
SC Freiburg	9	2	22.2

Evian Thonon Gaillard	9	2	22.2	
Brest	23	2	8.7	
Toulouse	13	2	15.4	
FC Augsburg	11	2	18.2	
Nurnberg	13	1	7.7	

only showing top 10 rows

Batch 3 adăugat: 100 rânduri noi

Batch: 3

event_team	suturi	goluri	rata_conversie_pct	
Borussia Dortmund	16	3	18.8	
Lyon	22	3	13.6	
VfB Stuttgart	12	3	25.0	
VfL Wolfsburg	18	3	16.7	
Montpellier	13	3	23.1	
Werder Bremen	19	2	10.5	
SC Freiburg	9	2	22.2	
Evian Thonon Gaillard	9	2	22.2	
Brest	23	2	8.7	
Sochaux	8	2	25.0	

only showing top 10 rows

Batch 4 adăugat: 100 rânduri noi

Batch: 4

event_team	suturi	goluri	rata_conversie_pct	
Stade Rennes	16	5	31.3	
Borussia Dortmund	16	3	18.8	
Lyon	22	3	13.6	
VfB Stuttgart	12	3	25.0	
VfL Wolfsburg	18	3	16.7	
Montpellier	13	3	23.1	
Mainz	23	2	8.7	
Werder Bremen	19	2	10.5	
SC Freiburg	9	2	22.2	
Evian Thonon Gaillard	9	2	22.2	

only showing top 10 rows

Query 1 oprit.

7.5 Query 2 — Predicții individuale pe șuturi

Dacă modelul ML este disponibil, afișăm pentru fiecare șut nou: echipa, minutul, și probabilitatea predicției.

```

# Resetăm directorul de intrare
shutil.rmtree(STREAM_INPUT_DIR)
os.makedirs(STREAM_INPUT_DIR)
shutil.rmtree(CHECKPOINT_DIR)
os.makedirs(CHECKPOINT_DIR)

# Re-citim stream-ul
stream_df2 = spark.readStream \
    .schema(stream_schema) \
    .option('header', 'true') \
    .option('maxFilesPerTrigger', 1) \
    .csv(STREAM_INPUT_DIR)

if use_ml_predictions:
    stream_enriched2 = stream_df2 \
        .withColumn('is_central_shot',
                    when(col('location').isin(3, 5, 10, 12, 13, 14),
1).otherwise(0)) \
        .withColumn('time_phase',
                    when(col('time') <= 45, 1).when(col('time') <=
70, 2).otherwise(3)) \
        .withColumn('is_set_piece',
                    when(col('situation').isin(2, 3, 4),
1).otherwise(0))

    predictions_stream = pipeline_model.transform(stream_enriched2)

    from pyspark.sql.functions import round as spark_round2
    output_stream = predictions_stream.select(
        'event_team',
        col('time').cast('int').alias('minut'),
        'location',
        'is_goal',
        'prediction',
    )
else:
    # Fără ML – afișăm datele brute
    output_stream = stream_df2.select(
        'event_team',
        col('time').cast('int').alias('minut'),
        'location',
        'bodypart',
        'is_goal'
    )

# Query cu outputMode 'append'
query_predictions = output_stream.writeStream \
    .outputMode('append') \
    .format('console') \
    .option('truncate', False) \
    .option('numRows', 20) \
    .trigger(processingTime='5 seconds') \
    .option('checkpointLocation', CHECKPOINT_DIR) \
    .start()

```

```

print('Query 2 pornit: predicții individuale pe șuturi
(outputMode=append)')

# Adăugăm primele 3 batch-uri
for i in range(3):
    batches[i].to_csv(os.path.join(STREAM_INPUT_DIR,
f'batch_{i}.csv'), index=False)
    print(f' Batch {i} adăugat')
    time.sleep(7)

time.sleep(5)
query_predictions.stop()
print('Query 2 oprit.')
Query 2 pornit: predicții individuale pe șuturi (outputMode=append)
Batch 0 adăugat

```

Batch: 0

event_team	minut	location	is_goal	prediction
Hamburg SV	2	9	0	0.0
Borussia Dortmund	14	15	0	0.0
Borussia Dortmund	17	9	1	0.0
Borussia Dortmund	19	15	0	0.0
Hamburg SV	20	15	0	0.0
Borussia Dortmund	25	3	0	0.0
Borussia Dortmund	25	15	0	0.0
Borussia Dortmund	26	3	0	0.0
Borussia Dortmund	28	9	0	0.0
Borussia Dortmund	29	3	1	0.0
Borussia Dortmund	36	11	0	0.0
Hamburg SV	38	15	0	0.0
Borussia Dortmund	39	9	0	0.0
Borussia Dortmund	45	11	0	0.0
Borussia Dortmund	48	3	0	0.0
Borussia Dortmund	49	13	1	1.0
Borussia Dortmund	54	15	0	0.0
Hamburg SV	62	16	0	0.0
Hamburg SV	64	3	0	0.0
Hamburg SV	79	3	0	0.0

only showing top 20 rows
Batch 1 adăugat

Batch: 1

event_team	minut	location	is_goal	prediction
Caen	33	15	0	0.0
Caen	33	3	0	0.0
Caen	34	15	1	0.0

Valenciennes	37	3	0	0.0	
Caen	40	15	0	0.0	
Valenciennes	43	15	0	0.0	
Valenciennes	43	12	0	0.0	
Valenciennes	45	3	0	0.0	
Valenciennes	48	15	0	0.0	
Caen	52	15	0	0.0	
Caen	54	3	0	0.0	
Valenciennes	67	9	0	0.0	
Caen	87	15	0	0.0	
Valenciennes	88	11	0	0.0	
Valenciennes	88	3	0	0.0	
Caen	90	15	0	0.0	
Nurnberg	8	3	0	0.0	
Hertha Berlin	10	15	0	0.0	
Nurnberg	10	15	0	0.0	
Nurnberg	22	17	0	0.0	
+-----+-----+-----+-----+-----+					

only showing top 20 rows

Batch 2 adäugat

Batch: 2

event_team	minut	location	is_goal	prediction	
Nice	13	9	0	0.0	
Nice	13	15	0	0.0	
Lyon	14	15	0	0.0	
Nice	15	17	0	0.0	
Lyon	16	3	0	0.0	
Nice	23	3	0	0.0	
Lyon	25	6	0	0.0	
Nice	27	15	0	0.0	
Lyon	29	17	0	0.0	
Lyon	31	15	0	0.0	
Lyon	32	15	0	0.0	
Lyon	32	13	0	0.0	
Lyon	33	13	1	1.0	
Lyon	36	15	0	0.0	
Lyon	36	9	0	0.0	
Lyon	36	3	0	0.0	
Nice	41	11	0	0.0	
Lyon	45	3	0	0.0	
Lyon	48	16	0	0.0	
Lyon	51	13	0	0.0	
+-----+-----+-----+-----+-----+					

only showing top 20 rows

Query 2 oprit.

7.6 Query 3 — Statistici finale agregate (batch mode)

Procesăm toate datele simulate ca un job batch final pentru a valida că stream-ul a prelucrat corect toate evenimentele.

```
# Scriem toate batch-urile simulate
all_batches_path = 'data/stream_all_batches.csv'
events_pd.to_csv(all_batches_path, index=False)

# Procesare batch finală pentru validare
final_df = spark.read.csv(all_batches_path, header=True,
                           schema=stream_schema)

print('=== Statistici finale după procesarea stream-ului ===')
final_df.createOrReplaceTempView('stream_final')

spark.sql("""
    SELECT
        event_team,
        COUNT(*)                AS total_suturi,
        SUM(is_goal)             AS goluri_marcate,
        ROUND(SUM(is_goal)*100.0/COUNT(*), 1) AS rata_conv_pct,
        ROUND(AVG(time), 1)      AS min_mediu_sut
    FROM stream_final
    GROUP BY event_team
    ORDER BY goluri_marcate DESC
    LIMIT 10
""").show(truncate=False)

# Curățare
for d in [STREAM_INPUT_DIR, STREAM_OUTPUT_DIR, CHECKPOINT_DIR]:
    if os.path.exists(d):
        shutil.rmtree(d)

spark.stop()
print('\nSesiune Spark oprită. Streaming demonstrat cu succes.')
=== Statistici finale după procesarea stream-ului ===
+-----+-----+-----+-----+-----+
+-----+
|event_team|total_suturi|goluri_marcate|rata_conv_pct|min_mediu_sut|
+-----+-----+-----+-----+-----+
+-----+
|Stade Rennes|16|5|31.3|38.7|
|
|Lyon|22|3|13.6|46.0|
|
|Borussia Dortmund|16|3|18.8|38.9|
|
|VfL Wolfsburg|18|3|16.7|48.8|
|
|VfB Stuttgart|12|3|25.0|52.8|
|
```

Montpellier	13	3	23.1	52.5
Toulouse	13	2	15.4	58.1
Brest	23	2	8.7	52.2
Mainz	23	2	8.7	46.7
Hannover 96	7	2	28.6	38.6
+-----+	+-----+	+-----+	+-----+	+-----+
-----+				

Sesiune Spark oprită. Streaming demonstrat cu succes.